

Gestaltung und faktorielle Evaluation modularer Laufzeitinterventionen für transaktionale LLM-Agenten im Kundenservice

Arbeit zur Erlangung des akademischen Grades Master of Science

im Studiengang Consulting & Business Analytics
an der Hochschule Reutlingen

Vorgelegt von:

Moritz von Angern

814919

Vorgelegt bei:

Prof. Dr. Tim Straub

Prof. Dr. Stephan Seiter

Reutlingen, 27.08.2026

Abstract

Transaktionale Agenten auf Basis großer Sprachmodelle (Large Language Models, LLM) können Kundenanliegen interpretieren und über externe Werkzeuge Geschäftsdaten verändern. Zusätzliche Laufzeitmechanismen können dabei Fehler begrenzen, aber auch korrekte Agentenpfade beeinträchtigen und den Ressourcenaufwand erhöhen. Die Arbeit untersucht, wie Komponente A (semantische Unterstützung) und Komponente B (Pre-Write-Kontrolle) einzeln und gemeinsam das Leistungsprofil transaktionaler LLM-Agenten im Kundenservice verändern.

In einem Design-Science-Vorgehen wird eine modular schaltbare Laufzeitinterventionsschicht gestaltet und technisch instanziiert. Komponente A stellt belegte Nutzerziele vor der Agentenentscheidung bereit. Komponente B prüft geplante Zustandsänderungen vor ihrer Ausführung. Beide werden in einem vollständigen 2×2 -Design unter gleicher Informations- und Bewertungsgrundlage evaluiert. Untersucht werden 80 Retail-Aufgaben in vier Bedingungen und sechs Wiederholungen, insgesamt 1.920 Zellen. Analysiert werden fachlicher Endzustand, exakte Mutationsausführung, Wiederholungsstabilität, Ergebniswechsel, Mechanismusspuren, Kosten und Laufzeit.

Die Bedingung C1 mit ausschließlich aktivierter Komponente A erzielt die höchsten beobachteten Mittelwerte. Gegenüber dem nativen Agenten steigt der Datenbankabgleich von 71,3 auf 77,3 Prozent und die exakte Mutationsausführung von 67,8 auf 74,1 Prozent. Die positive Richtung bleibt beim Auslassen einzelner Wiederholungen erhalten. Ein konservatives Unsicherheitsverfahren schließt einen Nulleffekt jedoch nicht aus. Die kombinierte Bedingung erreicht die höchste strenge Stabilität, aber keinen belastbaren additiven Vorteil. Komponente B schafft eine nachvollziehbare Kontrollstelle, zeigt allein jedoch keinen belastbaren Leistungsbeitrag. Alle Interventionen erzeugen Verbesserungen und Regressionen. Die Bedingungen C1 und C3 erhöhen zudem die Agenten-API-Kosten um rund 46 Prozent und die technische Laufzeit um rund 27 Prozent.

Die Ergebnisse zeigen, dass zusätzliche Laufzeitarchitektur nicht pauschal überlegen ist. Ihr Nutzen ist als kontextabhängiges Profil aus mittlerem Erfolg, Wiederholungsstabilität, Regressionen, Kontrollwirkung und Ressourcenaufwand zu bewerten. Der Beitrag der Arbeit liegt damit in einem modularen Artefakt sowie einem Evaluations- und Entscheidungsrahmen, der technische Funktionsfähigkeit von empirischem Nettonutzen trennt.

Schlagwörter: LLM-Agenten, Kundenservice, Laufzeitinterventionen, Design Science Research, faktorielle Evaluation, Wiederholungsstabilität

Inhaltsverzeichnis	
Tabellenverzeichnis	v
Abbildungsverzeichnis	vi
Listingverzeichnis	vii
Abkürzungsverzeichnis	viii
Symbolverzeichnis	ix
1 Einleitung	1
2 Theoretische Grundlagen	6
2.1 Large Language Models und toolnutzende Agenten	6
2.2 Transaktionaler Kundenservice als Geschäftsprozess	9
2.3 Hybride Orchestrierung und Laufzeitkontrolle	11
2.4 Evaluation agentischer Geschäftsprozesse	15
3 Verwandte Arbeiten	19
3.1 Methodisches Vorgehen der Literaturstudie	19
3.2 Konzeptzentrierte Synthese des Forschungsstands	24
3.3 Gegenarbeiten, Forschungslücke und Gestaltungsfolgen	29
4 Methodik	32
4.1 Forschungsdesign und Design-Science-Prozess	32
4.2 Testumgebung und Stichprobe	33
4.3 Experimentelles Design	38
4.4 Mess- und Analysevertrag	41
4.5 Güte, Reproduzierbarkeit und Aussagegrenzen	46
5 Anforderungsanalyse	48
5.1 Ableitungslogik und Gestaltungsziel	48
5.2 Gestaltungsanforderungen	50
5.3 Retail-Instanziierung und Nachvollziehbarkeit	53
6 Design und Entwicklung	54
6.1 Artefaktgrenze und Entwurfslogik	54
6.2 Architektur und Komponenten	56
6.3 Laufzeitverhalten	59
6.4 Technische Umsetzung und Designverifikation	62
7 Demonstration des Artefakts	65
7.1 Demonstrationsrahmen und Prozessübersicht	65
7.2 Komponenteninteraktion im Laufzeitpfad	67

8	Ergebnisse	70
8.1	Fachliche Wirkung	70
8.2	Stabilität und Ergebniswechsel	72
8.3	Wirkmechanismen und Kontrollwirkung	76
8.4	Ressourcenprofil	80
8.5	Zusammenfassung der empirischen Antworten	81
9	Diskussion	82
9.1	Beantwortung der Forschungsfragen	82
9.2	Einordnung in den Forschungsstand	88
9.3	Designwissen und betriebswirtschaftliche Implikationen	89
9.4	Güte, Geltungsgrenzen und weiterer Forschungsbedarf	93
10	Fazit und Ausblick	95
	Literaturverzeichnis	98
Anhang A	Ergänzende Forschungsartefakte	105
A.1	Ergänzende Tabellen des Argumentations- und Nachweispfads	105
A.2	Auditspur der Literaturstudie	108
A.3	Finaler Synthesekorpus der Literaturstudie	109
A.4	Eingefrorener Versuchs- und Laufzeitvertrag	113
A.5	Fairness- und Informationsparitätsprüfung	114
A.6	Ergänzende Stabilitätswerte	115
A.7	Implementations- und Prüfausschnitte	117
A.8	Mechanismusprüfung und qualitative Ereignisspurvignetten	124
A.9	Verzeichnis des ergänzenden digitalen Materials	126
A.10	Endpunkt-, Analyse- und Anspruchsgrenzen	127

Tabellenverzeichnis

Tabelle 1.	Strukturierung der Forschungsfragen	4
Tabelle 2.	Methodisches Vorgehen der Literaturstudie	19
Tabelle 3.	Ein- und Ausschlusskriterien	21
Tabelle 4.	Zentrale Befundlinien der Literatursynthese	29
Tabelle 5.	Konzeptmatrix der engsten Gegenarbeiten	30
Tabelle 6.	Eignung des Retail-Prüfstands	35
Tabelle 7.	Faktorielle Bedingungen	38
Tabelle 8.	Prüfannahmen und Operationalisierung	39
Tabelle 9.	Notation der faktoriellen Estimanden	40
Tabelle 10.	Primäre Endpunkte	42
Tabelle 11.	Gestaltungsanforderungen der Laufzeitinterventionsschicht	51
Tabelle 12.	Komponenten und Autoritäten	58
Tabelle 13.	Chronologische Ereignisspur des Artefakts	66
Tabelle 14.	Primäre Ergebnisse der Retail-Evaluation	70
Tabelle 15.	Direkte Kontraste gegenüber Native	71
Tabelle 16.	Faktorielle Effekte	72
Tabelle 17.	DB-Verteilungskennzahlen der vier Bedingungen	73
Tabelle 18.	Pass-Stabilität über sechs Seeds	75
Tabelle 19.	Ergebniswechsel gegenüber Native	75
Tabelle 20.	Ressourcenprofil der Bedingungen	80
Tabelle 21.	Betriebliche Überführung der empirischen Befunde	92
Tabelle 22.	Herleitung des analytischen Bezugsrahmens	106
Tabelle 23.	Evidenzlandkarte der Literatursynthese	106
Tabelle 24.	Formative DSR-Iterationskette	107
Tabelle 25.	Leseschlüssel der Ereignisspur-IDs	108
Tabelle 26.	Auditspur der Literaturstudie	109
Tabelle 27.	Finaler Synthesekorpus der Literaturstudie	110
Tabelle 28.	Laufzeitvertrag der finalen Evaluation	114
Tabelle 29.	Fairness- und Informationsparitätsprüfung	114
Tabelle 30.	Seedweise DB-Differenzen gegenüber Native	115
Tabelle 31.	DB-Aufgabenverteilung innerhalb der sechs Seeds	115
Tabelle 32.	Aufgabenbezogene Ergebnisrichtung	115
Tabelle 33.	Vollständige DB-Match-Stabilitätskurven	116
Tabelle 34.	Vollständige Exact-Mutation-Stabilitätskurven	116
Tabelle 35.	Exact-Seedverteilung und Kontraste	117
Tabelle 36.	Verifikationsmatrix der Gestaltungsanforderungen	123
Tabelle 37.	Zensus des Mechanismusaudits	124
Tabelle 38.	Übersicht der sechs Ereignisspurvignetten	125
Tabelle 39.	Ergänzendes digitales Material	126

Abbildungsverzeichnis

Abbildung 1. Agent–Tool–Umwelt-Schleife	7
Abbildung 2. PRISMA-Flussdiagramm der Literatursauswahl	23
Abbildung 3. Zuordnung der DSRM-Aktivitäten	33
Abbildung 4. Systemkontext des Laufzeitökosystems	56
Abbildung 5. Laufzeitsequenz des Artefakts	60
Abbildung 6. DB-Match-Verteilung und gepaarte Seedkontraste	73
Abbildung 7. Bruttobewegung gegenüber Native	76

Listingverzeichnis

1	Gekürzte A-Ereignisspur: initiale Aktualisierung des Zielzustands	67
2	Gekürzte A-Ereignisspur: materialisierte Unterstützungskarte	68
3	Gekürzte B-Ereignisspur: Evidenzbedarf und unveränderte Freigabe	69
4	Konfigurations-Freeze und Informationsgrenzen	118
5	Gekürzte Quellen- und Autoritätsprüfung von Komponente A	119
6	Dispositionen und Evidenzinvariante von Komponente B	119
7	Deterministische Laufzeitentscheidung von Komponente B	120
8	Serialisierung des faktoriellen Laufzeitdatensatzes	121
9	Gekürzter Zell-, Seed- und Request-Isolationpfad	122

Abkürzungsverzeichnis

API	Application Programming Interface
BPM	Business Process Management
CRM	Customer Relationship Management
CSV	Comma-Separated Values
DB	Datenbank
DSR	Design Science Research
DSRM	Design Science Research Methodology
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LLM	Large Language Model
MD	Markdown
NLP	Natural Language Processing
PICOC	Population, Intervention, Comparison, Outcome, Context
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PRISMA-S	PRISMA for Searching
SD	Standardabweichung
SLA	Service Level Agreement
USD	US-Dollar
W3C	World Wide Web Consortium

Symbolverzeichnis

T	Anzahl der Evaluationsaufgaben
n	Anzahl der Wiederholungen je Aufgabe
k	Anzahl gemeinsam betrachteter Wiederholungen, $1 \leq k \leq n$
c_i	Anzahl erfolgreicher Wiederholungen für Aufgabe i
H_r	Anzahl der Aufgaben mit genau r erfolgreichen Wiederholungen
D	Mittlere normalisierte paarweise Uneinigkeit der Wiederholungsergebnisse
a, b	Binäre Faktorindikatoren für semantische Unterstützung und Pre-Write-Kontrolle
$\bar{Y}_{ab}$	Mittlerer binärer Endpunkt unter der Faktorbelegung (a, b)
pass^k	Anteil der Aufgaben, die in allen k betrachteten Wiederholungen erfolgreich sind
$\text{pass}@k$	Anteil der Aufgaben, die in mindestens einer von k betrachteten Wiederholungen erfolgreich sind

1 Einleitung

Problemstellung und Motivation

Sprachbasierte Künstliche Intelligenz (KI) gewinnt als betrieblicher Gestaltungsfaktor an Bedeutung. Im Jahr 2025 nutzten rund 20 % der Unternehmen in der Europäischen Union mit mindestens zehn Beschäftigten mindestens eine KI-Technologie (vgl. Eurostat (2026), S. 13). Genau 8,8 % setzten Technologien zur Erzeugung geschriebener oder gesprochener Sprache oder von Programmcode ein (vgl. Eurostat (2026), S. 20–21). Diese Verbreitungsmaße belegen noch keinen wirtschaftlichen Nutzen einzelner Anwendungen. Sie zeigen jedoch, dass die Gestaltung und Kontrolle sprachbasierter KI zunehmend eine reale betriebliche Entscheidung und nicht nur ein technisches Zukunftsszenario ist.

Ein besonders naheliegendes Anwendungsfeld ist der Kundenservice. Er ist nicht nur eine Kommunikationsschnittstelle, sondern ein betrieblicher Leistungsprozess, in dem Informationsverarbeitung, Ressourcen und konkrete Fallausführung gemeinsam die Serviceleistung bestimmen (vgl. Ray; Muhanna; Barney (2005), S. 625–629). Eine Feldstudie mit 5.172 Supportmitarbeitenden und mehr als drei Millionen Chats zeigt, dass KI-generierte Empfehlungen die Zahl gelöster Kundenanliegen pro Stunde im Mittel um 15 % erhöhten, wobei die Wirkung zwischen den Beschäftigtengruppen variierte (vgl. Brynjolfsson; Li; Raymond (2025), S. 889–891 und 913–915). Der Befund belegt die wirtschaftliche Relevanz KI-gestützter Kundenserviceprozesse und verdeutlicht zugleich, dass ihr Nutzen kontextabhängig zu bewerten ist.

Für transaktionalen Kundenservice genügt eine isolierte Steigerung der Bearbeitungseffizienz nicht. Eine Verkürzung der Bearbeitung oder Reduktion menschlichen Aufwands stiftet nur dann Nutzen, wenn das Kundenanliegen korrekt und vollständig abgeschlossen wird. Umgekehrt können zusätzliche Prüfungen Fehler begrenzen, aber Zeit, Interaktion und Kosten erhöhen. Das betriebliche Leistungsprofil ergibt sich deshalb aus dem Verhältnis von Ressourceneinsatz, Prozessverlauf und fachlichem Fallabschluss (vgl. Grönroos; Ojasalo (2004), S. 414–416).

Mit leistungsfähigen Large Language Models (LLMs) verschiebt sich diese Abwägung. Die in der Feldstudie untersuchte KI unterstützte weiterhin einen verantwortlichen Menschen. LLM-basierte Agenten können dagegen Äußerungen interpretieren, Handlungen planen und über externe Tools selbst mit Geschäftssystemen interagieren (vgl. Dong et al. (2024), S. 407–411; vgl. Qu et al. (2025), S. 1 und 5–6). Im Kundenservice können sie über mehrere Dialogschritte Kunden- und Bestellinformationen abrufen, Rückfragen stellen und operative Zustände verändern. Damit verschiebt sich die Verantwortung von der bloßen Unterstützung einer menschlichen Entscheidung zur eigenständigen Ausführung betrieblicher Transaktionen.

Mit der Ausführungsautorität steigt jedoch die Tragweite eines Fehlers. Eine sprachlich plausible Antwort genügt in einem transaktionalen Fall nicht. Ein Agent kann beispielsweise eine formal gültige Retoure auslösen und den Fall dennoch falsch bearbeiten, wenn er die falsche Bestellung adressiert, nur einen Teil eines zusammengesetzten Anliegens berücksichtigt oder eine nicht bestätigte Produktvariante verwendet. Eine belastbare Falllösung setzt daher voraus, dass Anliegen, verfügbare Evidenz, Regeln und operative Wirkung über mehrere Dialog- und Toolschritte konsistent zusammengeführt werden. Toolnutzende Agenten können an unterschiedlichen Stellen dieser Kette scheitern, etwa bei Planung, Toolauswahl, Ausführung oder Verarbeitung eines Toolresultats (vgl. Qu et al. (2025), S. 5–11; vgl. Lu; Li; Huo (2025), S. 1 und 3).

Die Benchmarkfamilie τ -bench macht diese Herausforderung in kontrollierten Supportdialogen mit öffentlichen Policies, domänenspezifischen Tools und veränderbaren Datenbankzuständen ausführbar. Sie prüft nicht nur eine sprachliche Antwort, sondern den erreichten Geschäftszustand und vorgegebene Aktionskriterien. Zugleich variiert der Erfolg leistungsfähiger Agenten über wiederholte Ausführungen deutlich (vgl. Yao et al. (2025), S. 1 und 4–6). Für Betreiber ist daher nicht allein relevant, ob ein Agent einen Fall einmal lösen kann. Entscheidend ist, wie verlässlich dies gelingt, welche Fehler eine zusätzliche Laufzeitintervention verhindert oder neu erzeugt und welcher Prozess- und Ressourcenaufwand damit verbunden ist.

Aus dieser Entwicklung entsteht ein betriebliches Gestaltungsproblem. Ein vollständig deterministischer Workflow kann bekannte Zustände und Übergänge konsistent steuern, ist jedoch bei sprachlicher Vielfalt, unvollständigen Angaben und Ausnahmen begrenzt. Ein weitgehend autonomer LLM-Agent erhält größeren Handlungsspielraum, überlässt aber auch die Bindung von Dialog, Evidenz und operativer Wirkung einem probabilistischen Modell. Zu entscheiden ist deshalb nicht pauschal zwischen Workflow und Agent. Zu bestimmen ist, welche semantische Unterstützung und welche maschinenprüfbare Kontrolle an welcher Stelle der Ausführung ein günstigeres Ergebnis-, Kontroll- und Aufwandsprofil erzeugen. Die Forschung zum Business Process Management (BPM) beschreibt die Verbindung generativer Modellfähigkeiten mit explizitem Prozesswissen als entsprechenden Gestaltungsraum (vgl. van der Aalst (2022), Abschn. 2–3; vgl. Kampik et al. (2025), Abschn. 2–4).

Für den Betreiber verdichtet sich dieses Gestaltungsproblem auf drei Fragen: Verbessert ein zusätzlicher Mechanismus den vollständigen Fallabschluss? Welche zuvor korrekten Verläufe beschädigt er dabei? Und rechtfertigt die Bilanz aus vermiedenen und neu erzeugten Fehlern seinen zusätzlichen Aufwand? Diese drei Fragen bilden den roten Faden der Arbeit.

Die Forschung adressiert bereits einzelne Teile dieses Gestaltungsproblems. Zustands- und workflowbasierte Architekturen externalisieren Prozessfortschritt oder Übergangslogik (vgl. Wu et al. (2024), Abschn. 3–4; vgl. Shi et al. (2025), Abschn. 3–4). Andere Ansätze bereiten Ziele, Regeln oder Evidenz für nachfolgende Entscheidungen auf oder kontrollieren Toolaktionen an einer Laufzeitgrenze (vgl. Mishra et al. (2025), Abschn. 3–5; vgl. Cuadron et al. (2025), Abschn. 4–6; vgl. Wang; Poskitt; Sun (2026), Abschn. 4–7). Aktuelle Arbeiten untersuchen darüber hinaus die Kompilierung von Prozesswissen und deklarativen Fähigkeiten in kontrollierbare Agentenlaufzeiten (vgl. Lim et al. (2026), Abschn. 3–5; vgl. Zhang et al. (2026), Abschn. 3–5).

Damit sind Nutzerzielverfolgung, explizite Zustandsrepräsentation, hybride Orchestrierung und Kontrolle vor zustandsverändernden Aktionen keine unbekannten Einzelideen. Offen bleibt jedoch, wie sich solche Mechanismen unter gleicher Informations- und Bewertungsgrundlage gegeneinander abwägen lassen. Zusätzliche Steuerung kann Fehler verhindern, aber auch zuvor korrekte Agentenpfade beschädigen oder zusätzlichen Aufwand erzeugen. Die engsten Gegenarbeiten koppeln mehrere Eingriffe oder betrachten Aufgabenerfolg, Fehlerfolgen, Korrekturpfade und Ressourcenverbrauch nur teilweise gemeinsam (vgl. Sah et al. (2026), Abschn. 3–5; vgl. Satija et al. (2026), Abschn. 3–5 und Anhang B; vgl. Shi et al. (2026), Abschn. 4–7; vgl. Cuadron et al. (2025), Abschn. 5–8 sowie Anhang; vgl. Chen et al. (2026), Abschn. 3–6).

Auf Grundlage dieser Synthese formuliert die Arbeit die Forschungslücke als Design- und Entscheidungslücke. Es fehlt kontextgebundenes Wissen darüber, wie semantische Unterstützung, Pre-Write-Kontrolle und ihre Kombination den gesamten Transaktionsverlauf verändern und wann ihre Aktivierung vertretbar wird. Die Lücke liegt nicht in der bloßen Existenz eines weiteren Orchestrierungsartefakts.

Ziel der Arbeit ist daher die Gestaltung, technische Instanziierung und empirische Evaluation einer modularen Laufzeitinterventionsschicht für transaktionalen agentischen Kundenservice. Der native LLM-Agent bleibt für Dialog, Planung und Handlungsvorschläge verantwortlich. Ergänzt wird er um zwei unabhängig schaltbare Komponenten:

1. **Komponente A: semantische Unterstützung** macht offene Kundenanliegen und belegte Informationen vor der nächsten Agentenentscheidung sichtbar.
2. **Komponente B: Pre-Write-Kontrolle** prüft zustandsverändernde Kandidaten unmittelbar vor ihrer Ausführung anhand expliziter Regeln und Evidenz.

Im weiteren Verlauf bezeichnen *Komponente A* und *Komponente B* stets diese beiden Artefaktbestandteile. *Faktor A* und *Faktor B* bezeichnen ihre Aktivierung im faktoriellen Design. Die vier Versuchsbedingungen sind C0 Native, C1 Semantische Unterstützung, C2 Pre-Write-Kontrolle und C3 Kombination.

Die beobachteten Folgen werden entlang von Effektivität, Zuverlässigkeit, Kontrollqualität und Effizienz eingeordnet.

Die zentrale Forschungsfrage lautet:

Wie verändern semantische Unterstützung und Pre-Write-Kontrolle einzeln und gemeinsam den Erfolg, die Fehlerverläufe, die Stabilität und den Aufwand transaktionaler LLM-Supportagenten unter gleichen Informations- und Bewertungsbedingungen?

Fünf Teilfragen strukturieren Gestaltung und Evaluation. Tabelle 1 ordnet sie den zentralen Erkenntnisinteressen der Arbeit zu.

ID	Teilforschungsfrage
TF1	Welche Anforderungen an semantische Unterstützung, Pre-Write-Kontrolle und deren Messung lassen sich aus Literatur und transaktionalen Prozessanforderungen ableiten?
TF2	Wie können beide Mechanismen als unabhängig schaltbare Laufzeitschicht umgesetzt werden, ohne zusätzliche Aufgabeninformationen oder versteckte Referenzinformationen des Benchmarks einzuführen?
TF3	Welche isolierten und kombinierten Wirkungen zeigen beide Komponenten auf Aufgabenerfolg, Datenbankabgleich, exakte Mutationsausführung und Wiederholungsstabilität?
TF4	In welchen Fällen führen die Komponenten zu geretteten, beschädigten oder unveränderten Ergebnisverläufen, und welche beobachtbaren Komponentenpfade und Fehlerarten gehen mit diesen Wechseln einher?
TF5	Welche zusätzlichen Kosten der Agenten-Programmierschnittstelle (API), Laufzeiten und Interaktionsaufwände entstehen, und unter welchen betrieblichen Annahmen erscheint eine Aktivierung der Komponenten vertretbar?

Tabelle 1. Thematische Strukturierung der Forschungsfragen

Zur Beantwortung der Fragen vergleicht die Arbeit den nativen Agenten, beide Einzelbedingungen und ihre Kombination in einem kontrollierten faktoriellen Evaluationsdesign. Bewertet werden fachlicher Erfolg, Fehlerverläufe, Wiederholungsstabilität und zusätzlicher Aufwand. Kapitel 4 begründet das Forschungsdesign und definiert Kontraste, Messgrößen und Auswertungsverfahren.

Wissenschaftlicher und betrieblicher Beitrag

Die Arbeit leistet einen wissenschaftlichen Beitrag auf drei Ebenen. Erstens wird ein ausführbares Artefakt gestaltet, das semantische Unterstützung und Pre-Write-Kontrolle unabhängig voneinander aktivierbar macht. Zweitens untersucht die Evaluation nicht nur Erfolgsraten, sondern auch Schadensfälle, Wiederholungsstabilität und zusätzlichen Aufwand. Drittens werden die Befunde zu kontextgebundenem Designwissen über Nutzen und Grenzen modularer Laufzeitinterventionen verdichtet. Damit folgt die Arbeit dem Design-Science-Verständnis von Gestaltung und Evaluation (vgl. Hevner et al. (2004), S. 82–83; vgl. Peffers et al. (2007), S. 55–56).

Der betriebliche Beitrag liegt darin, technische Laufzeitmechanismen nicht isoliert als Qualitätsverbesserung zu betrachten. Stattdessen werden Ergebnisqualität, Stabilität, Eingriffe und Ressourcenaufwand gemeinsam als Konfigurationsprofile verglichen.

Die Aussagekraft bleibt auf Artefaktstand, Modellkonfiguration und Retail-Domäne begrenzt. Kosten, Laufzeit, Nachrichten und Tool-Aufrufe sind betriebliche Näherungsgrößen, keine Messung realer Contact-Center-Kosten oder kundenseitiger Servicequalität (vgl. Braggaar et al. (2026), S. 3 und 24–26; vgl. Grönroos; Ojasalo (2004), S. 414–416).

Aufbau der Arbeit

Kapitel 2 entwickelt die theoretischen Grundlagen. Kapitel 3 dokumentiert die systematische Literaturrecherche und präzisiert die Forschungslücke. Kapitel 4 erläutert Forschungs- und Experimentaldesign, Kapitel 5 leitet die Gestaltungsanforderungen ab und Kapitel 6 beschreibt Architektur und Instanziierung der Laufzeitinterventionsschicht. Nach der Demonstration des Artefakts folgen Ergebnisdarstellung und Diskussion. Das Fazit bündelt die Antworten auf die Forschungsfragen und das resultierende Design- und Entscheidungswissen.

2 Theoretische Grundlagen

Dieses Kapitel definiert die zentralen Konstrukte der Arbeit. Es führt vom betrieblichen Einsatz von KI in ausführbaren Geschäftsprozessen zu einem LLM-basierten Agenten, der in einem mehrstufigen Kundendialog Tools verwendet und operative Geschäftszustände verändert. Darauf aufbauend grenzt es transaktionale Prozessqualität, hybride Orchestrierung sowie die ergebnis- und aufwandsbezogene Evaluation ab.

2.1 Large Language Models und toolnutzende Agenten

Von betrieblicher KI-Nutzung zu agentischer Prozessausführung

Informationstechnologie erzeugt betrieblichen Wert nicht als isolierte Ressource, sondern durch ihr Zusammenwirken mit Prozessen, Daten und organisationalen Fähigkeiten. Für den Kundenservice zeigt sich, dass Prozessleistung aus komplementären technischen und organisatorischen Ressourcen entsteht (vgl. Ray; Muhanna; Barney (2005), S. 625–629). Die relevante Analyseeinheit dieser Arbeit ist deshalb nicht ein Sprachmodell im Labor, sondern ein in einen operativen Serviceprozess eingebettetes Informationssystem.

Generative KI erweitert klassische Prozessunterstützung um die Fähigkeit, unstrukturierte Sprache kontextabhängig zu verarbeiten und neue Ausgaben zu erzeugen. Erst durch die Verbindung mit Gedächtnis, Planung und externen Aktionen entsteht daraus ein agentisches System (vgl. Wang et al. (2024), Abschn. 2; vgl. Dong et al. (2024), S. 407–411). Für Unternehmen verschiebt sich damit zugleich der Wirkungsbereich. Ein System kann nicht nur Informationen präsentieren oder Entscheidungen empfehlen, sondern über Tools selbst Schritte eines Geschäftsprozesses ausführen. Je näher diese Schritte an einer operativen Zustandsänderung liegen, desto weniger genügt es, ausschließlich die sprachliche Qualität der Modellausgabe zu betrachten.

Für die weitere Abgrenzung sind drei Ebenen relevant: das probabilistische Sprachmodell als Basiskomponente, der Agent als handelndes System und die Tool-Nutzung als Schnittstelle zwischen Modellentscheidung und betrieblicher Prozesswirkung.

Probabilistische Sprachmodelle als Basiskomponente

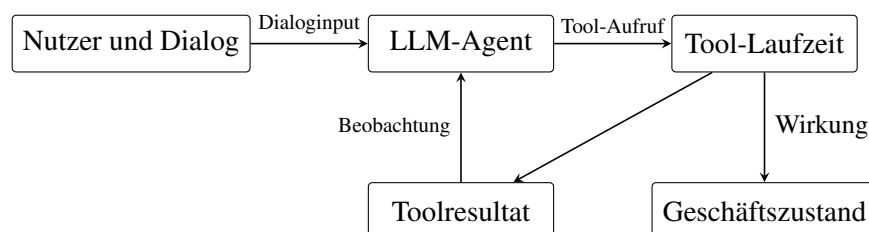
LLMs verarbeiten Eingabesequenzen und erzeugen Wahrscheinlichkeitsverteilungen über mögliche Fortsetzungen. Die technische Grundlage moderner Modelle bildet überwiegend die Transformer-Architektur (vgl. Vaswani et al. (2017), S. 1–3). Durch Vortraining und Instruktionsanpassung können sie Aufgabenbeschreibungen, Nutzeräußerungen und Kontextinformationen gemeinsam verarbeiten (vgl. Ouyang et al. (2022), S. 27730–27731).

Diese probabilistische Funktionsweise ermöglicht flexible sprachliche Interaktion, garantiert aber weder deterministische Wiederholbarkeit noch eine korrekte Wirkung in einer externen Umgebung. Die Modellausgabe ist zunächst eine sprachliche beziehungsweise strukturierte Kandidatenausgabe. Ob eine Aufgabe damit erfüllt wurde, hängt zusätzlich davon ab, wie die Ausgabe interpretiert wird, welche Umgebungsinformationen verfügbar sind und welche Folgen eine ausgeführte Aktion erzeugt. Ein Modell kann daher eine plausible Antwort oder einen syntaktisch gültigen Tool-Aufruf erzeugen, obwohl eine Voraussetzung fehlt, das falsche Geschäftsobjekt adressiert wird oder der angestrebte Zielzustand unerreicht bleibt. Für diese Arbeit ist deshalb die Trennung zwischen sprachlicher Plausibilität, korrekter Entscheidung und ausführbarer Prozesswirkung grundlegend (vgl. Yao et al. (2025), S. 1–2).

Vom Sprachmodell zum handelnden Agenten

Der Agentenbegriff erweitert Sprachgenerierung um zielgerichtetes Handeln in einer Umgebung. Klassisch werden Agenten durch Autonomie, Reaktivität, Proaktivität und soziale Interaktion charakterisiert (vgl. Wooldridge; Jennings (1995), S. 118–119). LLM-basierte Agenten verwenden ein Sprachmodell als zentrale Entscheidungsinstanz und ergänzen es um Wahrnehmung, Planung, Gedächtnis und Aktionen (vgl. Dong et al. (2024), S. 407–411; vgl. Wang et al. (2024), Abschn. 2). Der relevante Systemgegenstand ist damit nicht das Modell allein, sondern die geschlossene Interaktion aus Nutzer, Agent und Umgebung.

In einer solchen Agent-Umwelt-Schleife erhält der Agent Beobachtungen aus dem Dialog und aus früheren Aktionen, bildet daraus einen Arbeitskontext und wählt einen nächsten Schritt. Dieser kann eine Antwort, eine Rückfrage oder ein Werkzeugaufruf sein. Die Umgebung verarbeitet die Aktion und liefert ein neues Resultat, das wiederum zur nächsten Beobachtung wird. ReAct konkretisiert diese Grundidee, indem Schlussfolgern und Handeln wiederholt miteinander verknüpft werden (vgl. Yao et al. (2023), S. 1–4). Die Schleife endet nicht bereits mit einer lokal plausiblen Aktion, sondern erst durch eine Terminierungsentscheidung des Agenten oder der umgebenden Laufzeit. Abbildung 1 verdichtet diesen rekursiven Zusammenhang zwischen Dialog, Agentenentscheidung, Werkzeugresultat und Geschäftszustand.



Quelle: vgl. Wooldridge; Jennings (1995), S. 118–119; Yao et al. (2023), S. 1–4; Qu et al. (2025), S. 5–11.

Abbildung 1. Allgemeine Agent–Tool–Umwelt-Schleife

Für mehrstufigen Kundenservice entstehen daraus drei voneinander abhängige Fähigkeiten. Der Agent muss erstens das aktuelle Kundenanliegen und bereits vorhandene Informationen im Dialogkontext fortschreiben. Zweitens muss er aus möglichen Aktionen eine zum aktuellen Prozessstand passende Handlung wählen. Drittens muss er die Folgen dieser Handlung aus dem Toolresultat korrekt in seine weitere Planung übernehmen. Ein Fehler auf einer dieser Ebenen kann sich über nachfolgende Schritte fortpflanzen, obwohl die einzelnen Nachrichten sprachlich kohärent bleiben. Tool- und Agentenforschung ordnet entsprechende Fehler unter anderem der Planung, Ausführung, Resultatverarbeitung und Answererzeugung zu (vgl. Qu et al. (2025), S. 5–11; vgl. Lu; Li; Huo (2025), S. 1 und 3).

Tool-Nutzung als strukturierte Umweltinteraktion

Tools stellen typisierte Funktionen bereit, mit denen ein Agent Informationen lesen, Berechnungen durchführen oder Zustände verändern kann (vgl. Qu et al. (2025), S. 3). Tool-Nutzung umfasst dabei nicht nur die Ausgabe eines Funktionsnamens. Qu et al. unterscheiden im Tool-Learning-Prozess Aufgabenplanung, Toolauswahl, Tool-Aufruf und die Verarbeitung des Resultats (vgl. Qu et al. (2025), S. 5–11). Ein erfolgreicher Aufruf setzt somit voraus, dass der Agent das benötigte Werkzeug auswählt, seine Argumente schema- und kontextgerecht belegt und das Resultat anschließend korrekt weiterverwendet.

Für diese Arbeit ist die Wirkung eines Tools auf die Geschäftsumgebung entscheidend. *Leseaktionen (Reads)* liefern Informationen über Kunden-, Bestell- oder Produktobjekte, ohne den operativen Zustand zu verändern. *Schreibaktionen (Writes)* verändern diesen Zustand, beispielsweise durch eine Stornierung, Retoure, Erstattung oder einen Umtausch. Die Unterscheidung bildet keine vollständige Risikoklassifikation. Auch eine falsche Leseaktion kann die weitere Planung verfälschen oder Daten offenlegen. Ein fehlerhafter Write erzeugt jedoch zusätzlich eine unmittelbare operative Wirkung, deren Korrektur einen weiteren Geschäftsprozess erfordern kann. Die besondere Fehlerrelevanz mutierender Agentenaktionen wird auch in Ereignisspuranalysen toolnutzender Supportagenten hervorgehoben (vgl. Cuadron et al. (2025), Abschn. 3–6).

Die Qualität des Gesamtverlaufs ist deshalb nicht durch die syntaktische Gültigkeit einzelner Aufrufe bestimmt. Erforderlich ist vielmehr, dass Kundenanliegen, beobachtbare Informationen, gewählte Aktion und tatsächliche Wirkung über den gesamten Verlauf zusammenpassen (vgl. Qu et al. (2025), S. 5–11; vgl. Yao et al. (2025), S. 1–2). Der folgende Abschnitt präzisiert diesen Zusammenhang als Geschäftsprozess.

2.2 Transaktionaler Kundenservice als Geschäftsprozess

Vom Kundendialog zum ausführbaren Geschäftsprozess

Ein Geschäftsprozess verbindet Ereignisse, Aktivitäten und Entscheidungen zu einer Leistung für Kunden oder andere organisationale Anspruchsgruppen. Business Process Management betrachtet deshalb nicht nur die Optimierung einzelner Tätigkeiten, sondern die konsistente Ausführung und Verbesserung der gesamten Leistungskette (vgl. Dumas et al. (2018), Kap. 1). Für den Kundenservice ist diese Prozessebene auch betriebswirtschaftlich relevant: Empirische IS-Forschung untersucht den Wert von Informationstechnologie gerade auf Ebene des Kundenserviceprozesses und zeigt, dass technische Ressourcen nicht losgelöst von ihrer organisationalen Einbettung bewertet werden können (vgl. Ray; Muhanna; Barney (2005), S. 625–629).

Task-orientierte Dialogsysteme unterstützen Nutzer bei einem konkreten Ziel und unterscheiden sich damit von offenen Konversationssystemen. Ihre Qualität kann sich auf sprachliche, technische und aufgabenbezogene Konstrukte beziehen. Ein systematisches Review von 122 Studien zeigt unter anderem Korrektheit, Qualität, Effizienz, Zufriedenheit und Aufgabenerfolg (*Task Success*) als häufig untersuchte, aber uneinheitlich operationalisierte Größen (vgl. Braggaar et al. (2026), S. 8–13 und 24–26). Bereits diese Vielfalt macht deutlich, dass ein gut formulierter Dialogbeitrag und ein erfolgreich ausgeführter Serviceprozess nicht identisch sind.

Im *transaktionalen* Kundenservice führt der Dialog zu einer beabsichtigten Veränderung betrieblicher Objekte. Der Agent muss ein Anliegen erheben, Kunden- und Auftragsobjekte identifizieren, fehlende Informationen beschaffen, Geschäftsregeln anwenden, gegebenenfalls eine Bestätigung einholen, ein Tool ausführen und das Ergebnis kommunizieren. Eine Rückgabe oder ein Umtausch ist somit keine einzelne Textklassifikation, sondern eine zusammenhängende Folge von Dialog-, Entscheidungs- und Ausführungsschritten. Bei zusammengesetzten Anliegen können mehrere Teilziele parallel offen sein. Ein formal zulässiger Teilschritt löst den Fall dann noch nicht, wenn ein weiteres Kundenanliegen unerledigt bleibt. Der in τ -bench operationalisierte Supportkontext verlangt entsprechend die schrittweise Sammlung von Informationen, Policybefolgung und API-Ausführung über mehrere Nutzerinteraktionen (vgl. Yao et al. (2025), S. 1–2).

Analytischer Bezugsrahmen für transaktionale Agentenfehler

Um diesen Zusammenhang untersuchbar zu machen, synthetisiert die Arbeit vier Dimensionen aus Dialog-, Tool-, Provenienz- und Prozessliteratur. Task-orientierte Dialogforschung stellt das zu erreichende Nutzerziel in den Mittelpunkt (vgl. Braggaar et al. (2026), S. 2–3). Tool-Literatur beschreibt Beobachtungen und Aktionen an externen Funktionen (vgl. Qu et al.

(2025), S. 3 und 5–11). Das Provenienzmodell des World Wide Web Consortium (W3C) begründet die Rückführbarkeit erzeugter Informationen auf Entitäten und Aktivitäten (vgl. Moreau; Missier (2013), Abschn. 2–5). Workflow-Management repräsentiert die Ausführung eines Geschäftsprozesses über Fälle, Aktivitäten und deren fortgeschrittenen Zustand (vgl. van der Aalst; van Hee (2002), Kap. 2–3; vgl. Dumas et al. (2018), Kap. 1). Klassische Transaktionsverarbeitung schließlich behandelt logisch zusammengehörige Zustandsänderungen und deren verlässliche Ausführung (vgl. Härder; Reuter (1983), S. 289–290).

Tabelle 22 im Anhang zeigt, wie diese getrennten Ursprünge für den Untersuchungsgegenstand präzisiert werden. Die Verbindung der vier Dimensionen ist ein analytischer Bezugsrahmen dieser Arbeit. Sie bezeichnet kein eigenes Artefakt und keine zusätzliche Architekturkomponente, sondern dient der begrifflichen Trennung von Fehler-, Kontroll- und Bewertungslogik.

Der hier verwendete Transaktionsbegriff ist bewusst enger als der gesamte Kundendialog, aber weiter als ein einzelner API-Aufruf. Er bezeichnet die aus Geschäftssicht zusammengehörige operative Änderung. Daraus folgt nicht, dass die verwendeten Support-Tools vollständige Datenbanktransaktionen mit allen klassischen ACID-Eigenschaften bereitstellen. Begriffe wie atomare Ausführung, Rollback oder Isolation werden daher nur dort beansprucht, wo die konkrete Laufzeit sie technisch tatsächlich unterstützt (vgl. Härder; Reuter (1983), S. 289–290).

Der Bezugsrahmen verbindet Ablauf und Ergebnis. Das Nutzerziel gibt die Sollrichtung vor, Evidenz begrenzt die als beobachtet geltenden Tatsachen, der Prozesszustand hält deren Entwicklung fest und die Zustandsänderung erzeugt die operative Wirkung. Ihre Trennung macht sichtbar, warum vorhandene Information nicht automatisch dem richtigen Ziel oder Objekt zugeordnet und eine policyzulässige Aktion nicht zwangsläufig vollständig oder zielrichtig ist. Die Verbindung ist eine analytische Arbeitsgrundlage und keine Wirksamkeitsannahme für das spätere Artefakt (vgl. Braggaar et al. (2026), S. 2–3; vgl. Qu et al. (2025), S. 3 und 5–11; vgl. Moreau; Missier (2013), Abschn. 2–5; vgl. Dumas et al. (2018), Kap. 1).

Prozess- und Betriebsqualität

Geschäftsprozessleistung ist mehrdimensional. Die BPM-Literatur verwendet insbesondere Zeit, Kosten, Qualität und Flexibilität als typische Leistungsperspektiven. Konkrete Indikatoren müssen aus Ziel und Kontext des jeweiligen Prozesses abgeleitet werden (vgl. Dumas et al. (2018), Kap. 1; vgl. Van Looy; Shafagatova (2016), Abschn. 3–4). Für Serviceprozesse kommt hinzu, dass Effizienz und wahrgenommene Qualität nicht unabhängig behandelt werden können. Veränderungen des Ressourceneinsatzes und der Kundenmitwirkung können zugleich die erbrachte beziehungsweise wahrgenommene Servicequalität verändern (vgl. Grönroos; Ojasalo (2004), S. 414–416).

Auf dieser Grundlage synthetisiert die Arbeit für agentischen Kundenservice vier Perspektiven. *Effektivität* bezeichnet die korrekte und vollständige Lösung des Kundenanliegens. *Effizienz* betrifft den dafür benötigten Zeit-, Modell-, Tool- und Kommunikationsaufwand. *Zuverlässigkeit* bezeichnet die Stabilität der Leistung über wiederholte Ausführungen. *Kontrollqualität* betrifft sowohl verhinderte Fehlaktionen als auch unnötige Eingriffe in bereits erfolgreiche Verläufe. Diese Perspektiven können konfliktieren. Eine zusätzliche Prüfung kann eine falsche Transaktion verhindern, zugleich jedoch Laufzeit, Modellkosten oder die Zahl erforderlicher Kundeninteraktionen erhöhen. Die Zuordnung verbindet die BPM-Dimensionen mit den in der Dialogevaluation unterschiedenen Konstrukten Task Success, Effizienz, Robustheit und Fehler (vgl. Braggaar et al. (2026), S. 12–13).

Technischer Erfolg und betrieblicher Wert sind daher verwandt, aber nicht identisch. Ein korrekter finaler Datenbankzustand bildet einen wesentlichen Teil der operativen Falllösung ab. Laufzeit, Tool-Aufrufe, Nachrichten und Modellkosten beschreiben beobachtbaren Systemaufwand. Sie messen jedoch weder reale Vollkosten noch wahrgenommene Servicequalität, Kundenzufriedenheit oder langfristige Kundenwirkung. Diese Unterscheidung begründet die spätere gemeinsame Qualitäts-, Aufwands- und Kontrollperspektive, ohne aus Benchmarkdaten unmittelbar eine Unternehmensrendite abzuleiten. Diese Begrenzung folgt der Prozessperspektive des IT-Geschäftswerts und der Serviceproduktivität, nach denen technische Ressourcen beziehungsweise interne Inputs nicht isoliert von Prozesskontext und wahrgenommener Servicequalität bewertet werden können (vgl. Ray; Muhanna; Barney (2005), S. 625–629; vgl. Grönroos; Ojasalo (2004), S. 414–416).

2.3 Hybride Orchestrierung und Laufzeitkontrolle

Kontinuum zwischen autonomer und deterministischer Steuerung

In einer weitgehend autonomen Agentenarchitektur interpretiert das LLM den Dialog, plant Schritte, wählt Tools und entscheidet über den Abschluss. Diese Form erhält einen großen Verhaltensraum, überlässt aber auch die Prozessdisziplin dem Modell. Eine deterministische Workflowarchitektur kodiert dagegen zulässige Zustände und Übergänge explizit. Sie bietet hohe Kontrollierbarkeit für bekannte Pfade, kann jedoch bei sprachlicher Vielfalt, Ausnahmen und unvollständigen Informationen unflexibel werden. Workflow-Management versteht ein Prozessmodell entsprechend als explizite Spezifikation dafür, welche Aktivitäten unter welchen Bedingungen innerhalb eines Falls ausgeführt werden (vgl. van der Aalst; van Hee (2002), Kap. 2–3; vgl. Dumas et al. (2018), Kap. 1). Die BPM-Forschung beschreibt die Verbindung von Prozesswissen und generativer KI entsprechend als Gestaltungsraum, in dem flexible Modellfähigkeiten mit expliziten Prozessrepräsentationen verbunden werden können (vgl. Kampik et al. (2025), Abschn. 2–4).

Autonome Agenten und deterministische Workflows bilden jedoch keine erschöpfende Dichotomie. Zwischen ihnen liegt ein Kontinuum, auf dem Dialoginterpretation, Kontextverwaltung, Planung, Toolauswahl, Aktionszulassung und Terminierung unterschiedlich verteilt werden können. Auch das Konzept des Autonomous Process Execution Management beschreibt Prozessautonomie als graduelle Stufung und nicht als binären Wechsel zwischen manueller Steuerung und vollständiger Autonomie (vgl. van der Aalst (2022), Abschn. 2–3). *Hybride Orchestrierung* bezeichnet in dieser Arbeit eine explizite Aufgabenteilung. Das Sprachmodell bearbeitet semantisch mehrdeutige Aufgaben, während Softwarelogik begrenzte, maschinenprüfbare Zustände oder Entscheidungsgrenzen durchsetzt. Hybrid bedeutet damit nicht bloß, dass ein Modell in ein Softwaresystem eingebettet ist. Erforderlich ist eine sichtbare Zuordnung von Entscheidungsverantwortung.

Eine solche Aufgabenteilung ist eine Gestaltungsentscheidung und keine allgemeine Überlegenheitsannahme. Modellautonomie kann auf sprachliche Vielfalt und unerwartete Anliegen reagieren. Explizite Steuerung kann bekannte Prozessbedingungen konsistent anwenden. Zugleich erzeugt jede zusätzliche Kontrollschicht Integrationsaufwand und kann den Verhaltensraum eines an sich erfolgreichen Agenten unnötig begrenzen. Die geeignete Verteilung hängt daher von Formalisierbarkeit, Fehlerfolgen und erforderlicher Flexibilität des jeweiligen Prozessschritts ab. Aus der theoretischen Plausibilität einer Aufgabenteilung folgt deshalb noch keine empirische Überlegenheit. Kapitel 3 prüft getrennt, welche Wirkungen und Grenzen die Forschung für entsprechende Laufzeitverfahren tatsächlich berichtet (vgl. Sah et al. (2026), Abschn. 3–5; vgl. Cuadron et al. (2025), Abschn. 5–8).

Für die vorliegende Arbeit wird diese Funktionsverteilung um die Entscheidungsautorität ergänzt. Ein probabilistisch extrahierter Zielzustand wird dabei nur beratend genutzt, etwa um einen fehlenden Informationsschritt anzuzeigen oder die Fortsetzung eines offenen Angelegens zu unterstützen. Ein hartes Veto vor einer Transaktion benötigt dagegen eine stärkere Begründungsbasis, weil ein Fehlblockieren selbst den Prozessverlauf beeinträchtigen kann. In dieser Arbeit bezeichnet *Autoritätstrennung* deshalb das Prinzip, semantisch unsichere Interpretationen beratend zu nutzen, harte Zulassungs- oder Ablehnungsentscheidungen jedoch auf explizite Policy, Toolverträge und sichtbare deterministisch prüfbare Fakten zu begrenzen. Das Prinzip operationalisiert die zuvor beschriebene sichtbare Entscheidungsverantwortung. Seine konkrete Instanziierung ist Gegenstand von Kapitel 6.

Expliziter Prozesszustand und Evidenzprovenienz

Der freie Modellkontext enthält Dialognachrichten, Toolresultate und Zwischenschritte in überwiegend unstrukturierter Form. Ein expliziter Prozesszustand projiziert daraus ausgewählte Informationen in typisierte Felder, beispielsweise offene Ziele, identifizierte Objekte, bestätigte Argumente oder bereits ausgeführte Aktionen. Die Projektion ersetzt den Dia-

logkontext nicht. Sie schafft eine zusätzliche Repräsentation, auf die Kontrollkomponenten konsistent zugreifen können. Damit überträgt sie das Workflow-Prinzip eines fortgeschriebenen Fallzustands (vgl. van der Aalst; van Hee (2002), Kap. 2–3) auf eine Laufzeit, in der nicht alle Übergänge vorab vollständig festgelegt sind.

Modellkontext und Prozesszustand sind daher nicht gleichzusetzen. Der Modellkontext ist die für einen einzelnen Aufruf bereitgestellte Folge von Nachrichten und Beobachtungen. Ein persistenter Prozesszustand ist dagegen eine von der Anwendung kontrollierte Repräsentation, die ausgewählte Informationen über Aufrufe und Dialogturns hinweg fortschreibt. Eine solche Repräsentation erzeugt kein autonomes Langzeitgedächtnis des Sprachmodells. Die Laufzeit legt explizit fest, welche belegten Informationen gespeichert, aktualisiert und erneut bereitgestellt werden. Nicht das Modell entscheidet damit verdeckt, was dauerhaft erinnert wird. Die Anwendung materialisiert einen prüfbaren Fallzustand (vgl. van der Aalst; van Hee (2002), Kap. 2–3; vgl. Moreau; Missier (2013), Abschn. 2–5).

Entscheidend ist dabei die Provenienz. Ein Zustandswert sollte auf ein sichtbares Ereignis wie eine Nutzeräußerung oder ein Toolresultat rückführbar sein. Andernfalls könnte eine strukturierte Darstellung lediglich eine nicht belegte Modellannahme mit höherem Anschein von Sicherheit versehen. Neben dem Wert sind deshalb Quelle, zeitliche Ordnung und gegebenenfalls ein Konfliktstatus relevant. Geänderte Kundenwünsche oder neuere Toolresultate können ältere Angaben überholen, ohne dass diese aus der Historie verschwinden. Diese Forderung folgt dem Grundgedanken des PROV-Datenmodells, Informationen über die an der Erzeugung eines Datums beteiligten Entitäten und Aktivitäten explizit darstellbar zu machen (vgl. Moreau; Missier (2013), Abschn. 2–5). Die Arbeit übernimmt daraus das Prinzip der Rückführbarkeit, nicht das vollständige W3C-Datenmodell.

Die externe Zustandsrepräsentation reduziert damit nicht automatisch semantische Unsicherheit. Sie macht jedoch sichtbar, welche Informationen als belegt, offen oder widersprüchlich behandelt werden. Im späteren Artefakt bildet diese Logik die Grundlage für quellengebundene Nutzerziel-Unterstützung. Konkrete Schemas, Extraktionsverfahren und Budgets werden erst in Kapitel 6 beschrieben. Die Einschränkung folgt daraus, dass ein Prozesszustand nur die projizierten Ereignisse repräsentiert und Provenienz Rückführbarkeit, nicht inhaltliche Wahrheit garantiert (vgl. van der Aalst; van Hee (2002), Kap. 2–3; vgl. Moreau; Missier (2013), Abschn. 2–5).

Deterministische Laufzeitkontrolle vor Zustandsänderungen

Prozess-Compliance bezeichnet die Übereinstimmung ausgeführter Prozesse mit externen Regeln oder internen Geschäftsbedingungen. Sie kann bereits am Prozessmodell oder während laufender Prozessinstanzen geprüft werden (vgl. Rinderle-Ma; Winter; Benzin (2023), Abschn. 2–3). Eine Laufzeitkontrolle beobachtet hierzu Ereignisse und bewertet eine beabsichtig-

te oder bereits erfolgte Handlung gegen spezifizierte Bedingungen. Für zustandsverändernde Aktionen ist eine Prüfung vor der Ausführung präventiv. Die operative Wirkung wird erst nach einer Zulassungsentscheidung ausgelöst.

Das Prinzip ähnelt der vollständigen Vermittlung sicherheitsrelevanter Zugriffe, nach der jeder Zugriff auf ein geschütztes Objekt geprüft werden sollte (vgl. Saltzer; Schroeder (1975), S. 1282). Diese Analogie ist begrenzt. Eine geschäftliche Transaktion verlangt nicht nur Autorisierung, sondern kann zusätzlich von Kundenintention, Objektbezug, materiellen Argumenten, Prozessstatus und Bestätigung abhängen. Eine Aktion kann somit technisch autorisiert und policyzulässig sein, ohne das richtige Kundenanliegen vollständig zu erfüllen.

Eine Kontrolle vor der Zustandsänderung prüft deshalb eine konkrete Kandidatenaktion unmittelbar vor ihrer Wirkung. Für die vorliegende Arbeit ergeben sich daraus als mögliche Prüfobjekte die Identität und Zugehörigkeit des Geschäftsobjekts, Pflichtargumente, Geschäftsregeln, Reihenfolgebedingungen, bereits erfolgte Writes und eine hinreichend konkrete Bestätigung. Sie bewertet nicht die gesamte interne Modellbegründung. Ihr Gegenstand ist die beobachtbare Beziehung zwischen aktuellem Prozesszustand und vorgeschlagener Zustandsänderung. Während klassische Transaktionsverarbeitung zusätzlich Atomarität, Konsistenz, Isolation und Dauerhaftigkeit adressiert (vgl. Härder; Reuter (1983), S. 289–290), übernimmt die hier begründete Grenze nur das engere Prinzip, logisch zusammengehörige Änderungsbedingungen vor der sichtbaren Wirkung gemeinsam zu prüfen. Die technische Ausgestaltung und die tatsächlich gewährleisteten Eigenschaften werden erst in Kapitel 6 festgelegt.

Unzureichende Evidenz, Fail-open und Over-Control

Eine Kontrollentscheidung kann drei semantische Situationen vorfinden: Die Aktion ist durch sichtbare Fakten gestützt, sie widerspricht sichtbaren Fakten oder die vorhandene Evidenz reicht für eine Entscheidung nicht aus. Die dritte Situation darf nicht stillschweigend mit Unzulässigkeit gleichgesetzt werden. Sie erfordert eine Strategie für unzureichende Evidenz, beispielsweise einen weiteren Informationsschritt, eine Eskalation oder eine begrenzte Freigabe. Das verwandte Konzept selektiver Klassifikation formalisiert eine Reject-Option, um Abdeckung gegen Fehlerrisiko abzuwägen (vgl. El-Yaniv; Wiener (2010), S. 1605–1606). Die Übertragung auf Agentenaktionen ist begrenzt. Eine abgelehnte Klassifikation verändert keinen Geschäftszustand, während eine zurückgehaltene Aktion einen laufenden Prozess blockieren kann.

Bei *Fail-closed* wird eine nicht positiv belegte Aktion zunächst nicht ausgeführt. Dies begrenzt unerwünschte Writes, kann aber korrekte Prozesse bei unvollständigem Zustand blockieren. *Fail-open* erhält den nativen Handlungspfad, akzeptiert jedoch das verbleibende Fehlerrisiko. Beide Strategien erzeugen daher unterschiedliche Fehlerkosten. Das überge-

ordnete Fail-safe-Prinzip verlangt, dass ein erkannter Fehler in einen Zustand überführt wird, der Schäden an den zu schützenden Ressourcen begrenzt (vgl. National Institute of Standards and Technology (n. d.), Stichwort „Fail Safe“). Welcher Zustand im Kundenservice tatsächlich schadensärmer ist, kann jedoch vom Prozess abhängen. Das Blockieren einer unbelegten Retoure schützt den Geschäftszustand, während das Blockieren eines notwendigen Transfers den Kundenfall verschlechtern kann. Fail-open und Fail-closed werden deshalb als kontextspezifische Kontrollpolitiken und nicht als allgemeine Qualitätsrangfolge verwendet. Hinzu kommt *Over-Control*: Eine Komponente kann auch dort aktiv werden, wo der native Agent korrekt gehandelt hätte, und dadurch zusätzliche Reads, Nachrichten, Laufzeit oder Abbrüche verursachen. Empirisch wird diese Fehlerseite später durch Eingriffe in zuvor erfolgreiche Verläufe (*Broken*) und den damit verbundenen Zusatzaufwand beobachtbar gemacht.

Die Wirkungen mehrerer Kontrollmechanismen sind aus ihren Einzelzwecken nicht ableitbar. Eine bessere Zustandsrepräsentation kann eine nachfolgende Transaktionsprüfung unterstützen. Sie kann aber ebenso falsche Sicherheit erzeugen oder den Agenten auf eine unvollständige Interpretation festlegen. Ob sich Mechanismen ergänzen, ersetzen oder behindern, ist daher eine empirische Frage. Diese Schlussfolgerung ist keine aus einer einzelnen Quelle übernommene Wirksamkeitsannahme, sondern folgt aus den gegensätzlichen Fehlermöglichkeiten unzureichender und übermäßiger Kontrolle. Kapitel 3 ordnet hierzu konkrete Gegenarbeiten ein. Kapitel 4 operationalisiert die getrennte Prüfung der Komponenten.

Zusammenfassend synthetisiert die Arbeit daraus zwei unterschiedliche Interventionspunkte. Ein transaktionaler Agent kann vor seiner Entscheidung an der Interpretation und Planung scheitern oder nach der Entscheidung eine ungeeignete Zustandsänderung zur Ausführung vorschlagen. Semantische Unterstützung vor der nativen Entscheidung und Pre-Write-Kontrolle vor der Ausführung sind daher zwei verschiedene Interventionsprinzipien. Sie unterscheiden sich in Informationsbedarf, Unsicherheit und Autorität. Aus ihrer theoretischen Trennung folgt noch keine Einzel- oder Kombinationswirkung (vgl. Qu et al. (2025), S. 5–11; vgl. Rinderle-Ma; Winter; Benzin (2023), Abschn. 2–3).

2.4 Evaluation agentischer Geschäftsprozesse

Konstrukt und Operationalisierung

Eine Metrik ist nicht mit dem theoretischen Konstrukt gleichzusetzen, das sie abbilden soll. Braggaar et al. zeigen für task-orientierte Dialogsysteme eine große Vielfalt von Konstrukten und Operationalisierungen. Identisch benannte Größen wie Qualität oder Effizienz werden in der Literatur teilweise durch unterschiedliche automatische und menschliche Verfahren gemessen (vgl. Braggaar et al. (2026), S. 4–5 und 12–13). Eine belastbare Evaluation muss

deshalb zunächst festlegen, welche Eigenschaft beurteilt wird, und anschließend begründen, inwieweit die gewählte Messgröße diese Eigenschaft erfasst.

Für die weitere Arbeit bilden die in Abschnitt 2.2 hergeleiteten Perspektiven *Effektivität*, *Zuverlässigkeit*, *Kontrollqualität* und *Effizienz* den einheitlichen Bewertungsrahmen. Endzustand, Prozessverlauf und wiederholte Ausführung sind keine konkurrierende zweite Taxonomie, sondern unterschiedliche Messobjekte innerhalb dieses Rahmens. Der Endzustand zeigt, ob das Kundenanliegen fachlich gelöst wurde. Der Prozessverlauf zeigt, ob erforderliche beziehungsweise unzulässige Schritte und Eingriffe auftraten. Wiederholungen zeigen, wie stabil diese Leistung ist. Kosten, Laufzeit, Tools und Nachrichten bilden den beobachtbaren Aufwand ab (vgl. Braggaa et al. (2026), S. 12–13; vgl. Ma et al. (2024), S. 1–3).

Die Messobjekte sind verbunden, aber nicht austauschbar. Ein richtiger Endzustand kann über einen problematischen Pfad entstanden sein, ein weitgehend korrekter Pfad an einer einzigen fehlenden Zustandsänderung scheitern und ein einmaliger Erfolg sich in einer Wiederholung nicht bestätigen. τ -bench stützt insbesondere die getrennte Beobachtung von Zielzustand, Aktionskriterien und wiederholtem Erfolg (vgl. Yao et al. (2025), S. 1–2).

Ausführbare Referenzgrundlage und wiederholter Erfolg

Sprachliche Ähnlichkeitsmaße, menschliche Beurteilungen und LLM-basierte Bewertungen können relevante Dialogmerkmale erfassen, bilden operative Zustandsänderungen jedoch nur indirekt ab. τ -bench verbindet interaktive Dialoge mit domänenspezifischen Tools, Policies und einem veränderbaren Datenbankzustand. Die Evaluation vergleicht den Zustand am Dialogende mit einem annotierten Zielzustand und ergänzt ihn um Aktionskriterien (vgl. Yao et al. (2025), S. 1–2 und 4–6). Für die Ergebnisqualität eines transaktionalen Prozesses entsteht dadurch eine ausführbare Referenzgrundlage (*Ground Truth*), die nicht von der Selbsteinschätzung des untersuchten Agenten abhängt.

Auch diese Referenzgrundlage ist eine Operationalisierung. Sie erfasst nur die im Benchmark kodierten Zustände und Kriterien. Nicht kodierte Aspekte wie Empathie, Verständlichkeit, wahrgenommene Fairness oder langfristige Kundenwirkung bleiben außerhalb der Referenzlösung. Ein Datenbankabgleich ist daher für die untersuchte operative Falllösung besonders passend, aber keine vollständige Messung von Servicequalität. Dialog- und Serviceforschung behandelt solche nutzer- und wahrnehmungsbezogenen Konstrukte ausdrücklich als eigenständige Qualitätsdimensionen (vgl. Braggaa et al. (2026), S. 12–13; vgl. Grönroos; Ojasalo (2004), S. 414–416).

Wiederholte Agentenausführungen können trotz identischer Aufgaben und Konfigurationen unterschiedliche Trajektorien und Ergebnisse hervorbringen. Ursachen liegen nicht nur in der Tokenauswahl, sondern in der verketteten Interaktion von Modellaufrufen, User-Simulator,

Toolresultaten und Folgeentscheidungen. Eine Temperatur von null konzentriert die Ausgabeverteilung und reduziert damit die Vielfalt der erzeugten Ausgaben. Sie garantiert in gehosteten LLM-Systemen jedoch keine vollständige Reproduzierbarkeit (vgl. Renze (2024), S. 7347–7353; vgl. Atıl et al. (2025), S. 135–142). Stochastische Variation bleibt deshalb eine Eigenschaft des beobachteten Gesamtsystems und darf nicht aus einem einzelnen Lauf oder allein aus der Temperatureinstellung abgeleitet werden.

Wiederholungen dienen in diesem Zusammenhang nicht dazu, die Zahl unabhängiger Aufgaben künstlich zu erhöhen. Sie beobachten vielmehr die Ausführungsstabilität derselben Aufgabe unter kontrollierten Bedingungen. Mittelwert und Streuung beantworten dabei unterschiedliche Fragen. Der Mittelwert beschreibt das durchschnittlich beobachtete Leistungsniveau, die Streuung dessen Empfindlichkeit gegenüber den realisierten Läufen. Die Verteilung über Wiederholungen sollte daher gemeinsam mit Punktschätzungen berichtet werden (vgl. Reimers; Gurevych (2017), S. 338–346; vgl. Agarwal et al. (2021), Abschn. 1–4).

Für verlässliche Prozessausführung genügt folglich ein einzelner Lauf nicht. Das aus der Codegenerierung stammende $\text{pass}@k$ misst die Wahrscheinlichkeit, dass mindestens einer von k Versuchen erfolgreich ist (vgl. Chen et al. (2021), Abschn. 2.1). τ -bench stellt dem das Zuverlässigkeitsmaß pass^k gegenüber. Eine Aufgabe gilt nur dann als zuverlässig gelöst, wenn alle k betrachteten Versuche erfolgreich sind. Daher steigt $\text{pass}@k$ mit k , während pass^k strenger wird und fällt (vgl. Yao et al. (2025), S. 4). Mehrere Ausführungen derselben Aufgabe bleiben Wiederholungen desselben Aufgabenkonstrukts. Die konkrete Seed-, Schätz- und Testlogik wird deshalb erst im experimentellen Design festgelegt.

Betriebliche Proxys und Entscheidungsgrenzen

Die in Abschnitt 2.2 eingeführten Leistungsdimensionen bilden den Bewertungsrahmen der weiteren Arbeit: Effektivität, Zuverlässigkeit, Kontrollqualität und Effizienz. Sie werden im Benchmark nicht direkt beobachtet, sondern über drei Messobjekte erschlossen. Erstens zeigt der Endzustand, ob das Kundenanliegen fachlich korrekt abgeschlossen wurde. Zweitens zeigt der Prozessverlauf, welche Schritte, Eingriffe und Fehlerpfade auf dem Weg zu diesem Endzustand auftreten. Drittens zeigen Wiederholungen, wie stabil dieselbe Aufgabe unter gleicher Konfiguration ausgeführt wird. Kosten, Laufzeit, Tool-Aufrufe und Nachrichten ergänzen diese drei Messobjekte als beobachtbare Aufwandsgrößen. Diese Zuordnung knüpft an die Forderung der Agentenevaluation an, neben Enderfolg auch Prozessfortschritt, Robustheit und Kosteneffizienz zu betrachten (vgl. Ma et al. (2024), S. 1–3; vgl. Yehudai et al. (2025), S. 1).

Die Messobjekte sind verbunden, aber nicht austauschbar. Ein richtiger Endzustand kann über einen fehleranfälligen oder unnötig teuren Prozess entstehen. Umgekehrt kompensiert eine kurze Laufzeit keine falsche Retoure. Ebenso zeigt eine einzelne erfolgreiche Ausführung

noch nicht, ob derselbe Fall bei Wiederholung stabil gelöst wird. Die Auswertung berichtet die Größen deshalb getrennt, statt sie zu einem gewichteten Gesamtscore zu verdichten. Ein solcher Score wäre ohne konkreten Prozesskontext willkürlich. BPM- und Serviceforschung begründen entsprechend die getrennte Betrachtung mehrerer Leistungsperspektiven und deren kontextspezifische Gewichtung (vgl. Van Looy; Shafagatova (2016), Abschn. 3–4; vgl. Grönroos; Ojasalo (2004), S. 414–416).

Die Übertragung auf Unternehmen bleibt begrenzt. Benchmarkkosten entsprechen nicht den Vollkosten eines Contact Centers. Die Nachrichtenzahl ist kein validiertes Maß für den Kundenaufwand. Aufgabenerfolg ist nicht automatisch eine Lösung beim ersten Kontakt (*First Contact Resolution*) oder Kundenzufriedenheit. Die Messgrößen erlauben eine vergleichende Aussage innerhalb der kontrollierten Systemumgebung. Reale Investitionsentscheidungen benötigen zusätzlich Prozessvolumen, Personal- und Fehlerfolgekosten, Service-Level und Kundenwahrnehmung. Diese Grenze folgt der Service- und Dialogforschung, die interne Ressourceneffizienz, Task-Erfolg und wahrgenommene Qualität als unterschiedliche Konstrukte behandelt (vgl. Grönroos; Ojasalo (2004), S. 414–416; vgl. Braggaar et al. (2026), S. 12–13).

Der theoretische Bezugsrahmen verbindet damit vier Ebenen: LLM-Agenten liefern flexibles, aber stochastisches Handeln. Transaktionaler Kundenservice verlangt die ausführbar korrekte Bindung von Nutzerziel, Evidenz, Prozesszustand und Zustandsänderung. Hybride Orchestrierung verteilt Verantwortung zwischen Modell und deterministischer Kontrolle. Die Evaluation unterscheidet Effektivität, Zuverlässigkeit, Kontrollqualität und Effizienz. Für die weitere Arbeit folgt daraus eine klare Prüflogik. Semantische Unterstützung darf nur helfen, wenn sie belegte Nutzerziele und Prozessinformationen zielrichtig verfügbar macht. Pre-Write-Kontrolle darf nur nützen, wenn sie riskante oder unbelegte Zustandsänderungen reduziert, ohne zu viele korrekte Prozesse zu stören. Die Kombination beider Komponenten ist deshalb nicht automatisch überlegen, sondern muss als eigene Gestaltungsalternative empirisch bewertet werden. Auf dieser Grundlage ordnet Kapitel 3 die vorhandenen Lösungsansätze und die verbleibende Forschungslücke ein.

3 Verwandte Arbeiten

Dieses Kapitel ordnet den Forschungsstand zur Laufzeitunterstützung und -kontrolle tool-nutzender LLM-Agenten ein. Die Literaturstudie fundiert den Problemraum, synthetisiert Mechanismen samt Evidenzgrenzen und prüft die Forschungslücke gegen direkte Gegenarbeiten. Damit verbindet sie theoretische Grundlagen, Gestaltungsanforderungen und Evaluation. Die Ordnung folgt ergebnisoffen Problem, Mechanismus und Wirkungsgrenze.

3.1 Methodisches Vorgehen der Literaturstudie

Die Recherche wurde als systematische Literaturstudie durchgeführt. Die Reviewauslegung orientiert sich an Tranfield et al., Okoli sowie Kitchenham und Charters (vgl. Tranfield; Denyer; Smart (2003), S. 208–215; vgl. Okoli (2015), S. 883–901; vgl. Kitchenham; Charters (2007), S. 3–20). Die Berichtsstandards Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 und PRISMA for Searching (PRISMA-S) leiten die transparente Berichterstattung (vgl. Page et al. (2021), S. 3–6; vgl. Rethlefsen et al. (2021), Abschn. 2–4). Wohlin begründet das ergänzende Snowballing, Webster und Watson die konzeptzentrierte Synthese (vgl. Wohlin (2014), S. 1–8; vgl. Webster; Watson (2002), S. xvi–xvii). Das Population, Intervention, Comparison, Outcome, Context (PICOC)-Schema strukturiert Population, Intervention, Vergleich, Ergebnis und Kontext der Suche. Das Vorgehen gliedert sich in die drei Hauptphasen (1) Planung, (2) Durchführung und (3) Analyse, die jeweils in nachvollziehbare Einzelschritte unterteilt werden. Tabelle 2 fasst den Ablauf zusammen. Die anschließenden Ausführungen dokumentieren seine konkrete Umsetzung.

Phase	Einzelschritte
(1) Planung	(i) Leitfragen und PICOC festlegen; (ii) Informationsquellen und Suchzeitraum bestimmen; (iii) Auswahlregeln und Evidenzstufen definieren; (iv) Extraktion und Synthese planen
(2) Durchführung	(i) Suchstrategie operationalisieren; (ii) Datenbanksuche und Vorverarbeitung durchführen; (iii) Titel, Abstracts und Volltexte prüfen; (iv) fokussiertes Snowballing durchführen; (v) Auswahlprozess dokumentieren
(3) Analyse	(i) analytische Rollen codieren; (ii) Studienqualität und Evidenzgewicht bewerten; (iii) Befunde konzeptzentriert synthetisieren; (iv) Forschungslücke gegen direkte Gegenarbeiten prüfen

Tabelle 2. Methodisches Vorgehen der Literaturstudie

(1) Planung

(i) In der Planungsphase wurden zunächst drei Leitfragen formuliert: Welche Zuverlässigkeits- und Prozessprobleme sind für toolnutzende LLM-Agenten belegt? Welche semantischen und

deterministischen Laufzeitmechanismen wurden entwickelt und wie wurden sie evaluiert? Welcher Gestaltungs- und Entscheidungsbedarf bleibt gegenüber den engsten Gegenarbeiten bestehen? Das PICOC-Schema grenzt dialogische, toolnutzende Agenten als Population, Laufzeitunterstützung und -kontrolle als Intervention, native oder deaktivierte Agenten als Vergleich sowie Geschäftszustand, Mutationsausführung, Prozessfehler, Wiederherstellung, Stabilität und Ressourcenaufwand als Ergebnisse ab. Der Kontext umfasst transaktionalen Kundenservice und unmittelbar vergleichbare mehrstufige Geschäftsprozesse.

(ii) Anschließend wurden Informationsquellen und Suchzeitraum bestimmt. Die Datenbank-suchen wurden auf die Veröffentlichungsjahre 2022 bis 2026 begrenzt. Als gemeinsame Publikationsgrenze wurde der 26. Juni 2026 festgelegt. Dieses Datum bezeichnet zugleich den für Auswahl und Synthese verwendeten Literaturstand. Die Recherche wurde in Scopus, ACL Anthology, arXiv, IEEE Xplore und der ACM Digital Library mit derselben Publikationsgrenze durchgeführt. Ältere methodische und theoretische Grundlagen wurden getrennt über Zitationsverfolgung berücksichtigt, begründen aber keine Neuheitsbehauptung für aktuelle Agentenmechanismen. Die Quellenwahl folgt dem interdisziplinären und zugleich schnelllebigen Charakter des Feldes. Scopus bildet die fachübergreifende bibliografische Basis, da die Forschungsfrage Arbeiten aus Informatik, Wirtschaftsinformatik und Management berührt. Die ACL Anthology ergänzt diese Breite durch einen kuratierten Fachkorpus der Computerlinguistik, in dem Dialogsysteme, Tool-Nutzung und agentische Ansätze fachspezifisch erschlossen werden. IEEE Xplore und die ACM Digital Library erschließen zusätzlich begutachtete Informatik- und Systempublikationen unmittelbar in den beiden fokalen Suchfamilien. arXiv dient demgegenüber als ergänzender Frühzugang zu einem jungen Forschungsfeld, in dem neue Architekturen, Benchmarks und Laufzeitmechanismen häufig bereits vor Abschluss regulärer Begutachtungsverfahren verfügbar sind. Die Kombination balanciert damit fachliche Breite, Spezifität und Aktualität, ohne den Publikationsort mit der Evidenzqualität gleichzusetzen. Insbesondere wird ein arXiv-Nachweis nicht als gleichwertiger Qualitätsnachweis behandelt. Der unterschiedliche Quellenstatus wird durch die nachgelagerte Qualitätsprüfung und die Evidenzklassen E1 bis E3 berücksichtigt (vgl. Rethlefsen et al. (2021), S. 3–5; vgl. Adams; Smart; Huff (2017), S. 432–435; vgl. Paez (2017), S. 233–236). (iii) Im nächsten Planungsschritt wurden die fachlichen Ein- und Ausschlussregeln sowie die Evidenzstufen vor dem Screening festgelegt. Da zentrale Arbeiten des jungen Feldes zunächst als Preprints erscheinen, wurde arXiv nicht pauschal ausgeschlossen. Vor Festschreibung der finalen Synthese wurde die bereits im Codierschema enthaltene Ausnahme für nicht regulär begutachtete Beiträge präzisiert. Dies veränderte weder Suchstrings noch bereits getroffene fachliche Ausschlüsse. Tabelle 3 fasst die für das Screening verbindliche Logik zusammen.

Kategorie	Kriterien
Basisein-schluss	Deutsch- oder englischsprachiger wissenschaftlicher Beitrag im Suchzeitraum; nachvollziehbare Methode; LLM-agentische Population; dialogische, toolbasierte, workflowbezogene oder zustandsverändernde Ausführung; substanzieller Problem-, Interventions- oder Evaluationsbeitrag
Regelfall	Regulär peer-reviewte Journal-, Konferenz- oder Findings-Publikation mit unmittelbarer Passung zu mindestens einer Syntheserolle
Ausnahme	Workshopbeitrag oder Preprint nur bei unmittelbarer Passung, prüfbarer Methode beziehungsweise prüfbaren Ergebnissen und eigenständiger, für die Synthese benötigter Erkenntnisfunktion; Preprints dürfen keine zentrale Aussage allein tragen
Ausschluss	Dublette oder Beitrag außerhalb des Suchzeitraums; fehlender Agenten-, Tool-, Zustands-, Workflow- oder Kontrollbezug; reine Textgenerierungs-, RAG-, Moderations- oder Sicherheitsarbeit ohne übertragbare Prozessausführung; fachlich nicht passende Volltextbefunde; nicht erfüllte Ausnahme für nicht regulär begutachtete Beiträge

Tabelle 3. Ein- und Ausschlusslogik der Literaturstudie

Für die Evidenzgewichtung werden drei Stufen unterschieden. *E1* bezeichnet regulär peer-reviewte Beiträge, die bei hinreichender Studienqualität zentrale Aussagen tragen dürfen. *E2* bezeichnet begutachtete Workshopbeiträge, deren Befunde bei zentralen Aussagen mit stärkerer Evidenz trianguliert werden. *E3* bezeichnet aktuelle Preprints. Sie ergänzen Frontier-Befunde, werden sichtbar gekennzeichnet und dürfen keine zentrale Aussage allein begründen. Damit werden Relevanz und Quellenstatus nicht gleichgesetzt. Die Einbeziehung grauer Literatur kann Publikationsverzögerungen reduzieren, erfordert wegen heterogener Qualität aber eine gesonderte und transparente Prüfung (vgl. Adams; Smart; Huff (2017), S. 432–435; vgl. Paez (2017), S. 233–236).

(iv) Abschließend wurden vor der Auswertung der Extraktionskern und die Syntheseform festgelegt. Erfasst wurden bibliografische Identität, Identifikationsweg, Publikationsstatus, Einschlussbegründung, analytische Rolle, Systembeschreibung, Vergleichsbedingung, Referenzgrundlage, Ergebnismaße und berichtete Validitätsgrenzen. Die Synthese wurde wegen unterschiedlicher Aufgaben, Modelle und Endpunkte qualitativ und konzeptzentriert geplant. Die Studien wurden daher inhaltlich nach gemeinsamen Konzepten und Befundmustern verglichen.

(2) Durchführung

(i) Zu Beginn der Durchführung wurden die Suchbegriffe aus PICOC abgeleitet und in zwei Suchfamilien gebündelt. Die erste adressierte Zuverlässigkeitsprobleme und Evaluati-

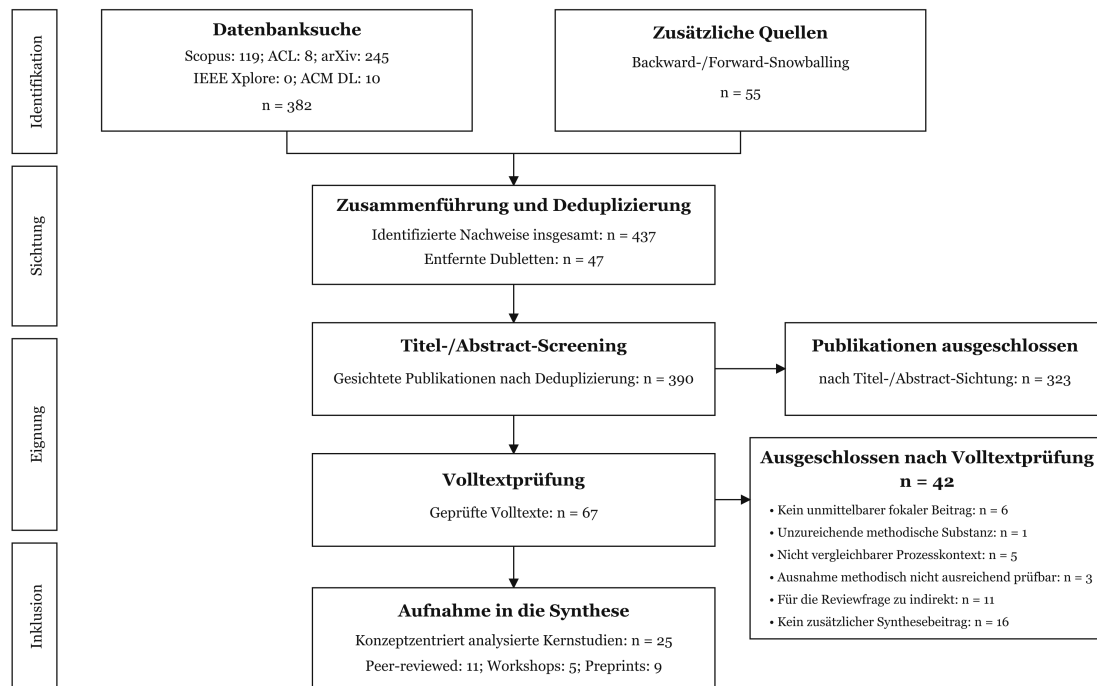
on, die zweite semantische Unterstützung, Laufzeitkontrolle, Workflows, Verifikation und transaktionale Ausführung. Die Begriffe wurden nur syntaktisch an die jeweilige Quelle angepasst. Suchrahmen, Auswahlregeln, finale PRISMA-Arithmetik und Evidenztabelle der auf den 26. Juni 2026 begrenzten Literaturrecherche sind im digitalen Forschungsanhang dokumentiert.

(ii) Auf dieser Grundlage wurden die Datenbanksuche und die ergänzende Quellsuche durchgeführt. Die Datenbanksuche identifizierte 119 Quellennachweise aus Scopus, acht aus der ACL Anthology, 245 aus arXiv, keinen aus IEEE Xplore und zehn aus der ACM Digital Library, insgesamt 382. Das fokussierte Backward- und Forward-Snowballing ergänzte 55 Quellennachweise. Damit umfasste die gemeinsame Identifikationsmenge 437 Nachweise. Die quellenübergreifende Deduplikation anhand von DOI, arXiv-ID und normiertem Titel entfernte 47 Mehrfachnachweise, sodass 390 eindeutige Publikationen in die inhaltliche Sichtung eingingen.

(iii) Alle 390 Publikationen wurden anhand der festgelegten PICOC- und Auswahlregeln auf Titel- und Abstractebene gesichtet. 323 Publikationen wurden in dieser Stufe ausgeschlossen. Die verbleibenden 67 Volltexte wurden vollständig gegen die fachlichen Rollen, die methodische Auswertbarkeit, den Prozesskontext und die Evidenzregeln geprüft. Davon wurden 42 mit genau einem primären Ausschlussgrund ausgeschlossen. Inhaltliche Ergebnisrichtungen waren zu keiner Zeit ein Auswahlkriterium. 25 Studien erfüllten sämtliche Aufnahmebedingungen und gingen in die konzeptzentrierte Synthese ein.

(iv) Das Snowballing wurde bewusst auf einen Rückwärts- und Vorwärtsdurchgang aus fünf regulär peer-reviewten Ankerstudien begrenzt. Die Anker deckten Kundenservice, ausführbare Referenzgrundlagen, Tool-Agent-Evaluation, Policy-Adhärenz und den Nutzen-Aufwands-Konflikt der Laufzeitverifikation ab. Alle 55 auf diesem Weg identifizierten Kandidaten durchliefen dieselben Auswahlregeln wie die Datenbanktreffer. Da der Durchgang keine weitere unmittelbar relevante Mechanismus-, Evaluations- oder Domänenklasse erschloss, war die vorab definierte Abbruchregel erfüllt und eine zweite Runde nicht erforderlich.

(v) Abschließend wurde der gesamte Auswahlprozess dokumentiert. Abbildung 2 stellt beide Auswahlpfade und ihre Zusammenführung dar. Der finale Synthesekorpus umfasst 25 Studien. Davon stammen 20 aus der Datenbanksuche und fünf aus dem Snowballing. Davon sind elf regulär peer-reviewt, fünf begutachtete Workshopbeiträge und neun aktuelle Preprints. Anhang A.3 weist alle 25 Studien einzeln mit Identifikationsweg, Evidenzstufe, analytischer Rolle und Synthesefunktion aus.



Quelle: vgl. Page et al. (2021), S. 3–6.

Abbildung 2. PRISMA-Flussdiagramm der Identifikation, Sichtung und Auswahl

Technische Hilfsmittel unterstützten Quellenabruf, Deduplikation, Volltextauffindung und die konsistente Dokumentation. Entscheidungen wurden auf Publikationsebene anhand des fixierten Codierschemas mit genau einem primären Ausschlussgrund gespeichert. Automatische Scores oder gewünschte Zielzahlen bestimmten keine Aufnahme. Die kanonische Studienmatrix, aggregierten Ausschlusszahlen, Qualitätsurteile und Claim-Evidence-Zuordnungen liegen im digitalen Forschungsanhang.

(3) Analyse

(i) In der Analysephase wurden zunächst alle 25 Studien mindestens einer analytischen Rolle zugeordnet: Problem- oder Fehlerbeleg (R1), semantische Unterstützung (R2), deterministische oder prüfende Laufzeitkontrolle (R3), Benchmark und Evaluation (R4) sowie Korrektur und Wiederherstellung (R5). Mehrfachzuordnungen sind zulässig, weil einzelne Arbeiten mehrere Erkenntnisfunktionen erfüllen. Die Rollen beschreiben konzeptionelle Abdeckung und keine disjunkten Studiengruppen. Tabelle 23 fasst diese Rollen und ihren Beitrag zur Arbeit zusammen.

(ii) Anschließend wurden Studienqualität und Evidenzgewicht getrennt bewertet. Die Qualitätsprüfung erfasste fünf Merkmale, ohne daraus einen scheinpräzisen Gesamttranz zu bilden. Analyseeinheit und Population, Systembeschreibung sowie Referenzgrundlage beziehungsweise Ergebnisdefinition waren bei allen 25 Arbeiten nachvollziehbar. 21 Studien

verfügten über eine angemessene Vergleichsbedingung. Bei vier war sie nur teilweise passend. Zehn Arbeiten berichteten Wiederholung, Unsicherheit oder offene Artefakte umfassend, 15 nur teilweise. Diese letzte Dimension begrenzt insbesondere Aussagen über Stabilität und Generalisierbarkeit. Publikationsstatus und Einzelurteile bleiben in der Qualitätsmatrix getrennt sichtbar.

(iii) Darauf aufbauend wurden die extrahierten Befunde konzeptzentriert synthetisiert. Die Synthese folgt Konzepten und widersprüchlichen Befunden statt einer Folge einzelner Paper-zusammenfassungen (vgl. Webster; Watson (2002), S. xvi–xvii). Wegen der unterschiedlichen Aufgabenpopulationen, Modelle, Interventionen und Zielgrößen wurden die Ergebnisse inhaltlich verglichen und nicht zu einer gemeinsamen statistischen Kennzahl verdichtet. Für jede zentrale Aussage weist eine Claim-Evidence-Matrix die tragende Evidenz nach Evidenzstufe, ergänzende Belege und die verbleibende Geltungsgrenze aus. Soweit keine direkte E1-Evidenz vorliegt, wird der Befund als begrenzte Prüfimplikation statt als allgemeine Wirkungsbehauptung formuliert.

(iv) Zum Abschluss der Analyse wurde die verbleibende Forschungslücke gegen die engsten Gegenarbeiten geprüft. Dazu wurden die Arbeiten danach verglichen, ob sie semantische Unterstützung, deterministische Pre-Write-Kontrolle, eine unabhängige Faktortrennung, gleiche Informations- und Bewertungsbedingungen sowie eine gemeinsame Nutzen-, Schaden-, Wiederherstellungs- und Aufwandsbewertung abdecken. Eine Forschungslücke wurde damit nicht aus der bloßen Abwesenheit einer identischen Architektur, sondern aus der systematischen Gegenprüfung dieser Merkmalskombination abgeleitet. Die Review besitzt dennoch Grenzen. Die Auswahl und Codierung erfolgten in einem Ein-Reviewer-Verfahren, die Suchsyntax musste quellspezifisch angepasst werden und ein Teil der unmittelbaren Mechanismenliteratur liegt nur als Workshopbeitrag oder Preprint vor. Daher wird weder vollständige Literatursättigung noch universelle Architekturneuheit beansprucht. Die folgende Synthese prüft stattdessen, welche eng abgegrenzte Lücke innerhalb des identifizierten Evidenzstands bestehen bleibt.

3.2 Konzeptzentrierte Synthese des Forschungsstands

Die Synthese verbindet die fünf codierten Rollen zu vier Erkenntniszusammenhängen: Problem und Referenzgrundlage, semantische Unterstützung, deterministische Laufzeitkontrolle sowie Korrekturpfad, Stabilität und Aufwand. In jedem Zusammenhang werden Nutzenbefunde und ihre Grenzen gemeinsam betrachtet. Dadurch entsteht keine additive Liste technischer Funktionen, sondern eine Argumentationskette von beobachteten Fehlern über mögliche Eingriffspunkte bis zu den Anforderungen an einen fairen Komponentenvergleich.

Problemraum und ausführbare Referenzgrundlage

Für den Kundenservice ist der Problemraum fachlich und betrieblich direkt belegt. Beyond IVR macht neben Aufgabenlösung auch Business-Adherence und operative Ablaufkonformität zum Bewertungsgegenstand. CRMArena bildet realistische Customer Relationship Management (CRM)-Objekte, Rollen und mehrstufige Aufgaben ab (vgl. Balaji et al. (2026), S. 193–208; vgl. Huang et al. (2025), Abschn. 3–5). Die Workshopstudie ALMITA zeigt ergänzend, dass gute Einzelturn- und Function-Calling-Ergebnisse nicht notwendig auf vollständige Supportdialoge übertragbar sind (vgl. Arcadinho; Aparicio; Almeida (2024), Abschn. 3–5). Die Domäne ist damit nicht nur ein Anschauungsbeispiel. Ihre Verbindung aus Dialog, operativen Regeln, Tools und persistentem Geschäftszustand erzeugt genau die mehrstufige Ausführung, für die Laufzeitinterventionen relevant werden.

Die Benchmarkliteratur zeigt zugleich, warum sprachliche Plausibilität kein hinreichendes Erfolgskriterium ist. τ -bench verbindet Dialog, Policies, Tools und einen veränderbaren Datenbankzustand. τ^2 -bench ergänzt eine Dual-Control-Umgebung, in der Agent und Nutzer gemeinsam auf den Weltzustand einwirken (vgl. Yao et al. (2025), Abschn. 3–5; vgl. Barres et al. (2026), Abschn. 3–5). ToolSandbox prüft dynamische Zwischen- und Endmeilensteine in zustandsabhängigen Dialogen, während SOPBench neben dem Datenbankende auch Funktionszulässigkeit und Verfahrensvollständigkeit mit ausführbaren Oracles erfasst (vgl. Lu et al. (2025), Abschn. 3–5; vgl. Li et al. (2025), Abschn. 3–6). Effective Red-Teaming belegt darüber hinaus, dass Policy-Adhärenz gezielt unter adversarialen Varianten geprüft werden muss (vgl. Nakash et al. (2025), Abschn. 3–5).

fastWorkflow verbindet diesen Problem- und Evaluationsstrang mit einer modularen Laufzeitarchitektur. Die Arbeit leitet aus einer Fehlerklassifikation eine kaskadierte Intent- und Parameterextraktion, hierarchische Kontextrepräsentationen sowie adaptive Planung ab und prüft die Komponenten in Ablationen auf den Retail- und Airline-Aufgaben von τ -bench. Drei unabhängige Durchläufe, Kostenanalysen und veröffentlichte Traces machen dabei auch den Abstand zwischen mittlerem Erfolg und wiederholungsstabiler Leistung sichtbar (vgl. Satija et al. (2026), Abschn. 3–5 und Anhang B.3–B.5).

Aus diesen Arbeiten folgen drei nicht austauschbare Messebenen. Das *Ergebnisoracle* prüft den geforderten Geschäftszustand. Das *Prozessoracle* prüft erforderliche, verbotene oder sequenzabhängige Aktionen. Die *Zuverlässigkeitsebene* prüft, ob der beobachtete Erfolg über wiederholte Ausführungen und Störungen stabil bleibt. Ein korrekter Endzustand kann einen regelwidrigen Pfad verdecken. Umgekehrt beweist eine abgefangene Regelverletzung noch keinen erfolgreichen Abschluss (vgl. Yao et al. (2025), Abschn. 3–5; vgl. Sah et al. (2026), Abschn. 3–5; vgl. Gupta (2026), Abschn. 3–5).

Aktuelle ergänzende Evidenz schärft diese Grenze, ohne sie allein zu tragen. AgentChangeBench unterscheidet bei Zieländerungen Aufgabenerfolg, Redundanz, Effizienz und Wiederherstellungszeit. ReliabilityBench erweitert die Einzelausführung um wiederholte Belastungs- und Störungsfälle. Catching One in Five berichtet für produktive Transaktionsdialoge strukturierte Blindstellen automatisierter LLM-Judges, insbesondere bei turnübergreifendem Zustand (vgl. Rana et al. (2025), Abschn. 3–5; vgl. Gupta (2026), Abschn. 3–5; vgl. Zhang; Wang; Lei (2026), Abschn. 4–7). Die Arbeiten begründen damit die getrennte Beobachtung von Ergebnis, Prozess und Streuung. Als Workshop- beziehungsweise Preprint-Evidenz begründen sie jedoch keine populationsweite Fehlerhäufigkeit.

Auch eine ausführbare Referenzgrundlage bildet nur die formalisierten Ziele und Regeln des jeweiligen Benchmarks ab. Empathie, langfristige Kundenwirkung, Organisationskosten oder Kundenzufriedenheit werden nicht unmittelbar gemessen. Erfolgs-, Fehler-, Latenz- und Kostenmaße sind deshalb betriebliche Näherungsgrößen, keine direkten Nachweise von Unternehmensrendite oder Servicezufriedenheit. Für eine Einsatzentscheidung bleiben sie dennoch relevant, weil korrekte Fallbearbeitung, Fehlerfolgen, Wiederherstellung und Ressourcenverbrauch gemeinsam abgewogen werden müssen (vgl. Braggaar et al. (2026), S. 12–13; vgl. Grönroos; Ojasalo (2004), S. 414–416).

Semantische Unterstützung

Semantische Unterstützung greift vor einer konkreten Ausführungsentscheidung ein. Sie externalisiert oder strukturiert Ziele, Policies, Evidenz und Fortschritt, ohne selbst eine Schreibaktion zu autorisieren. Flexible Agent Alignment modelliert aus offenem Dialog dynamisch aktualisierte Zielhypothesen. PolicyBank führt turnbezogene Policy-Informationen über längere Interaktionen fort, und Push Your Agent verknüpft quantitative Ziele mit einem verifizierten Fortschrittszustand (vgl. Ma et al. (2025), Abschn. 4–6; vgl. Choi et al. (2026), Abschn. 3 und 6; vgl. Cai et al. (2026), Abschn. 3–7). Diese Mechanismen stützen die Annahme, dass eine explizite, aktualisierbare Repräsentation gegenüber einem ausschließlich impliziten Dialogzustand nützlich sein kann. Die direkte Evidenz verteilt sich jedoch auf unterschiedliche Aufgaben, Repräsentationen und Publikationsstufen.

τ -Knowledge trennt den Zugang zu unstrukturierter Evidenz von deren Nutzung. Selbst bereitgestellte Gold-Dokumente beseitigen Reasoningfehler nicht vollständig. Stärkere Suche verursacht zugleich zusätzliche Calls und Latenz (vgl. Shi et al. (2026), Abschn. 4–6). Die Preprints zu Policy-Signal und Environmental Evidence ergänzen, dass hohe Retrievalkennzahlen nicht automatisch entscheidungsrelevante Policy-Signale liefern und dass Herkunft, Aktualität und Autorität beobachteter Evidenz geprüft werden müssen (vgl. Ding; De la Cruz Weinstein (2026), Abschn. 2–4; vgl. Sheng; Wang; Zhou (2026), Abschn. 3–5).

Die Synthese begründet daher keinen generellen Haupteffekt semantischer Unterstützung. Ihre Wirkung hängt mindestens von Informationsparität, Retrievalqualität, Repräsentationsform, Modell und Aufgabe ab. Für einen kausalen Komponentenvergleich ist besonders wichtig, dass die unterstützte Bedingung keine zusätzlichen fachlichen Fakten erhält. Andernfalls wäre nicht die Strukturierung, sondern ein Wissensvorsprung die alternative Erklärung. Komponente A dieser Arbeit darf deshalb nur bereits sichtbare Informationen in eine persistente Ziel-, Evidenz- und Fortschrittsstruktur überführen.

Deterministische Laufzeitkontrolle an der Ausführungsgrenze

Deterministische Laufzeitkontrolle greift an einem anderen Interventionspunkt ein. Sie bewertet eine bereits vorgeschlagene Aktion unmittelbar vor ihrer Zustandswirkung. Towards Enforcing Company Policy Adherence prüft Toolaktionen gegen explizite Policybedingungen. Agent-C kompiliert temporale Regeln über Aktionsfolgen (vgl. Zwerdling et al. (2025), Abschn. 3–6; vgl. Kamath et al. (2026), Abschn. 3–6). Solver-Aided Verification kombiniert semantische Faktextraktion mit symbolischer Prüfung, während Cordon mehrere Toolwirkungen in einem transaktionalen Staging- und Commitrahmen bündelt (vgl. Winston; Winston; Just (2026), Abschn. 3–5; vgl. Chen et al. (2026), Abschn. 3–6). Gemeinsam zeigen diese Arbeiten, dass explizite Prädikate definierte Verstöße vor einer irreversiblen Wirkung erkennen oder zurückhalten können.

Die Kontrollwirkung besitzt jedoch eine klare Geltungsgrenze. Sie gilt nur für formalisierte Prädikate, verfügbare Evidenz und korrekt instrumentierte Tools. Unvollständige Policyabdeckung oder falsch interpretierte Fakten erzeugen falsche Freigaben, falsche Blockierungen oder einen ungelösten Evidenzbedarf. Eine erfolgreiche Blockierung ist deshalb nicht mit Aufgabenerfolg gleichzusetzen. Der regulär peer-reviewte Verifier-Tax-Befund zeigt in transaktionalen τ -bench-Szenarien, dass ein hoher Anteil abgefangener nicht konformer Aktionen nicht automatisch in sicheren Aufgabenerfolg übergeht. Die Wiederherstellung nach Intervention bleibt häufig gering und der Ausführungsaufwand steigt (vgl. Sah et al. (2026), Abschn. 3–5).

SABER bildet die engste direkte Evidenz zur Kombination unterschiedlicher Kontrollprinzipien. In einer Ablation auf τ -Bench Verified steigt die Airline-Erfolgsrate von 58,0 % ohne die untersuchten SABER-Komponenten auf 68,0 % mit Reflection, 68,7 % mit Verification und 78,7 % mit beiden. Im Retail-Teil stehen dagegen 66,9 %, 80,8 %, 80,5 % und 77,7 %: Die Kombination liegt dort hinter beiden Einzelbedingungen (vgl. Cuadron et al. (2025), Abschn. 5–8 sowie Anhang). Da eine weitere Kontextbereinigung in allen Bedingungen aktiv bleibt und die Arbeit als Workshopbeitrag vorliegt, beweist dieser Befund keine allgemeine negative Interaktion. Er widerlegt aber eine unbedingte Additivitätsannahme und begründet die Notwendigkeit, Einzel- und Kombinationseffekte getrennt zu prüfen.

Wiederherstellung, Stabilität und betrieblicher Aufwand

Eine Intervention ist erst dann betrieblich nützlich, wenn sie nicht nur einen Fehler erkennt, sondern einen korrekten weiteren Verlauf ermöglicht. Hell or High Water isoliert externe Toolausfälle bei weiterhin lösbaren Aufgaben und zeigt, dass Agenten trotz verfügbarer Alternativpfade häufig keinen tragfähigen Ersatzplan ausführen (vgl. Wang et al. (2025), Abschn. 3–5). Der Verifier-Tax-Befund bestätigt diese Wiederherstellungslücke nach blockierten Aktionen. Cordon ergänzt Staging, Commit und Rollback als technische Schadenbegrenzung (vgl. Sah et al. (2026), Abschn. 3–5; vgl. Chen et al. (2026), Abschn. 3–6). Blockierung, Neuplanung und final korrekter Geschäftszustand sind folglich getrennte Ergebnisse.

Die Wiederholungsdimension erweitert diese Sicht. Ein Lauf kann aufgrund stochastischer Modellvariation erfolgreich sein, ohne dass der Mechanismus stabil wirkt. AgentChangeBench und ReliabilityBench motivieren daher die gemeinsame Beobachtung von Wiederherstellungszeit, Redundanz, Störungsrobustheit und Streuung über Wiederholungen (vgl. Rana et al. (2025), Abschn. 3–5; vgl. Gupta (2026), Abschn. 3–5). Für einen Komponentenvergleich reichen damit weder ein einzelner Lauf noch allein aggregierte Erfolgsraten. Gepaarte Übergänge müssen sichtbar machen, welche nativen Fehler eine Intervention korrigiert und welche nativen Erfolge sie beschädigt.

Aus der Synthese folgt ein betrieblicher Zielkonflikt. Semantische Unterstützung und deterministische Laufzeitkontrolle können Fehler reduzieren, zugleich aber Modellaufrufe, Tokens, Latenz, unnötige Toolnutzung, Fehlblockierungen oder zusätzliche Turns verursachen. Verifier Tax, τ -Knowledge und SABER berichten jeweils Teile dieses Konflikts, aber keine Arbeit verbindet im identifizierten Korpus semantische Unterstützung und deterministische Pre-Write-Kontrolle als unabhängig schaltbare Faktoren mit Ergebnisqualität, Prozessintegrität, Wiederherstellung, Regressionen und Aufwand in einer gemeinsamen Entscheidungsmatrix (vgl. Sah et al. (2026), Abschn. 3–5; vgl. Shi et al. (2026), Abschn. 4–7; vgl. Cuadron et al. (2025), Abschn. 4–6).

Die Evidenz begründet damit keine pauschale Überlegenheit zusätzlicher Laufzeitlogik. Belastbar ist vielmehr die Notwendigkeit, Nutzen, Schaden und Aufwand unter kontrollierten Bedingungen gemeinsam zu prüfen.

Tabelle 4 verdichtet die fünf Befundlinien und hält die jeweilige Geltungsgrenze unmittelbar neben der stärksten Evidenz fest.

Befundlinie	Stärkste Evidenz im Korpus	Geltungsgrenze
Ergebnis und Prozess müssen getrennt beobachtet werden	<i>τ-bench, ToolSandbox, CRMarena, Beyond IVR</i>	Formalisierte Benchmarkregeln bilden keine Kundenzufriedenheit oder Rendite ab
Explizite Ziele und Evidenz können Planung stützen	<i>τ-Knowledge, fastWorkflow, PolicyBank; ergänzend Goal Inference und Push Your Agent</i>	Wirkung abhängig von Informationsparität, Retrieval, Modell und Aufgabe
Pre-Write-Kontrolle kann definierte Verstöße verhindern	<i>ToolGuard, Verifier Tax; ergänzend Agent-C und SABER</i>	Begrenzung durch Prädikate, Evidenz und Wiederherstellungsfähigkeit
Blockierung ist kein sicherer Abschluss	<i>Verifier Tax, Hell or High Water, τ^2-bench</i>	Wiederherstellung variiert nach Fehlerart, Horizont und Alternativpfad
Additivität kann nicht vorausgesetzt werden	<i>SABER</i>	Direkte Interaktionsevidenz ist auf einen Workshopbefund begrenzt

Tabelle 4. Zentrale Befundlinien und ihre Geltungsgrenzen

3.3 Gegenarbeiten, Forschungslücke und Gestaltungsfolgen

Falsifikation einer Komponentenneuheit

Semantische Ziel- und Policyrepräsentationen, deterministische Aktionsprüfung, transaktionale Vorbereitung und Wiederherstellung existieren bereits. Die Forschungslücke betrifft daher nicht ihre bloße Kombination, sondern zurechenbares Gestaltungswissen zu ihrer Auswahl und gemeinsamen Bewertung (vgl. Choi et al. (2026), Abschn. 3 und 6; vgl. Zwerdling et al. (2025), Abschn. 3–6; vgl. Chen et al. (2026), Abschn. 3–6; vgl. Cuadron et al. (2025), Abschn. 5–8).

Tabelle 5 vergleicht dazu die engsten Gegenarbeiten aus dem finalen Korpus. K1 und K2 bezeichnen die beiden fokalen Interventionsprinzipien: semantische Unterstützung vor der nativen Entscheidung und deterministische Kontrolle vor der Ausführung einer Schreibaktion. K3 ist nur vollständig erfüllt, wenn beide Prinzipien im selben Versuchsdesign unabhängig aktiviert werden können. K4 verlangt für die Vergleichsbedingungen dieselben Aufgaben, denselben zulässigen Informationsbestand und dieselbe externe Bewertungsreferenz. K5 setzt voraus, dass Erfolg, Fehlerfolgen beziehungsweise Regressionen, Wiederherstellung und Aufwand gemeinsam betrachtet werden. Ein Haken bezeichnet die vollständige Erfüllung dieser Regel, ein Dreieck eine nur teilweise oder mit weiteren Mechanismen gekoppelte Abdeckung. Die Codierung beruht auf den Volltexten der Arbeiten und prüft ausschließlich deren Passung zur verbleibenden Forschungsfrage. Sie ist weder Qualitätsranking noch allgemeine Leistungsbewertung.

Arbeit	K1	K2	K3	K4	K5
PolicyBank	✓	×	×	△	△
τ -Knowledge	✓	×	×	✓	△
fastWorkflow	✓	△	×	✓	△
Agent-C	×	✓	×	✓	△
ToolGuard	×	✓	×	✓	△
Cordon	×	✓	×	△	✓
Verifier Tax	×	△	×	✓	✓
SABER	△	△	△	✓	△
Diese Arbeit	✓	✓	✓	✓	✓

✓ = vollständig abgedeckt; △ = teilweise oder gekoppelt abgedeckt; × = nicht abgedeckt.

Tabelle 5. Konzeptmatrix der engsten Gegenarbeiten und des Evaluationsdesigns

Die letzte Zeile bezeichnet den vorgesehenen Abdeckungsumfang des eigenen Evaluationsdesigns. Sie behauptet weder die Wirksamkeit der Komponenten noch eine qualitative Überlegenheit gegenüber den Gegenarbeiten. Die Einzelcodierung ist im digitalen Forschungsanhang mit Begründung und Fundstelle dokumentiert.

SABER setzt die engste empirische Neuheitsgrenze, weil es im selben transaktionalen Benchmarkökosystem Reflexion, mutationsgebundene Verifikation und ihre Kombination untersucht (vgl. Cuadron et al. (2025), Abschn. 5–8). Verifier Tax ist die stärkste Gegenarbeit zur betrieblichen Nutzen-Aufwands-Frage, weil Interventionswirkung, sicherer Erfolg, Wiederherstellung und zusätzlicher Aufwand getrennt werden (vgl. Sah et al. (2026), Abschn. 3–5). ToolGuard und Agent-C widerlegen eine Neuheit deterministischer Laufzeitprüfung. Policy-Bank und τ -Knowledge und fastWorkflow widerlegen eine Neuheit strukturierter semantischer Unterstützung. fastWorkflow zeigt darüber hinaus, dass modulare Teilmechanismen in einem gemeinsamen Benchmark durch Ablationen untersuchbar sind, und enthält deterministische Parameter- und Kontextvalidierung. Es untersucht jedoch keine als eigenständiger Faktor operationalisierte, policy- und evidenzgebundene Kontrolle mutierender Schreibkandidaten (vgl. Zwerdling et al. (2025), Abschn. 3–5; vgl. Kamath et al. (2026), Abschn. 4–7; vgl. Choi et al. (2026), Abschn. 3 und 6; vgl. Shi et al. (2026), Abschn. 4–7; vgl. Satija et al. (2026), Abschn. 4–5 und Anhang B).

Verbleibende Design- und Entscheidungslücke

Aus der vorstehenden Synthese verbleibt innerhalb des systematisch identifizierten Evidenzstands keine Architekturneuheitslücke, sondern eine Design- und Entscheidungslücke. Es fehlt belastbares Wissen darüber, unter welchen Aufgaben- und Interventionsbedingungen eine

semantisch beratende und eine deterministisch autoritative Laufzeitintervention einzeln oder gemeinsam einen positiven Nettonutzen für transaktionale Agentenprozesse erzeugen. Die Literatur zeigt sowohl Nutzenpotenzial als auch Fehlblockierungen, geringe Wiederherstellung und zusätzlichen Aufwand. Wegen gekoppelter Interventionen, unterschiedlicher Informationsstände und heterogener Endpunkte sind diese Befunde aber nur begrenzt zurechenbar (vgl. Sah et al. (2026), Abschn. 3–5; vgl. Shi et al. (2026), Abschn. 4–7; vgl. Cuadron et al. (2025), Abschn. 5–8 sowie Anhang).

Für den nachfolgenden Vergleich bezeichnet *gleiche Informationsgrundlage*, dass alle Bedingungen auf denselben fachlichen Informationsbestand zugreifen und sich nur dessen zulässige Aufbereitung unterscheidet. *Gleiche Bewertungsgrundlage* bedeutet, dass dieselbe vorab festgelegte Ergebnis- und Prozessreferenz alle Bedingungen bewertet und nicht durch die jeweilige Intervention verändert wird. Damit werden Wissensvorteile und verschobene Bewertungsmaßstäbe als alternative Erklärungen begrenzt.

Obwohl semantische Unterstützungsmechanismen und deterministische Laufzeitkontrollen bereits einzeln und in hybriden Systemen untersucht wurden, fehlt im identifizierten Evidenzstand ein informations- und bewertungsparitätischer Komponentenvergleich, der ihre Einzel- und Kombinationswirkungen auf Aufgabenerfolg, Prozessintegrität, Wiederherstellung, Regressionen, Stabilität und Aufwand gemeinsam bestimmt.

Damit ist die in Kapitel 1 formulierte Forschungsfrage aus dem identifizierten Evidenzstand hergeleitet.

Folgerung für Artefakt und Evaluation

Die Faktoren folgen aus zwei Interventionspunkten. Komponente A strukturiert Ziele, Evidenz und Prozessfortschritt vor der Agentenentscheidung semantisch, ohne Blockierungsautorität. Komponente B prüft den mutierenden Toolkandidaten vor seiner Ausführung anhand spezifizierbarer Prädikate, ohne die Nutzerintention neu zu interpretieren. Unterschiedlicher Informationsbedarf, Unsicherheit und Autorität begründen ihre Trennung im faktoriellen Design.

Das Artefakt operationalisiert beide Faktoren bei identischer fachlicher Informations- und externer Bewertungsgrundlage und macht sie unabhängig schaltbar. Kapitel 4 überführt sie in C0 Native, C1 Semantische Unterstützung, C2 Pre-Write-Kontrolle und C3 Kombination. Die Evaluation berichtet Ergebnis- und Prozesskorrektheit, gepaarte Fehlerübergänge, Wiederherstellung, Streuung und inkrementellen Ressourcenaufwand. Damit prüft sie keine vorab behauptete Architekturüberlegenheit, sondern erzeugt zurechenbares Gestaltungswissen zu Nutzen, Schaden und Aufwand beider Eingriffe.

4 Methodik

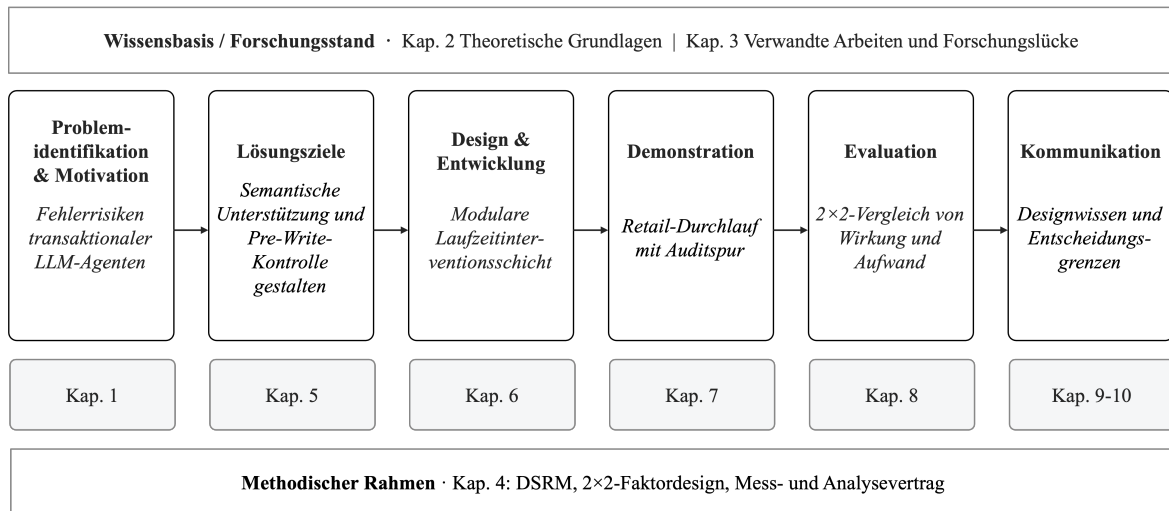
Dieses Kapitel verbindet die DSR-Entwicklung der in Kapitel 6 spezifizierten Laufzeitinterventionsschicht mit dem faktoriellen Vergleich auf 80 Retail-Aufgaben von τ^2 -bench. Derselbe Agent bearbeitet sie mit sechs Wiederholungen unter vier Bedingungen. Der native Agent bleibt die Referenz, Referenzlösungen bleiben dem Laufzeitpfad verborgen und technische Funktionsprüfung wird von empirischer Wirksamkeit getrennt. Bewertet werden Geschäftsendzustand, Schreibaktionen, Regressionen, Mechanismen und Aufwand.

4.1 Forschungsdesign und Design-Science-Prozess

Erkenntnislogik

Design Science Research (DSR) eignet sich für die Problemstellung, weil ein relevantes organisationales Problem durch ein gestaltetes Artefakt adressiert und dessen Nutzen systematisch geprüft wird (vgl. Hevner et al. (2004), S. 75–86). Die Build-and-Evaluate-Logik verbindet Konstruktion und empirische Prüfung (vgl. March; Smith (1995), S. 253–258). Als Prozessrahmen dient die Design Science Research Methodology (DSRM) nach Peffers et al. mit Problemidentifikation, Zieldefinition, Design und Entwicklung, Demonstration, Evaluation und Kommunikation (vgl. Peffers et al. (2007), S. 52–56).

Der primäre Artefaktbeitrag ist die Laufzeitinterventionsschicht mit zwei unabhängig schaltbaren Mechanismen: semantischer Unterstützung und Kontrolle vor Schreibaktionen (vgl. March; Smith (1995), S. 256–258). Dieses Artefakt wird vor der faktoriellen Evaluation technisch eingefroren. Davon getrennt ordnet eine nachgelagerte analytische Erweiterung die beobachteten Qualitäts-, Regressions- und Ressourcenprofile als betriebliche Konfigurationsalternativen ein. Sie gehört weder zum Laufzeitpfad noch zu den faktoriellen Interventionen und kann deren primäre Endpunkte nicht verändern. Das daraus gewonnene Designwissen betrifft damit neben der technischen Gestaltung auch die Frage, unter welchen Annahmen zusätzlicher Kontroll- oder Unterstützungsaufwand vertretbar erscheint. Abbildung 3 ordnet diese Schritte den Aktivitäten der Design-Science-Research-Methodology zu.



Quelle: vgl. Peffers et al. (2007), S. 52–56.

Abbildung 3. Zuordnung der DSRM-Aktivitäten zur Arbeit

Evaluationsfolge

Das *Framework for Evaluation in Design Science Research* verlangt eine zu Artefaktrisiko, Reifegrad und Erkenntnisziel passende Evaluationsfolge (vgl. Venable; Pries-Heje; Baskerville (2016), S. 80–83). Deshalb gingen der finalen Evaluation die Analyse nativer Fehlermuster, formative Designiterationen und automatisierte Tests von Schnittstellen, Autoritätsgrenzen und Laufzeit Spuren voraus. Danach wurden Implementierung, Konfiguration, Modell, Aufgaben, Seeds, Endpunkte und Analysegrenzen fixiert. Die abschließende 2×2 -Evaluation trennt damit technische Funktionsfähigkeit von den Ergebnis- und Aufwandsfolgen der unveränderten Bedingungen (vgl. Peffers et al. (2007), S. 55–56; vgl. Venable; Pries-Heje; Baskerville (2016), S. 78–83).

4.2 Testumgebung und Stichprobe

Herkunft und Auswahl des Benchmarks

Der Prüfstand muss die aus Kapitel 5 abgeleiteten Konstrukte beobachtbar machen: mehrstufigen Dialog mit verborgenen Nutzerzielen, verbindliche Prozessregeln, typisierte Lese- und Schreibwerkzeuge, einen veränderbaren Geschäftszustand und eine getrennte Referenzlösung. Erst diese Verbindung erlaubt, Prozess- und Ergebnisintegrität sowie Interventionsaufwand gemeinsam zu prüfen.

τ -bench bezeichnet das ursprüngliche Benchmarkkonzept. Es verbindet einen simulierten Nutzer mit einem Agenten, einer öffentlichen Policy, typisierten Werkzeugen und einer veränderbaren Geschäftsdatenbank (vgl. Yao et al. (2025), Abschn. 3–5). τ^2 -bench entwickelt diese technische Umgebung weiter und ergänzt Domänen, in denen Nutzer und Agent

Werkzeuge ausführen können (vgl. Barres et al. (2026), Abschn. 3–4). Die vorliegende Arbeit verwendet den lokal fixierten Tag v1.0.0 dieser Implementierung, beschränkt die Evaluation jedoch auf die darin fortgeführte Retail-Domäne.

Retail ist als Single-Control-Domäne konfiguriert. Der User-Simulator erzeugt ausschließlich Dialogbeiträge. Nur der Agent kann Werkzeuge aufrufen und den Geschäftszustand verändern. Dadurch liegt die untersuchte Ausführungsautorität eindeutig im Agentenpfad. Die Wirkungen der beiden Interventionen lassen sich somit an derselben Entscheidungs- und Schreibgrenze vergleichen, ohne dass Werkzeughandlungen des simulierten Nutzers eine zweite Ausführungsautorität in das faktorielle Design einführen.

Konstruktion und Episodenlogik

Eine Retail-Aufgabe besteht aus einer privaten Instruktion für den User-Simulator, einem initialen Datenbankzustand und einer ausführbaren Referenz der erwarteten Zustandsänderungen. Die private Instruktion enthält Ziel, Präferenzen und fallbezogene Angaben des simulierten Nutzers. Der Agent erhält sie nicht direkt. Er sieht nur die im Dialog geäußerten Informationen, die öffentliche Policy, Werkzeugbeschreibungen und Ergebnisse eigener Leseaufrufe. Schreibwerkzeuge verändern den gemeinsamen Retail-Zustand. Nach Episodenabschluss kann ein separater Evaluator den resultierenden Zustand und die protokollierten Aktionen mit der Referenz vergleichen (vgl. Yao et al. (2025), Abschn. 3–5).

Damit strukturiert Komponente A ausschließlich sichtbare Nutzeräußerungen, während Komponente B sichtbare Policy-, Argument- und Zustandsprädikate prüft. Private Aufgabenbeschreibung und Referenzlösung bleiben verborgen. Natürliche Eingriffsgelegenheiten entstehen deshalb ohne zusätzliches Lösungswissen. Die Kombination aus verborgenem Nutzerziel, einheitlicher Informationsgrundlage, ausführbaren Zustandsänderungen und getrennter Offline-Referenz macht die Umgebung für die Untersuchung geeignet. Sie bildet sowohl die semantische Entscheidung vor einer Aktion als auch deren operative Wirkung ab.

Aufgabenpopulation und Entwicklungsgrenze

Der gepinnte Retail-Bestand enthält 114 Aufgaben. Die finale Evaluationspopulation umfasst 80 vor der jeweiligen Ausführung festgelegte Aufgaben: den vollständigen offiziellen 40-Aufgaben-Testsplit sowie zwei disjunkte Erweiterungsblöcke mit jeweils 20 bis dahin nicht in der fokalen Evaluation verwendeten Retail-Aufgaben. Ihre Vereinigung ist in einem maschinenlesbaren Evaluationssplit fixiert. Die übrigen 34 Aufgaben bilden den davon disjunkten Entwicklungsbestand für Komponenten- und Integrationsprüfungen. Die Bezeichnung *Evaluationspopulation* meint damit ausdrücklich einen festgelegten Ausschnitt des Retail-Bestands und nicht den unveränderten offiziellen Testsplit oder eine Zufallsstichprobe realer Kundenkontakte.

Alle 80 Evaluationsaufgaben werden unter derselben eingefrorenen Artefaktlogik, denselben Faktorbedingungen und denselben sechs Seeds ausgewertet. Die Aufgabe bleibt die fachliche Analyseeinheit. Seeds sind wiederholte Ausführungen und keine zusätzlichen unabhängigen Kundenfälle.

Eignung und Evaluationsanpassung

Statt des ursprünglichen Gesamtscores verwendet die Arbeit zwei deterministische Offline-Endpunkte: *DB Match* prüft auf allen 80 Aufgaben den finalen Geschäftszustand. *Exact Mutation* prüft auf 72 write-positiven Aufgaben die exakte kanonische Schreibausführung. Tabelle 6 ordnet den Eigenschaften des Prüfstands Evaluationsnutzen und Aussagegrenzen zu (vgl. Yao et al. (2025), Abschn. 3–6; vgl. Barres et al. (2026), Abschn. 3–5).

Auswahlkriterium	Retail-Instanziierung	Nutzen für die Evaluation	Aussagegrenze
Dialogischer Prozess	Private Nutzerziele werden schrittweise durch einen LLM-basierten User-Simulator geäußert.	Prüft Interpretation, Revision und mehrstufige Fallbearbeitung.	Synthetischer Nutzer statt realer Kundschaft.
Transaktionale Wirkung	Typisierte Lese- und Schreibtools wirken auf einen veränderbaren Retail-Zustand.	Macht dauerhafte Zustandswirkung und Pre-Write-Gelegenheiten beobachtbar.	Vereinfachte Retail-Prozesse ohne vollständige Organisationsrealität.
Regelgebundene Autorität	Öffentliche Policy und Toolverträge begrenzen zulässige Aktionen.	Erlaubt die Prüfung von Informations- und Autoritätsgrenzen.	Kanonische Benchmarkregeln statt lokaler Unternehmenspolicy.
Kontrollierter Vergleich	Gleiche Aufgaben, Startzustände, Tools und Seeds liegen für C0 bis C3 vor.	Stützt gepaarte direkte und faktorielle Kontraste.	Interne Aussage für die fixierte Systemkonfiguration.
Ausführbare Referenz	Referenzlösung und finaler Datenbankzustand werden erst offline ausgewertet.	Ermöglicht deterministische Ergebnis- und Mutationsendpunkte ohne Laufzeitleckage.	Misst weder Gesprächsqualität noch Kundenzufriedenheit.

Tabelle 6. Passung des Retail-Prüfstands zur Forschungsfrage

Der Laufumfang verbindet einen gepaarten Vergleich identischer Aufgaben mit der Beobachtung stochastischer Laufvariation und vollständigen Ereignisspuren. Auf einen ersten Block aus drei gemeinsamen Seeds folgten deshalb drei weitere Seeds zur Belastbarkeits-

prüfung. Alle sechs Wiederholungen wurden unter identischen Aufgaben, Konfigurationen und Analysegrenzen ausgeführt und gleichrangig ausgewertet. Sie vervielfachen nicht die unabhängigen Aufgaben, sondern zeigen Größenordnung und Richtung der Variation zwischen Agentenausführungen.

Sechs Seeds sind damit der studienspezifische Wiederholungsumfang, weder eine allgemeine Mindestzahl noch eine Powergarantie. Der erforderliche Umfang hängt von Effektgröße, Laufvarianz und gewünschter Präzision ab (vgl. Colas; Sigaud; Oudeyer (2018), Abschn. 2 und 4–6). Berichtet werden daher Seedverteilungen, `pass6`, `pass@6` und Seed-Auslassungsanalysen statt eines ausgewählten Einzellaufs (vgl. Reimers; Gurevych (2017), S. 338–346; vgl. Agarwal et al. (2021), Abschn. 1–4). Die Wiederholungen reduzieren die Abhängigkeit von einem Lauf, ersetzen aber keine Replikation auf anderen Aufgaben und Modellen. Zur Einordnung verwenden τ -bench und τ^2 -bench mindestens drei beziehungsweise vier Durchläufe je Aufgabe. Wegen abweichender Designs begründet dieser Vergleich keine gleiche Präzision (vgl. Yao et al. (2025), Abschn. 5.1; vgl. Barres et al. (2026), Abschn. 4.1).

Modelle und technische Konstanten

Agent und Komponente A verwenden `openai/gpt-5.4-nano-2026-03-17`. Der User-Simulator verwendet `openai/gpt-4.1-2025-04-14`. Die Temperatur beträgt null und der Reasoning Effort des Agenten *medium*. Pro Episode gelten höchstens 120 Plattformschritte, fünf Laufzeitfehler und 900 Sekunden. Einzelne API-Anfragen besitzen ein Timeout von 120 Sekunden und bis zu zwei providerinterne Retries. Davon getrennt sind höchstens zwei technische Versuche einer Zelle. Verhaltensbedingte Wiederholungen sind nicht erlaubt.

Die Temperatur-null-Einstellung ist eine Entscheidung zugunsten interner Vergleichbarkeit. Eine niedrige Sampling-Temperatur konzentriert die Tokenauswahl stärker auf hochwahrscheinliche Fortsetzungen und reduziert damit zufallsgetriebene Ausgabevielfalt. Sie wird hier nicht als Entfernung einer gemessenen “Kreativität” interpretiert. Kreativität ist kein Konstrukt der Evaluation, und ein monotoner Zusammenhang zwischen Temperatur und Aufgabenleistung kann nicht vorausgesetzt werden (vgl. Renze (2024), S. 7347–7353). Die Ergebnisse gelten folglich für die fixierte Niedrigtemperaturkonfiguration. Eine Übertragung auf bewusst diversifizierende Produktionssettings mit höherer Temperatur bedürfte einer eigenen Untersuchung.

Temperatur null und ein fester Provider-Seed garantieren zugleich keine bitidentischen Antworten. Empirische Untersuchungen gehosteter LLMs beobachten auch unter nominell deterministischen Einstellungen abweichende Ausgaben und Leistungswerte (vgl. Atil et al. (2025), S. 135–142). Die verwendete API definiert den Seed als Initialwert für die Zufallssteuerung beim Sampling, nicht als Garantie identischer Antworten (vgl. OpenAI (2026), Parameter `seed`). Der jeweilige Wiederholungs-Seed wird an Agent, Modellaufrufe

von Komponente A und User-Simulator weitergegeben. Er kontrolliert damit die providerseitig adressierbare Stichprobenziehung, beseitigt aber nicht die verbleibende stochastische Inferenz- und Laufvariabilität.

Policy, Datensatz, Tools, Usermodell, Laufzeitparameter und Evaluator sind gepinnt. Die vier Bedingungen verwenden denselben Retail-Adapter und unterscheiden sich ausschließlich in den beiden Faktorflags.

Zwischen Task-Seed-Zellen besteht zudem Zustandsisolation. Für jede Zelle werden Retail-Umgebung und Ausgangsdatenbank, Agent, User-Simulator und Orchestrator neu konstruiert. Jeder Modellaufruf erhält nur die vollständige Nachrichtenhistorie der aktuellen Episode. Fortgesetzte Conversation-, Thread- oder Previous-Response-Referenzen werden nicht verwendet. Damit wird kein Gesprächszustand eines zuvor bearbeiteten Falls in den Modellkontext einer späteren Zelle übernommen. Diese Aussage betrifft den für die Inferenz bereitgestellten Kontext und behauptet nicht, dass ein API-Anbieter keinerlei technische Logs führt. Der ausführbare Isolations- und Request-Pfad ist in Listing 9 dokumentiert.

Fokale Nano-Evaluation

Die Nano-Evaluation bildet den fokalen Vergleich der Arbeit. Alle 1.920 Zellen wurden mit identischen Aufgaben, Seeds, User-Simulator, Policy, Tools, Faktorlogik, Budgets, Endpunkten und Offline-Evaluator ausgewertet. Berichtet werden bedingungsspezifische Endpunkte, direkte und faktorielle Kontraste, Seed-Stabilität, Saved-/Broken-Übergänge, Fehlerdiagnosen, Mechanismusgrößen und Aufwand. Daten werden weder mit anderen Modellkonfigurationen gepoolt noch über die beobachtete Modell-, Domänen- und Artefaktkonfiguration hinaus verallgemeinert.

Die Evaluation trennt Artefaktentwicklung und Bewertung auf Aufgabenebene. Die Artefaktlogik wurde auf Entwicklungsfällen stabilisiert. Die anschließende Nano-Evaluation verwendet Aufgaben, die nicht zur Anpassung dieser Logik herangezogen wurden. Nach Aufgaben geclusterte Intervalle quantifizieren Richtung und Präzision der beobachteten Effekte. Die vorab festgelegte Aufgabenpopulation verhindert eine fallweise Auswahl günstiger Aufgaben, ersetzt aber keine Zufallsstichprobe realer Unternehmensfälle.

Referenz- und Informationsgrenze

Trennung von Laufzeit und Referenzlösung bedeutet, dass keine Laufzeitbedingung auf Task-ID, User-Simulator-Instruktion, erwartete Aktionen, Evaluationskriterien, Gesamtscore, Offline-Codierschema oder erwarteten Datenbankzustand zugreifen kann. Diese Objekte werden erst nach Episodenende durch den Offline-Evaluator verwendet. Statische Importprüfungen und Laufzeit-Vertragstests sichern diese Trennung.

Informationsparität bedeutet, dass alle Bedingungen dieselbe öffentliche Policy, dieselben nativen Tools und dieselben grundsätzlich sichtbaren Ereignisse als zulässige Quellen besitzen. Die realisierten Informationsverläufe dürfen interventionsbedingt abweichen. Ein durch Komponente B ausgelöster Evidenzabruf ist daher eine messbare Interventionsfolge und kein kostenloser Informationsvorteil. Komponente A verwendet ausschließlich sichtbare Nutzerereignisse als Autorität für transaktionale Nutzerziele. Alle Bedingungen nutzen denselben nativen Agenten, dieselben Aufgaben und Seeds sowie dieselben Laufzeitgrenzen. Variiert wird ausschließlich die Aktivierung der Faktoren A und B. Zusätzliche Modellaufrufe, Informationsabrufe und Neuplanungen sind damit Folgen der Intervention. Sie werden nicht als kostenloser Vorteil behandelt, sondern gemeinsam mit Kosten und Laufzeit ausgewertet. Die zugehörige Fairness- und Informationsparitätsprüfung ist in Anhang A.5 zusammengefasst.

4.3 Experimentelles Design

Faktoren und Bedingungen

Das 2×2 -Design variiert zwei binäre Faktoren. Faktor A aktiviert Komponente A, die den nativen Agenten vor dessen Entscheidung durch quellengebundene Nutzerziele und selektive Unterstützungskarten unterstützt. Faktor B aktiviert Komponente B, die als LLM-freie Pre-Write-Kontrolle einen konkreten nativen Schreibkandidaten unmittelbar vor der Ausführung prüft.

Bedingung	A	B	Funktion
C0 Native	aus	aus	Unveränderter nativer Referenzpfad.
C1 Semantische Unterstützung	an	aus	Selektive, quellengebundene Unterstützungskarte; keine Schreibautorität.
C2 Pre-Write-Kontrolle	aus	an	Deterministische Machbarkeitsprüfung des exakten nativen Schreibkandidaten.
C3 Kombination	an	an	Beide Mechanismen bei unveränderter Autoritätstrennung.

Tabelle 7. Faktorielle Bedingungen der Retail-Evaluation

Komponente A darf keine Schreibaktion ausführen, freigeben, blockieren oder verändern. Komponente B erhält weder Nutzerziel-Einträge noch Unterstützungskarten von Komponente A, enthält kein LLM und darf einen gültigen Kandidaten nur unverändert freigeben. C3 führt daher keine dritte Autoritätslogik ein, sondern kombiniert zwei getrennte Interventionszeitpunkte.

Prüfannahmen und Operationalisierung

Fünf vor der Ergebnisanalyse festgelegte Prüfannahmen übersetzen Erfolg, Kontrolle und betriebliche Vorteilhaftigkeit in beobachtbare Endpunkte, Kontraste und Aufwandsgrößen (vgl. Venable; Pries-Heje; Baskerville (2016), S. 78–83; vgl. Wohlin et al. (2024), S. 73–83 und 115–184). Tabelle 8 verbindet die Einzelkomponenten, ihre Kombination sowie Nutzen und Robustheit mit der jeweiligen Messung und ökonomischen Lesart.

ID	Prüfannahme	Messung und ökonomische Lesart
H1	Die Aktivierung von Komponente A erhöht Erfolg und Stabilität gegenüber dem nativen Agenten.	C1–C0 bei DB Match, Exact Mutation, pass^k , $\text{pass}@k$ und Saved/Broken; zusätzlicher Nutzen wird im Verhältnis zu Modell- und Zeitaufwand gelesen.
H2	Die Aktivierung von Komponente B reduziert schreibbezogene Fehlerfolgen bei geringem Zusatzaufwand.	C2–C0 bei Exact Mutation, zusätzlichen und fehlenden Schreibaktionen, Entscheidungen von Komponente B, API-Kosten und Laufzeit; relevant ist die Pre-Write-Kontrolle vor dauerhaften Zustandsänderungen.
H3	Die gemeinsame Aktivierung beider Komponenten kann Wirkung, Fehlerprofil und Aufwand anders beeinflussen als jede Einzelkomponente.	C3–C0, C3–C1, C3–C2 und faktorielle Interaktion; geprüft wird kein Additivitätsversprechen, sondern der eigenständige Nutzen-Aufwands-Trade-off der Kombination.
H4	Der betriebliche Nutzen einer Variante hängt vom Verhältnis geretteter zu beschädigter Fälle ab.	Saved, Broken, Stable Pass und Stable Fail je Variante; entscheidend ist der Nettonutzen, nicht nur die Bruttozahl geretteter Fälle.
H5	Ein belastbarer Vorteil sollte über mehrere Seeds hinweg wiederholbar sein.	Seedweise Differenzen, pass^k , $\text{pass}@k$ und nach Aufgaben geclusterte Intervalle; relevant ist wiederholbare Zuverlässigkeit.

Tabelle 8. Prüfannahmen, Operationalisierung und ökonomische Lesart der Evaluation

Blockung, Randomisierung und Wiederholungen

Jede Aufgabe wird mit den sechs gemeinsamen Seeds 976301, 976302, 976303, 976304, 976305 und 976306 analysiert. Ein Seed ist eine festgelegte Wiederholungskennung für den simulierten Dialog- und Ausführungsverlauf einer Aufgabe. Er erzeugt keine neue Aufgabe, sondern eine wiederholte Ausführung derselben Aufgabe unter identischer Bedingungslogik. Aufgabe und Seed bilden den gepaarten Vergleichsschlüssel. Für jeden Task-Seed-Schlüssel liegen alle vier Bedingungen vor. Ihre Ausführungsreihenfolge wurde innerhalb des jeweiligen

Blocks randomisiert. Die Randomisierungseinheit ist damit der vollständige Vier-Bedingungs-Block aus Aufgabe und Seed.

Ein gemeinsamer Seed erzwingt keinen identischen Dialog. Eine Intervention kann Nutzerantworten und Folgeverlauf verändern. Deshalb ist die Task-Seed-Zelle die beobachtete Einheit. Unsicherheitsverfahren clustern zusätzlich auf Aufgabe. Die Paarung ist somit eine Blockungs- und Kontrollstrategie, keine Behauptung synchronisierter Zufallsziehungen über divergierende Trajektorien. Die 80 Aufgaben sind wiederholbare Aufgabenkonstrukte, keine Zufallsstichprobe realer Kundenkontakte. Die sechs Seeds erfassen Laufvariation, erhöhen aber nicht die Zahl unabhängiger fachlicher Aufgaben von 80 auf 480. Gepaarte Vergleiche halten Aufgabe und Seed konstant. Saved und Broken zeigen, ob eine Bedingung denselben Fall besser oder schlechter abschließt als C0. Die Komponentenprüfung ergänzt diese Ergebnisänderungen um den beobachtbaren Wirkpfad (vgl. Wohlin et al. (2024), S. 115–145 und 153–184). Die Paarung wird in Aggregation und Kontrastbildung erhalten, weil ein unabhängiger Gruppenvergleich die gemeinsame Aufgaben- und Seedstruktur verwerfen würde (vgl. Peyrard et al. (2021), S. 2301–2305).

Vergleichsgrößen

Direkte Zellkontraste vergleichen C1, C2 und C3 jeweils mit dem nativen Referenzpfad C0. Faktorielle Kontraste isolieren dagegen die mittleren Beiträge der Faktoren A und B sowie ihre Abweichung von Additivität. Tabelle 9 definiert die formale Notation.

Größe	Formale Definition	Interpretation
Indizes a, b	$a \in \{0, 1\}$ für Faktor A; $b \in \{0, 1\}$ für Faktor B	0 bezeichnet den deaktivierten, 1 den aktivierten Faktor.
Zellmittel $\bar{Y}_{ab}$	Mittlerer binärer Endpunkt in der durch a und b bestimmten Bedingung	Beobachtete Erfolgsquote der jeweiligen Faktorkombination.
Bedingungen C0–C3	C0: $\bar{Y}_{00}$; C1: $\bar{Y}_{10}$; C2: $\bar{Y}_{01}$; C3: $\bar{Y}_{11}$	Verknüpft die Konfigurationsnamen mit den vier Zellen des 2×2 -Designs.
Haupteffekt von Faktor A	$\frac{1}{2} [(\bar{Y}_{10} - \bar{Y}_{00}) + (\bar{Y}_{11} - \bar{Y}_{01})]$	Mittlerer Unterschied von Faktor A über beide Stufen von Faktor B.
Haupteffekt von Faktor B	$\frac{1}{2} [(\bar{Y}_{01} - \bar{Y}_{00}) + (\bar{Y}_{11} - \bar{Y}_{10})]$	Mittlerer Unterschied von Faktor B über beide Stufen von Faktor A.
Interaktion der Faktoren A $\times$ B	$\bar{Y}_{11} - \bar{Y}_{10} - \bar{Y}_{01} + \bar{Y}_{00}$	Abweichung der beobachteten Kombination von additiver Wirkung.

Tabelle 9. Notation, Definition und Interpretation der faktoriellen Estimanden

Effekt meint hier einen vorab definierten Kontrast im eingefrorenen, vollständig geblockten System. Die isolierte Faktormanipulation und randomisierte Ausführungsreihenfolge stützen

eine interne Interventionsinterpretation für die untersuchte Konfiguration. Sie begründen keinen populationsweiten Effekt. Aufgaben und Seeds sind keine Zufallsstichprobe realer Kundenfälle oder anderer Modelle. Maßgeblich bleiben deshalb Richtung, Größenordnung, Fallzahlen und Unsicherheit statt eines einzelnen Signifikanzurteils.

Hierarchie der empirischen Aussagen

Die Auswertung folgt einer festen Belegkette: Vollständigkeit der 1.920 Zellen, fachliche Endpunkte, gepaarte Saved-/Broken-Wechsel, Mechanismusergebnisse und schließlich Ressourcen- sowie Fehlerfolgen. Dadurch bleiben technische Korrektheit, Ergebniswirkung, Wirkpfad und betriebliche Abwägung unterscheidbar. Absolute Zähler, Nenner, Punktschätzer und Intervalle werden gemeinsam berichtet. So beantwortet dieselbe Kette getrennt, ob eine Konfiguration wirkt, wie beobachtete Wechsel zustande kommen und welchen Aufwand ein Betreiber abwägen müsste.

4.4 Mess- und Analysevertrag

Primäre Endpunkte

DB Match nimmt den Wert eins an, wenn der deterministische Environment-Evaluator den finalen Datenbankzustand als identisch mit dem erwarteten Zustand bewertet. Der Endpunkt wird auf allen 80 Evaluationsaufgaben berechnet. Aufgaben ohne erwartete Schreibaktion bleiben enthalten, weil eine unerwünschte Mutation ihren korrekten Endzustand beschädigen kann.

Exact Mutation nimmt den Wert eins an, wenn das kanonische Multiset aller erfolgreich emittierten Writes exakt dem zulässigen Goldmutationsmultiset entspricht. Write-Familie, Multiplizität und materiell relevante Argumente müssen übereinstimmen. Es darf weder eine Mutation fehlen noch eine zusätzliche ausgeführt worden sein. Aufgaben ohne gültiges write-positives Mutationsoracle bleiben Bestandteil von DB Match und der Fehleranalyse, werden aber nicht in Exact Mutation gezählt. Die eingefrorene Population dieses Endpunkts umfasst damit 72 schreibpositive Retail-Aufgaben.

Beide Endpunkte bleiben Bestandteil des vorab festgelegten Messvertrags. Für die Analyse der Seedvariation dient DB Match jedoch als darstellerischer Leitendpunkt. Er besitzt mit allen 80 Aufgaben die vollständigere Population und bildet den fachlich relevanten Geschäftsendzustand unmittelbar ab. Exact Mutation wird für diese Fragestellung als strengere Robustheitsprüfung im write-positiven Teilkollektiv berichtet. Diese Hierarchie betrifft die Darstellung, nicht die nachträgliche Auswahl eines günstigeren Ergebnisses. Tabelle 10 fasst Population und Berichtsumfang beider Endpunkte zusammen.

Endpunkt	Population	Bericht
DB Match	80 Aufgaben, 480 Zellen je Bedingung	absolute Erfolge, Nenner, Anteil und gepaarte Differenz
Exact Mutation	72 Aufgaben, 432 Zellen je Bedingung	absolute exakte Mutationen, Nenner, Anteil und gepaarte Differenz

Tabelle 10. Primäre deterministische Endpunkte der Retail-Evaluation

Ergebnisübergänge und Fehlerdiagnosen

Für jede Bedingungszone wird der gepaarte C0-Wert mit dem jeweiligen Behandlungswert verglichen. *Saved* bedeutet, dass eine Bedingung einen nativen Fehlschlag löst. *Broken* bedeutet, dass eine Bedingung einen nativen Erfolg verliert. *Stable Pass* und *Stable Fail* bezeichnen unveränderte positive beziehungsweise negative Verläufe. Berichtet werden absolute Zahlen, Nenner, betroffene Task-IDs und $\text{Net Saved} = \text{Saved} - \text{Broken}$.

Ergänzend berechnet der Offline-Evaluator Mutationspräzision und -recall sowie die Diagnosen *Missing Write*, *Extra Write*, *Wrong Target* und *Wrong Parameter*. Diese Diagnosen sind besonders für die Pre-Write-Kontrolle relevant, weil sie nicht nur den Endzustand, sondern die tatsächlich ausgeführten Mutationen betrachten. Ein *Extra Write* bezeichnet eine ausgeführte Schreibaktion ohne Entsprechung im erwarteten Mutationsmultiset. Ein *Missing Write* eine erwartete Schreibaktion, die nicht ausgeführt wurde. Natürlichsprachliche Assertions oder LLM-as-a-Judge ersetzen weder DB Match noch Exact Mutation.

Die Seed-Stabilität wird zunächst auf Bedingungszoneebene untersucht. Für C0 bis C3 werden sämtliche sechs Seedquoten, Mittelwert, Standardabweichung und Spannweite je Endpunkt berichtet. Linien zwischen den Bedingungen erhalten die Paarung desselben Seeds. Dadurch wird getrennt sichtbar, wie stark die vier Rohverteilungen überlappen und ob gepaarte Verschiebungen innerhalb desselben Seeds überwiegend dieselbe Richtung besitzen (vgl. Reimers; Gurevych (2017), S. 338–346; vgl. Peyrard et al. (2021), S. 2301–2305).

Auf Aufgabenebene wird Seed-Stabilität mit den komplementären Maßen pass^k und $\text{pass}@k$ beschrieben. Für T Aufgaben mit jeweils $n = 6$ Wiederholungen und c_i erfolgreichen Wiederholungen der Aufgabe i werden die unverzerrten Schätzer aus τ -bench verwendet:

$$\text{pass}^k = \frac{1}{T} \sum_{i=1}^T \frac{\binom{c_i}{k}}{\binom{n}{k}}, \quad \text{pass}@k = 1 - \frac{1}{T} \sum_{i=1}^T \frac{\binom{n-c_i}{k}}{\binom{n}{k}}.$$

pass^k schätzt die Wahrscheinlichkeit, dass alle k ausgewählten Wiederholungen erfolgreich sind. $\text{pass}@k$ schätzt die Wahrscheinlichkeit mindestens eines Erfolgs. Damit entspricht pass^1

der mittleren Einzellauf Erfolgsquote, während `pass6` bei sechs Seeds durchgängigen Erfolg verlangt. `pass@6` zeigt ergänzend, ob eine Aufgabe wenigstens einmal gelöst wurde (vgl. Yao et al. (2025), S. 4). Die Werte werden deskriptiv berichtet und nicht als zusätzliche unabhängige Hypothesentests interpretiert.

Um die Verteilung innerhalb der sechs Wiederholungen unmittelbar sichtbar zu machen, wird für jede Bedingung zusätzlich gezählt, wie viele Aufgaben exakt $r \in \{0, \dots, 6\}$ erfolgreiche Seeds erreichen. Mit c_i als Zahl der Erfolge von Aufgabe i werden zwei deskriptive Kennzahlen berechnet:

$$H_r = \sum_{i=1}^T \mathbf{1}\{c_i = r\}, \quad D = \frac{1}{T} \sum_{i=1}^T \frac{c_i(6 - c_i)}{\binom{6}{2}}.$$

H_0 , H_6 und $\sum_{r=1}^5 H_r$ bezeichnen stabile Fehl-, stabile Erfolgs- und seed-sensitive Aufgaben. Für Aufgabe i zählt $c_i(6 - c_i)$ die Seedpaare mit unterschiedlichem Ergebnis. D ist deren mittlerer Anteil. Die Kennzahl beschreibt die Laufvariabilität des Gesamtsystems, isoliert aber weder stochastische Modellvariation noch Beiträge von Agent, User-Simulator, Anbieterinfrastruktur und Dialogtrajektorie.

Mechanismusprüfung

Die Ergebnisanalyse wird durch Komponentenpfade ergänzt. Für Komponente A wird der Pfad von einer geeigneten Nutzeräußerung über Extraktion, akzeptierte und verworfene Einträge, Nutzerzielprotokoll und materialisierte Unterstützungskarte bis zu einem späteren Schreibkandidaten rekonstruiert. Für Komponente B wird jeder native Schreibkandidat über deterministische Disposition, gegebenenfalls exakten Evidenzabruf oder einmalige Neuplanung bis zur finalen Ausführung, Blockierung oder Übergabe verfolgt.

Eine Aktivierung gilt nicht automatisch als Erfolg. Komponentenereignisse werden deshalb mit Saved-/Broken-Übergängen, Mutationsdiagnosen und finalem DB Match verbunden. Für Komponente A unterscheidet die Auswertung Zellen mit und ohne sichtbar materialisierte Unterstützungskarte. Für Komponente B unterscheidet sie unveränderte Freigaben, Evidenzanforderungen, ungültige Kandidaten und fehlende Schreibgelegenheiten. Die Analyse prüft, ob die Komponente tatsächlich im Handlungspfad lag und ob der anschließende Fall besser, schlechter oder unverändert endete.

Die Verknüpfung erfolgt über technische Ereignis- und Kandidaten-IDs, nicht über eine nachträgliche Textdeutung. Dadurch lässt sich prüfen, ob ein Hinweis tatsächlich in den Agentenkontext gelangte oder ob eine Entscheidung von Komponente B genau den Kandidaten betraf, der anschließend freigegeben, neu geplant oder blockiert wurde. Gezielt ausgewählte Ereignisspurvignetten illustrieren je einen Korrektur- und Grenzfall. Sie ersetzen nicht

die vollständige Matrix, machen aber sichtbar, wie die Komponenten im Einzelfall im Handlungspfad lagen und welches Ergebnis darauf folgte.

Die Vignetten folgen einer gezielten Auswahl kontrastierender, informationsreicher Fälle. Für Komponente A, Komponente B und deren Kombination C3 wird jeweils eine Saved- und eine Broken-Zelle ausgewählt. Ein Fall zu Komponente A muss eine technisch mit einem späteren Schreibkandidaten verknüpfte Unterstützungskarte enthalten. Ein Fall zu Komponente B mindestens eine materielle Kontrollentscheidung. Ein C3-Fall beide Merkmale. Diese gezielte Auswahl dient der Erklärung positiver und negativer Pfade, nicht der Häufigkeitsschätzung (vgl. Palinkas et al. (2015), S. 533–537).

Alle sechs Ereignisspuren werden mit demselben Raster gelesen: sichtbares Nutzerziel, Revision oder Unsicherheit, materialisierter A-Inhalt, gebundener Schreibkandidat, Entscheidung von Komponente B und deren sichtbare Evidenz, ausgeführte Mutation sowie offline bestimmter Ergebnisübergang. Die sequenzielle Rekonstruktion prüft, ob die für den postulierten Mechanismus erforderliche Evidenzkette im Fall tatsächlich beobachtbar ist (vgl. Collier (2011), S. 823–828; vgl. Runeson; Höst (2009), S. 145–153). Dabei werden mechanismuskonsistente Deutung und alternative Erklärung ausdrücklich getrennt. Die Vignetten belegen daher weder einen isolierten Mediatoreffekt noch repräsentative Teilgruppenanteile. Sie erklären, wie ein bereits quantitativ gezählter Pfad konkret zustande kam oder an welcher Grenze er scheiterte.

Aufwand und betriebliche Abwägung

Pro Episode werden Gesamtkosten, Gesamtlaufzeit und Tool-Aufrufe erfasst. Die Ereignisspuren ergänzen den Aufwand von Komponente A sowie Evidenzabrufe und Neuplanungen von Komponente B. Unterschiede werden absolut und task-seed-gepaart berichtet. Agentenbenchmarks sollten neben Erfolgsmaßen auch Kosten, Sicherheit und Fehlerursachen sichtbar machen (vgl. Yehudai et al. (2025), S. 6–9; vgl. Lu; Li; Huo (2025), S. 1–3).

Agenten-API-Kosten bilden den tatsächlich vom Agentensystem verursachten variablen Modellaufwand ab. Die API-Kosten des User-Simulators gehören zur experimentellen Datenerhebung und werden nicht als betriebliche Agentenkosten interpretiert. Die technische Gesamtlaufzeit ist ebenfalls nur eine Näherungsgröße für den Ressourcenaufwand. Sie entspricht ohne externe Bewertung weder Personalarbeitszeit noch Kundenwarte- oder Kapazitätskosten.

Eine nachgelagerte betriebliche Leistungsperspektive betrachtet Effektivität, Zuverlässigkeit, Regressionen und Ressourcen als getrennte Konfigurationsdimensionen (vgl. Van Looy; Shafagatova (2016), Abschn. 3–4; vgl. Grönroos; Ojasalo (2004), S. 414–416). Da der Benchmark keine realen Prozess- oder Fehlerfolgekosten enthält, werden ausschließlich

gemessene Größen verglichen. Externe Bewertungen bleiben als Voraussetzung einer späteren Unternehmensentscheidung ausgewiesen.

Sie nutzt alle 480 Task-Seed-Zellen je Konfiguration und berichtet fachliche Endpunkte, Saved-/Broken-Wechsel, Agenten-API-Kosten, Laufzeit, Nachrichten und Tool-Aufrufe. Die deterministische Offline-Auswertung kann Episoden und Endpunkte nicht verändern. Sie prüft zuvor Matrixvollständigkeit, Kostenabgrenzung und eindeutige Zellschlüssel. Unternehmensentscheidungen benötigen zusätzlich Daten etwa zu Nachbearbeitung, Rückabwicklung und Kundenwartezeit.

Schätzung und Unsicherheit

Für jede Bedingung werden absolute Erfolge und Nenner, Prozentwerte sowie gepaarte Differenzen in Prozentpunkten berichtet. Die Robustheitsanalyse verwendet seedweise t-Intervalle, Auswertungen mit Clustering nach Aufgaben und einen konservativen nichtparametrischen Zweiweg-Bootstrap über Aufgaben und Seeds. Bootstrapverfahren approximieren die Stichprobenverteilung eines Schätzers durch wiederholtes Resampling der beobachteten Daten (vgl. Efron (1979), S. 1–4). Das hier verwendete unabhängige Resampling der gekreuzten Aufgaben- und Seeddimension folgt dem Grundprinzip des Pigeonhole-Bootstraps für zweidimensional strukturierte Beobachtungen (vgl. Owen (2007), S. 386–392). Die Verwendung mehrerer komplementärer Unsicherheitsansichten folgt der Forderung, statistische Verfahren an Metrik und Abhängigkeitsstruktur des Versuchs anzupassen (vgl. Dror et al. (2018), S. 1383–1387). Resampling über Aufgaben und Läufe greift zudem die Empfehlung auf, bei rechenintensiven Few-Run-Evaluationen neben Punktschätzern die endliche Run-Unsicherheit explizit auszuweisen (vgl. Agarwal et al. (2021), Abschn. 1–4).

Die drei Verfahren beantworten komplementäre Fragen. Clustering nach Aufgaben prüft die Abhängigkeit von der Aufgabenpopulation, seedweise Kontraste die Streuung über Wiederholungen und der Zweiweg-Bootstrap beide Dimensionen gemeinsam.

Auch diese Verfahren beseitigen die Few-Run-Grenze nicht. Ein seedweises t-Intervall beruht auf nur sechs realisierten Kontrasten. Der Zweiweg-Bootstrap resampelt ebenfalls lediglich sechs Seedcluster. Beide Ansichten sind deshalb Sensitivitäts- und Unsicherheitsdiagnosen, keine Garantie nominaler Präzision für eine unbeobachtete Run-Population. Stimmen Punktschätzer, Richtung der Seed-Auslassungsanalyse und Intervalle nicht vollständig überein, erhält die konservativere Aussage Vorrang.

Eine Seed-Auslassungsanalyse bestimmt Haupteffekte zudem sechsmal ohne jeweils einen Seed. Gleichbleibende Richtung schließt die Abhängigkeit von genau einer Wiederholung aus, erhöht aber nicht die Zahl unabhängiger Beobachtungen. Punktschätzer beschreiben daher die

Matrix, Robustheitsverfahren prüfen Teilperspektiven und der Zweiweg-Bootstrap begrenzt weiterreichende Aussagen.

Die Studie berichtet beobachtete Effekte, Unsicherheit und Komponentenmechanismen transparent. Etwaige p-Werte werden höchstens ergänzend ausgewiesen. Ein breites Intervall ist begrenzte Präzision und weder Evidenz für Äquivalenz noch Anlass für eine ergebnisabhängige Wiederholung.

Technische Fehler und fehlende Werte

Eine fachlich abgeschlossene Episode wird unabhängig von ihrem Ergebnis als Beobachtung übernommen. Technische Wiederholungsversuche sind ausschließlich auf Anfrageebene bei vorab definierten Provider-, Netzwerk- oder Infrastrukturfehlern zulässig und durch ein festes Cap begrenzt. Nach dessen Ausschöpfung bleibt die Zelle fehlend. Ursache und Versuchszahl werden berichtet.

Die Auswertung folgt einer unveränderlichen Verarbeitungskette. Zunächst werden nur technisch abgeschlossene Episodendatensätze in ein Offline-Format überführt. Danach prüft der Abgleich für jede erwartete Kombination aus Bedingung, Aufgabe und Seed Eindeutigkeit, Vollständigkeit und die Verknüpfung mit Simulation und Mechanismusspur. Erst nach Bestehen dieser Prüfung werden DB Match, Mutationsmultiset, Fehlerdiagnosen, Aufwand und Saved-/Broken-Übergänge berechnet. Die primären Endpunkte stammen dabei nicht aus den Online-Logs des Artefakts, sondern werden durch den getrennten Evaluator aus finalem Zustand, ausgeführten Aktionen und Referenzlösung neu bestimmt. Damit kann eine Komponente weder ihr eigenes Ergebnislabel schreiben noch eine fehlende Telemetrie als Erfolg maskieren.

Der Retrystatus bleibt als Herkunftsvariable erhalten. Anzahl und Ursache technischer Wiederholungsversuche werden bereits in der Vollständigkeitstabelle berichtet. Ein vorzeitiger Abbruch der Datenerhebung aufgrund günstiger oder ungünstiger Zwischenwerte war nicht vorgesehen. Die Matrix gilt erst als vollständig, wenn jede erwartete Zelle einen auswertbaren Episodendatensatz oder einen transparent ausgewiesenen technischen Fehlstatus besitzt. Diese Regel schützt vor optionalem Stoppen und selektiver Verfügbarkeit einzelner Bedingungen.

4.5 Güte, Reproduzierbarkeit und Aussagegrenzen

Validität

Die Konstruktvalidität stützen getrennte Maße für Geschäftsendzustand, Schreibaktionen, gepaarte Ergebniswechsel, Komponentenaktivität und Aufwand. Keine Metrik wird als vollständiges Maß realer Servicequalität ausgegeben.

Für die interne Validität sprechen die gepaarten Task-Seed-Schlüssel, die blockrandomisierte Reihenfolge, gepinnte Modelle und Laufzeitparameter, Trennung von Laufzeit und Referenzlösung sowie vollständige Telemetrie. C0 bis C3 liegen je Task-Seed-Schlüssel als vollständige Blöcke vor. Die Artefaktlogik blieb während der finalen Sechs-Seed-Evaluation unverändert. Die Evaluation bleibt auf diese konkrete Benchmarkdomäne, Modellkonfiguration und Artefaktversion bezogen.

Die externe Validität bleibt auf Benchmarkdomäne, Modellkonfiguration, Artefaktinstanziierung und simulierte Nutzer begrenzt. Eine unmittelbare Generalisierung auf Unternehmen, andere Policies oder Modelle ist nicht begründet.

Die Schlussfolgerungsalidität wird durch absolute Zähler und Nenner, gepaarte Kontraste sowie nach Aufgaben geclusterte Unsicherheitsintervalle gestützt. Die vier Kontraste wurden als zusammengehende deskriptive Estimandenfamilie festgelegt. Einzelne Grenzwerte werden nicht nachträglich als alleinige Entscheidungsregel ausgewählt. Breite Intervalle begrenzen die Präzision. In dieser Arbeit werden sie weder als Gleichheitsnachweis noch als Nachweis eines fehlenden Effekts gelesen. Umgekehrt wird ein Punktwert nicht allein wegen seiner Richtung als belastbarer Nutzen interpretiert. Für die betriebliche Entscheidung zählen zusätzlich die absolute Zahl beschädigter Fälle, die Fehlerart und der dafür erforderliche Aufwand.

Die mehrturnige Bewertung natürlich entstehender Reads, Antworten und Write-Kandidaten stützt die ökologische Validität innerhalb des Prüfstands. Sie ersetzt keine Produktivstudie mit realen Nutzer- und Organisationsfolgen.

Reproduzierbarkeit und Änderungskontrolle

Datensatz, Split, Policy, Tools, Laufzeit, finaler Artefaktcode, Tests, Offline-Evaluator und Ausführungsvertrag sind kryptografisch gebunden. Jede Zelle besitzt eine eindeutige ID aus Bedingung, Task und Seed. Simulation, Mechanismusspur, Kosten, Laufzeit, technischer Status und Versuchszahl werden getrennt persistiert. Der Runner ist resumierbar und überspringt vollständige Zellen.

Der kanonische Analysevertrag definiert 1.920 auswertbare Zellen mit genau einem Episodendatensatz für jede Kombination aus Bedingung, Aufgabe und Seed. Artefaktlogik, Modelle, Endpunkte und Budgets sind über diese Matrix einheitlich. Fachlich abgeschlossene Ergebnisse werden nicht verhaltensabhängig wiederholt. Die Auswertung bezieht sich auf diese vollständige Nano-Retail-Matrix.

5 Anforderungsanalyse

Dieses Kapitel leitet aus der Forschungslücke konkrete Anforderungen an das Artefakt ab. Die Anforderungen sind kein vollständiges industrielles Lastenheft. Sie bestimmen, was die entwickelte Laufzeitschicht leisten muss, damit sie die in Kapitel 3 beschriebene Lücke fair prüfbar macht. Damit verbindet das Kapitel Problem, Literaturbefund, Gestaltung und spätere Evaluation. Es folgt dem DSR-Grundsatz, ein Artefakt aus einem relevanten Problem abzuleiten und seinen Nutzen anschließend mit geeigneten Evaluationsmethoden nachzuweisen (vgl. Hevner et al. (2004), S. 82–83; vgl. Peffers et al. (2007), S. 55–56).

5.1 Ableitungslogik und Gestaltungsziel

Anwendungskontext und Anforderungsquellen

Die Anforderungsableitung beginnt nicht bei den später implementierten Komponenten, sondern bei vier Merkmalen des untersuchten Serviceprozesses. Erstens ist der Prozess *transaktional*. Werkzeugaufrufe können einen Geschäftszustand dauerhaft verändern. Ergebnisqualität umfasst daher nicht nur eine plausible Antwort, sondern auch die Vollständigkeit und Zulässigkeit der ausgeführten Zustandsänderungen. Zweitens ist der Prozess *dialogisch*. Nutzerziele können über mehrere Äußerungen ergänzt, korrigiert oder widerrufen werden. Die Laufzeit muss deshalb den jeweils sichtbaren Informationsstand erhalten, ohne spätere Referenzinformation vorwegzunehmen.

Drittens wird der Prozess im untersuchten Kundenservice als turnbasierte Chatinteraktion ausgeführt. Antwortzeit ist in Mensch–Chatbot-Interaktionen ein wahrnehmbares Gestaltungsmerkmal. Ihre Wirkung ist allerdings kontextabhängig und nicht monoton, sodass weder eine künstliche Verzögerung noch minimale Antwortzeit pauschal als optimal gelten kann (vgl. Gnewuch et al. (2022), S. 773–776 und 785–787). Daraus folgt für diese Arbeit kein vorgegebenes Service Level Agreement (SLA) und auch kein vorweggenommenes Urteil gegen umfangreichere Agentenarchitekturen. Wohl aber muss jede zusätzliche Laufzeitintervention ihren inkrementellen Zeit-, Modell- und Interaktionsaufwand offenlegen. Der Benchmark bildet damit einen technischen Aufwand unter interaktiver, turnbasierter Dialogausführung ab, nicht die reale Wartezeittoleranz von Kunden. Für asynchrone Servicekanäle wird diese Anforderung in der vorliegenden Arbeit nicht untersucht.

Viertens ist die Ausführung *stochastisch und experimentell*. Agent, Komponente A und User-Simulator können trotz fixierter Konfiguration unterschiedliche Trajektorien erzeugen (vgl. Atıl et al. (2025), S. 135–142; vgl. Renze (2024), S. 7347–7353). Anforderungen an Informationsparität, Faktortrennung, Zustandsisolation und wiederholte Messung sind deshalb keine technischen Nebenbedingungen, sondern Voraussetzungen dafür, beobachtete Unterschiede den Interventionen zurechnen zu können. Aus den vier Merkmalen folgen drei

Anforderungsklassen: funktionale und integritätsbezogene Grenzen für die Komponenten A und B, Identifikationsanforderungen für die faktorielle Evaluation sowie betriebliche Anforderungen an Ergebnis, Stabilität und Ressourcenaufwand.

Gestaltungsziel

Das übergeordnete Gestaltungsziel lautet:

Für transaktionale Serviceprozesse soll eine schaltbare Laufzeitschicht gestaltet werden. Sie unterstützt den Agenten vor seiner Entscheidung mit sichtbaren, belegten Nutzerzielen und kontrolliert geplante Schreibaktionen unmittelbar vor der Ausführung. Der Agent bleibt dabei für Dialog und Planung verantwortlich. Die Laufzeitschicht darf keine versteckte Benchmarkinformation nutzen. Bewertet werden anschließend Erfolg, Fehlerfolgen, Stabilität, Laufzeit und Kosten.

Die Anforderungen folgen aus vier Befundgruppen. Erstens können agentische Serviceprozesse trotz plausibler Kommunikation am finalen Geschäftszustand oder an der vollständigen Transaktionsausführung scheitern (vgl. Yao et al. (2025), S. 1–6; vgl. Li et al. (2025), Abschn. 3–5). Zweitens zeigen semantische Unterstützungsansätze, dass Ziel- und Evidenzbereitstellung den Verlauf verbessern kann, ihre Wirkung jedoch von Informationszugang und Operationalisierung abhängt (vgl. Mishra et al. (2025), Abschn. 3–5; vgl. Saeidi et al. (2026), Abschn. 3–5). Drittens motivieren Tool- und Policy-Kontrollen eine Prüfung unmittelbar vor folgenreichen Aktionen, machen breite Workflowsteuerung oder Fehlblockierung aber nicht automatisch unproblematisch (vgl. Cuadron et al. (2025), Abschn. 4–6; vgl. Zwerdling et al. (2025), Abschn. 3–5; vgl. Wang; Poskitt; Sun (2026), Abschn. 4–7). Viertens zeigen Evaluationsarbeiten, dass Erfolgsraten allein weder Regressionen korrekter Fälle noch Mechanismus, Sicherheit und Aufwand hinreichend erklären (vgl. Yehudai et al. (2025), S. 6–9; vgl. Lu; Li; Huo (2025), S. 1–3).

Die Zweiteilung in Komponente A und Komponente B folgt damit nicht aus einer beliebigen Komponentenliste, sondern aus dem Ablauf toolnutzender Agenten. Komponente A setzt im Interpretations- und Planungsteil an. Der Agent soll sichtbare Nutzerziele und Belege vor seiner nächsten Entscheidung besser nutzen. Komponente B setzt an der Ausführungsgrenze an. Ein bereits geplanter Schreibaufwurf soll vor der Zustandsänderung gegen beobachtbare Regeln geprüft werden. Diese Trennung entspricht der in der Agenten- und Tool-Use-Literatur üblichen Unterscheidung zwischen Entscheidung beziehungsweise Planung und externer Aktion (vgl. Wooldridge; Jennings (1995), S. 118–119; vgl. Yao et al. (2023), S. 1–4; vgl. Qu et al. (2025), S. 5–11).

Damit wird keine erschöpfende binäre Fehlertaxonomie behauptet. Planung, Toolauswahl, Kommunikation, Wiederherstellung und Abschluss können ebenfalls scheitern. Das Artefakt

fokussiert vielmehr zwei für transaktionale Zustandsänderungen zentrale Interventionsorte, die sich systematisch unterscheiden. Vor der Entscheidung ist semantisch mehrdeutige Information zu verarbeiten. A arbeitet deshalb probabilistisch, quellengebunden und rein beratend. Vor der Ausführung liegt dagegen eine konkrete Kandidatenaktion vor. Komponente B prüft nur explizierbare Zustands- und Policybedingungen deterministisch und mit eng begrenzter Zulassungsautorität. Das faktorielle Design variiert somit zwei unterschiedliche Eingriffsprinzipien entlang derselben Handlungskette und nicht zwei beliebig ausgewählte Softwarekomponenten.

Aus diesen Befunden folgt weder die Notwendigkeit noch die Überlegenheit einer umfangreichen Multi-Agenten- oder Hybridarchitektur. Der Umfang wird deshalb nicht über eine vorab bevorzugte Architekturklasse, sondern über die beiden offenen Betreiberentscheidungen festgelegt: Soll der Agent vor seiner Entscheidung zusätzliche semantische Unterstützung erhalten? Und sollen geplante Schreibaktionen vor der Ausführung kontrolliert werden? Das Artefakt realisiert dafür die kleinste in dieser Arbeit vergleichbar prüfbare Interventionsschicht. Ob der begrenzte Eingriff einen günstigeren Nutzen-Aufwands-Trade-off erzeugt, bleibt Gegenstand der Evaluation. Beide Eingriffe müssen dafür getrennt einschaltbar und unter denselben Bewertungsbedingungen prüfbar sein.

Damit ergeben sich vier Gestaltungsaufgaben. Erstens muss Komponente A sichtbare Nutz-erziele und Belege so strukturieren, dass der Agent sie leichter nutzen kann, ohne dadurch neue Fachinformation oder Ausführungsrechte zu erhalten. Zweitens muss Komponente B geplante Schreibaktionen unmittelbar vor der Ausführung gegen beobachtbare Zustands- und Policybedingungen prüfen. Drittens muss die Grenze zwischen Komponente A und Komponente B hart bleiben. Semantische Hinweise dürfen nicht zu automatischer Blockierung werden. Viertens müssen beide Faktoren getrennt einschaltbar und gemeinsam beobachtbar sein, damit Nutzen, Schaden und Aufwand später zurechenbar bleiben.

Die neun Anforderungen bündeln diese Aufgaben in vier Leitfragen: Nutzen Agent und Auswertung nur zulässige Information? Wird eine problematische Schreibaktion kontrolliert, ohne den Fall selbst zu übernehmen? Lassen sich die Faktoren A und B getrennt einschalten, messen und erklären? Und wird sichtbar, welche Fälle besser, schlechter oder teurer werden?

5.2 Gestaltungsanforderungen

Formulierungs- und Akzeptanzregel

Jede Anforderung muss auf einen dokumentierten Problem- oder Literaturbefund zurückgehen, eine prüfbare Gestaltungsgrenze setzen und technisch wie empirisch bewertet werden können. Dabei sind zwei Nachweise zu unterscheiden. Die *Designverifikation* zeigt durch Tests und Audits, ob eine Komponente wie spezifiziert arbeitet. Die *Evaluation* zeigt, welche Folgen

diese Funktion im vollständigen Agentenprozess erzeugt. Ein bestandener Komponententest ist daher noch kein Nutznachweis. Ein höherer Benchmarkscore belegt umgekehrt nicht automatisch eine korrekte Implementierung. Erst beide Ebenen gemeinsam schließen die Kette von Anforderung, Gestaltung und Evaluation.

ID	Gestaltungsdimension	Prüfbare Anforderung
DR1	Ergebnis und Prozess	Endzustand, Schreibkandidat, Kontrollentscheid und ausgeführte Aktion müssen getrennt beobachtbar und stabil zuordenbar sein.
DR2	Gleicher Informationszugang A	Komponente A darf nur sichtbare Ereignisse und reguläre Lese-Werkzeuge verwenden; jeder übernommene Hinweis benötigt eine sichtbare Herkunft.
DR3	Begrenzte Eingriffsrechte B	Komponente B prüft Schreibaktionen LLM-frei, plant oder repariert nicht und gibt bei Freigabe ausschließlich den unveränderten nativen Kandidaten weiter.
DR4	Aussagegrenze	Das Artefakt wird als Laufzeitinterventionsschicht beschrieben; formative Entwicklungsevidenz begründet keinen abschließenden Wirksamkeitsnachweis.
DR5	Trennbare Komponenten	Die Komponenten A und B müssen in einer gemeinsamen Laufzeit getrennt schaltbar sein; C1 bis C3 unterscheiden sich nur in den Faktorflags.
DR6	Korrektur und Abschluss	Eine verhinderte Aktion gilt nicht als Falllösung; Evidenz- und Neuplanungspfade bleiben begrenzt und legen ihren Abschlussstatus offen.
DR7	Betriebliche Effizienz und Nettotonnen	Ergebniswechsel, API-Kosten, technische Laufzeit, Nachrichten, Tool-Aufrufe und Transfers müssen gemeinsam messbar sein; gemessene und extern gesetzte Kosten bleiben getrennt.
DR8	Deterministische Referenzlösung	Die primäre Ergebnisreferenz liegt deterministisch und offline außerhalb der Laufzeit; LLM-gestützte Bewertungen bleiben sekundär.
DR9	Gleiche Eingriffsgelegenheiten	Alle Bedingungen verwenden dieselbe Definition von Eingriffsgelegenheiten und stabile IDs; Aktivierungen werden gegen diese Gelegenheiten normalisiert.

Tabelle 11. Prüfbare Gestaltungsanforderungen der Laufzeitinterventionsschicht

Aus dieser Ordnung folgen drei bewusste Zielkonflikte. Erstens erhöht mehr semantischer Kontext nicht monoton die Entscheidungsqualität. Eine Unterstützungskarte kann ein zuvor vergessenes Teilziel sichtbar machen, zugleich aber die Aufmerksamkeit des Agenten von einem bereits korrekten nativen Pfad verschieben (vgl. Shi et al. (2026), Abschn. 4–7; vgl. Ding; De la Cruz Weinstein (2026), Abschn. 2–4). DR2 verlangt deshalb nicht möglichst viele Informationen, sondern nachweisbar sichtbare, selektiv bereitgestellte Informationen. Zweitens erhöht eine strengere Kontrolle nicht automatisch den sicheren Fallabschluss. Komponente B kann einen unzulässigen Kandidaten erkennen, besitzt aber bewusst weder die Nutzerintention noch die Planungsautorität, um den ganzen Fall selbst zu lösen (vgl. Sah et al. (2026), Abschn. 3–5; vgl. Wang et al. (2025), Abschn. 3–5). DR3 und DR6 verbinden die enge Kontrolle deshalb mit begrenztem Korrekturpfad und einem separaten Ergebnis. Drittens darf umfangreiche Beobachtbarkeit nicht selbst zu einer verdeckten Bedingung werden. DR1 und DR9 verlangen ein passives, bedingungsübergreifend einheitliches Ereignisprotokoll, während nur die Komponenten A und B Verhalten verändern dürfen.

Die Akzeptanznachweise folgen vier aufeinander aufbauenden Ebenen: Schemas und Komponententests prüfen lokale Eigenschaften, Integrationstests deren Erhalt im gemeinsamen Laufzeitpfad, Ereignisspuren die Erreichbarkeit spezifizierter Pfade und das faktorielle Experiment deren Ergebnis-, Fehler- und Aufwandsfolgen. Keine Ebene ersetzt die folgende. Dadurch bleibt unterscheidbar, ob eine Komponente spezifikationsgemäß arbeitet und ob diese Arbeitsweise im vollständigen Serviceprozess nützt.

Priorisierung und Entscheidungsregel

DR2, DR3 und DR8 sind Ausschlusskriterien, weil ihre Verletzung das Konstrukt oder seine Bewertung verändert. DR1, DR5 und DR9 sichern die empirische Identifizierbarkeit. DR4 und DR6 begrenzen die Aussage, DR7 erweitert die Abnahme um betriebliche Folgen. Entsprechend müssen zuerst Integrität und Faktortrennung technisch bestehen und die spezifizierten Pfade in der Ereignisspur nachweisbar sein. Erst danach wird der Nettonutzen aus primärem Ergebnis, geretteten und beschädigten Ergebniswechseln sowie Aufwand bewertet. Technische Konformität und Einsatzentscheidung bleiben damit getrennt. Eine korrekte Implementierung garantiert keinen Scoregewinn, und eine günstige Punktschätzung kompensiert keine verletzte Autoritäts- oder Referenzgrenze.

Autoritäts- und Scopegrenzen

Die Anforderungen begrenzen die Eingriffsintensität bewusst. Komponente A darf keinen mutierenden Werkzeugaufruf ausführen, zulassen, blockieren, umschreiben oder eine Weitergabe erzwingen. Seine semantischen Nutzerziel-Datensätze und Unterstützungskarten sind beratend. Komponente B darf keine fehlende Nutzerintention erfinden, keine mehrdeutige

Entität erraten und keine Kandidatenargumente reparieren. Seine harte Autorität beginnt und endet unmittelbar vor einer Schreibaktion.

Die gemeinsame sichtbare Zustandsprojektion und das Audit-Protokoll sind in allen Bedingungen vorhanden, bleiben bei deaktivierten Faktoren aber verhaltenspassiv. Die technische C0-Paritätsprüfung muss zeigen, dass deaktivierte Faktorports Prompt, Werkzeuge und Kandidatenidentität des nativen Agenten nicht verändern. Für die empirische Retail-Basis wird davon getrennt der in Kapitel 4 definierte gepinnte native Referenzlauf verwendet. Damit wird Beobachtbarkeit nicht zu einer verdeckten weiteren Bedingung.

5.3 Retail-Instanziierung und Nachvollziehbarkeit

Bindung an den Untersuchungskontext

Die Gestaltungsanforderungen werden im Retail-Kontext von τ^2 -bench instanziiert. Der gemeinsame Kern besteht aus Adapter, zwei Faktorflags, getrennten Eingriffsrechten, einem Modell der Eingriffsgelegenheiten und einem einheitlichen Ereignisprotokoll. Der sichtbare Zustand umfasst Nutzer, Bestellung, Produkt, Variante, Zahlung und sichtbare Werkzeugergebnisse. Faktor A strukturiert daraus Hinweise zu Stornierung, Adress-, Artikel- und Zahlungsänderung, Retoure und Umtausch. Komponente B prüft vor Schreibaktionen unter anderem Authentifizierung, Eigentum, Prozessstatus, Artikel- und Produktbezug sowie Zahlungs- und Refundbedingungen.

Der konzeptionelle Zuschnitt der beiden Faktoren ist grundsätzlich auf andere transaktionale Servicedomänen übertragbar. Die vorliegende Arbeit prüft diese Übertragbarkeit jedoch nicht empirisch und erhebt deshalb keinen domänenübergreifenden Wirksamkeitsanspruch. Die Retail-Population ist die einzige summative Analysedomäne.

Nicht natürlich exponierte Regelpfade werden technisch durch deterministische Fixtures geprüft. Solche Tests belegen die Implementierung, tragen aber keine eigenständige empirische Wirkungsbehauptung.

Nachweiskette von Problem, Anforderung, Gestaltung und Prüfung

Die Anforderungen schließen unmittelbar an Kapitel 3 und 4 an. Kapitel 6 beschreibt, wie die Designantworten in Komponenten, Ports, Zustände, Prädikate und Konfiguration übersetzt wurden. Die faktorielle Studie prüft anschließend nicht, ob das Artefakt definitionsgemäß existiert, sondern ob und unter welchen Bedingungen seine beiden Mechanismen einen empirisch vertretbaren Nettonutzen erzeugen. Die technischen Akzeptanznachweise gelten als bestanden. Kapitel 8 stellt sie den tatsächlich beobachteten Wirkungen und Nebenwirkungen gegenüber.

6 Design und Entwicklung

Dieses Kapitel gestaltet die Laufzeitschicht um den weiterhin handelnden LLM-Agenten. Komponente A unterstützt ihn vor der Entscheidung mit belegten Nutzerzielen, Komponente B prüft geplante Schreibaktionen vor der Ausführung. Protokollierte Eingriffe machen Folgeaktion, Ergebnis und Aufwand nachvollziehbar. Nach Rollenabgrenzung und technischer Umsetzung folgen Funktionsprüfung, Demonstration in Kapitel 7 und empirische Bewertung in Kapitel 8.

6.1 Artefaktgrenze und Entwurfslogik

Zwei getrennte Interventionsentscheidungen

Ausgangspunkt ist ein nativer Agent, der Nutzeranfragen interpretiert, den Dialog führt und auf Grundlage einer öffentlichen Policy und verfügbarer Tools eigenständig handelt. Die Literatur zeigt sowohl Potenzial semantischer Reflexions- und Unterstützungsverfahren als auch enger formaler Kontrollen vor Toolaktionen. Zugleich bleiben die jeweiligen Beiträge in gekoppelten Gesamtsystemen häufig schwer trennbar (vgl. Mishra et al. (2025), Abschn. 3–5; vgl. Cuadron et al. (2025), Abschn. 5–8; vgl. Zwerdling et al. (2025), Abschn. 3–6). Die Entwurfslogik folgt deshalb dem Zeitpunkt und der Autorität des Eingriffs. Die semantische Unterstützung setzt vor der Entscheidung an. Die Pre-Write-Kontrolle greift vor der Ausführung ein.

Für das Verständnis reichen vier Rollen. Der native Agent handelt. Er liest Informationen, spricht mit dem Nutzer und erzeugt mögliche Tool-Aufrufe. Komponente A liegt davor. Sie erinnert den Agenten an belegte Nutzerziele, entscheidet aber nichts selbst. Komponente B liegt danach. Sie sieht einen bereits geplanten Schreibaufruf und prüft, ob genau dieser Aufruf mit dem sichtbaren Zustand vereinbar ist. Die Kombination beider Komponenten ist deshalb keine dritte Superkomponente. Es bedeutet nur, dass beide Eingriffspunkte im selben Agentenlauf aktiv sind.

Diese Trennung ist inhaltlich wichtig. Komponente A adressiert den möglichen Verlust mehrteiliger Nutzerziele im Dialogverlauf. Komponente B adressiert die Ausführbarkeit einer konkreten Schreibaktion. Komponente B erhält keine Unterstützungskarte von Komponente A. Ein kombinierter Effekt kann daher nur entstehen, wenn Komponente A den vorherigen Agentenpfad verändert und Komponente B anschließend den tatsächlich entstandenen Schreibkandidaten prüft.

Aus diesen beiden Eingriffspunkten entstehen die vier Bedingungen der Evaluation. C0 ist der unveränderte native Referenzpfad, C1 aktiviert nur A, C2 nur B und C3 beide Komponenten. C1 bis C3 verwenden denselben Adapter, dasselbe Agentenmodell, dieselben Tools und

dieselben Laufzeitgrenzen. Sie unterscheiden sich ausschließlich in den beiden Faktorflags. Die experimentelle Einbindung dieser Bedingungen wird in Kapitel 4 beschrieben.

Der Aufbau lässt sich auf fünf Anforderungsketten zurückführen. Erstens verlangen DR2 und DR8 Informationsintegrität. Laufzeitkomponenten dürfen nur sichtbare Quellen nutzen, während Referenzlösung und Evaluator physisch getrennt bleiben. Daraus folgen Zustandsprojektion, Quellenverweise und der Offline-Evaluator. Zweitens begrenzen DR3 und DR6 die Interventionsautorität. Komponente A bleibt beratend, Komponente B darf nur disponieren, und eine Zurückhaltung benötigt einen begrenzten Folgepfad. Daraus folgen Kandidatenidentität, Evidenzanforderung und höchstens eine native Neuplanung. Drittens sichern DR1, DR5 und DR9 die Identifizierbarkeit. Faktorflags, stabile Eingriffsgelegenheiten und ein gemeinsames Ereignisprotokoll machen sichtbar, welche Komponente zu welchem Zeitpunkt aktiv war. Viertens verlangt DR7 die gemeinsame Beobachtung von Ergebnis, Regression und Aufwand. Deshalb werden fachliche Bewertung und Ressourcenzurechnung erst offline verbunden. Fünftens begrenzt DR4 die Reichweite des Beitrags. Untersucht werden zwei modulare Laufzeitinterventionen und daraus abgeleitetes konditionales Designwissen. Diese Anspruchsgrenze wird in Artefaktbezeichnung, Evaluation und Ergebnisinterpretation durchgängig beibehalten.

Damit besitzt jede Komponente ein begründetes Kontrollziel. Die Unterstützungskarte begegnet dem Verlust mehrteiliger Nutzerziele ohne Schreibautorität. Die kandidatengebundene Pre-Write-Kontrolle begrenzt riskante Zustandsänderungen ohne verdeckte Neuplanung. Das Ereignisprotokoll verbindet beide mit Saved, Broken und Mechanismusereignissen.

Die Architektur trennt zugleich lokale und globale Korrektheit. Lokal korrekt ist Komponente A, wenn nur belegte Nutzerziel-Datensätze materialisiert werden, und Komponente B, wenn sie einen Kandidaten spezifikationsgemäß disponiert. Global erfolgreich ist eine Episode erst, wenn der vollständige Geschäftsfall mit richtigem Endzustand und richtigem Mutationsmultiset endet. Diese Ebenen können auseinanderfallen. Eine korrekte Unterstützungskarte kann vom Agenten unzweckmäßig genutzt werden. Eine korrekte Zurückhaltung durch Komponente B kann in einer erfolglosen Neuplanung enden. Das Design macht diese Differenz absichtlich messbar, statt lokale Komponentenfunktion als Nutzen des Gesamtverlaufs zu definieren.

Entwicklung zur finalen Artefaktversion

Der finale Zuschnitt ist das Ergebnis formativer Build-and-Evaluate-Zyklen. Tabelle 24 im Anhang dokumentiert die dabei maßgeblichen Gestaltungsentscheidungen. Die formativen Beobachtungen dienten ausschließlich der Konstrukt- und Vertragsverbesserung. Sie werden nicht als Nachweis der späteren Wirksamkeit von C1, C2 oder C3 verwendet.

(1) *Experimentsteuerung* und (2) *User-Simulator*. Die Experimentsteuerung bindet vor Episodenbeginn Task, Seed und Faktorbedingung zu einer Ausführungszelle und konfiguriert den gemeinsamen Laufzeitpfad. Der User-Simulator übersetzt das Szenario in Nutzeräußerungen, ohne private Aufgabenbeschreibung, Referenzlösung oder erwarteten Endzustand offenzulegen. Damit bestimmt die Steuerung die Versuchsbedingung, während der Simulator die dialogische Umwelt unter der Informationsparität aus DR2 realisiert.

(3) *Laufzeit-Orchestrator*. Auf dem kontrollierten Eingang setzt der Laufzeit-Orchestrator auf. Er aktualisiert den sichtbaren Prozesszustand, schaltet Komponente A oder deren No-op-Pfad, ruft den nativen Agenten auf und leitet Schreibkandidaten bei aktivierter Komponente B an die Pre-Write-Kontrolle. C0 bis C3 sind damit Konfigurationen desselben Programms und unterscheiden sich nur in den beiden Faktorflags. Der native Agent bleibt der gemeinsame Referenzkern. Diese Komposition sichert Faktortrennung und gleiche Eingriffsgelegenheiten.

Für die LLM-basierten Verarbeitungsschritte in den Bereichen (2) und (3) gilt eine gemeinsame Ausführungsbedingung: User-Simulator, semantische Extraktion in Komponente A und nativer Agent nutzen denselben externen LLM-Dienst, jedoch über voneinander getrennte API-Anfragen mit jeweils explizit übergebenem Kontext. Zwischen Task-Seed-Zellen wird kein gemeinsames anwendungsseitiges Sitzungs- oder Agentengedächtnis fortgeführt. Diese Zustandslosigkeit verhindert keine stochastische Modellvariation, schließt aber einen zellenübergreifend fortgeschriebenen Anwendungszustand als systematischen Erklärungspfad aus. Verbleibende Unterschiede zwischen Wiederholungen werden deshalb durch identische Konfigurationen, die Paarung der Bedingungen innerhalb eines Seeds und die Wiederholung über sechs Seeds empirisch adressiert.

(4) *Retail-Umgebung*. Die vom Laufzeit-Orchestrator erzeugten Aktionen treffen dort auf Policy, Toolverträge und den je Zelle zurückgesetzten Ausgangszustand. Leseergebnisse fließen als Beobachtungen zurück. Schreibaktionen erreichen die Umgebung bei aktivierter Komponente B erst nach Freigabe. Nur die Retail-Umgebung führt Werkzeuge aus und verändert persistenten Zustand.

(5) *Evaluation*. Nach Abschluss des Laufzeitpfads verbindet die deterministische Evaluation Episodendatensatz, finalen Datenbankzustand und Referenzdaten. Letztere gelangen erst jetzt in die Verarbeitung und können den Lauf nicht beeinflussen. So bleiben lokale Komponentenfunktion, Wirkung auf den Agentenpfad und episodischer Enderfolg getrennte Analyseebenen. Eine korrekte Entscheidung von Komponente A oder B gilt nicht automatisch als Falllösung.

Der sichtbare Zustand enthält ausschließlich Informationen, die aus regulären Nutzer-, Agenten- und Werkzeugereignissen rekonstruierbar sind. Dazu gehören Nutzeräußerungen, authentifizierte Identitäten, beobachtete Nutzer-, Bestell- und Produktdaten sowie erfolgreiche Schreibaktionen. Jeder relevante Wert trägt Quellenverweise und eine Zustandsrevision.

Task-ID, versteckte Simulatorinstruktion, erwartete Aktionen, Gesamtscore und erwarteter Datenbankzustand sind ausgeschlossen. Diese Grenze operationalisiert die Informationsparität aus DR2 und die Trennung von Laufzeit und Referenzlösung aus DR8. Tabelle 12 fasst Eingriffszeitpunkt, Ausgabe und Autorität der Komponenten zusammen.

Komponente	Zeitpunkt	Funktion und Ausgabe	Autorität
Laufzeit-Orchestrator	je Ereignis und Agententurn	Komponiert Zustandsaktualisierung, A- oder No-op-Pfad, nativen Agenten sowie B- oder No-op-Pfad in fester Reihenfolge.	Koordination, keine fachliche Entscheidung
Zustandsprojektion	fortlaufend	Rekonstruiert sichtbare Fakten, Quellen und Zustandsrevisionen.	beobachtend, keine Handlung
Komponente A: semantische Unterstützung	vor nativer Entscheidung	Erzeugt quellengebundene Nutzerziele und gegebenenfalls eine Unterstützungskarte.	beratend
Nativer Agent	je Dialogturn	Plant Leseaktionen, Antworten und mögliche Schreibkandidaten.	Planung
Komponente B: Pre-Write-Kontrolle	vor einer Mutation	Bewertet den exakten Kandidaten als gültig, evidenzbedürftig oder ungültig.	kontrollierend
Audit und Telemetrie	ereignisbasiert	Verknüpft Eingriffsgelegenheit, Entscheidung, Intervention, Folgeaktion, Kosten und Zeit.	beobachtend
Offline-Evaluator	nach Episodenabschluss	Ermittelt DB Match, Exact Mutation und nicht zugeordnete Aktionen aus getrennter Referenzlösung.	Offline-Bewertung

Tabelle 12. Komponenten, Zeitpunkte und Autoritätsgrenzen des Laufzeit- und Evaluationspfads

Schnittstellenverträge und Invarianten

Die technische Trennung wird durch explizite Ein- und Ausgabeverträge abgesichert. Jede Laufzeitkomponente erhält nur die Daten, die sie für ihre jeweilige Aufgabe benötigt. Die Zustandsprojektion erzeugt dafür einen versionierten Schnappschuss mit sichtbaren Fakten und deren Herkunft. Der semantische Modellaufruf von Komponente A erhält ausschließlich die neueste sichtbare Nutzeräußerung und das erlaubte Zielschema. Die anschließende deterministische Prüfung greift auf den sichtbaren Zustand zu, um Quellen, Nutzerautorität und Wertbindung der vorgeschlagenen Felder zu validieren. Die akzeptierte Ausgabe besteht ausschließlich aus typisierten Nutzerziel-Datensätzen und einer optionalen Textrepräsentation der Unterstützungskarte. Insbesondere fehlen Felder für Werkzeugname, Werkzeugargumente,

Freigaben oder Blockierungen. Damit kann Komponente A bereits auf Vertragsebene keine Mutation veranlassen.

Der native Agent erzeugt unabhängig davon einen Werkzeugkandidaten mit Werkzeugname, Argumenten und Call-ID. Unmittelbar danach bewertet Komponente B diesen Kandidaten gegen den zu diesem Zeitpunkt projizierten sichtbaren Zustand. Es erhält weder die Nutzerziel-Datensätze noch die Unterstützungskarte. Sein Ergebnis darf nur eine der drei Dispositionen sowie einen Begründungscode und gegebenenfalls einen eindeutig bestimmten Evidenzabruf enthalten. Bei einer Freigabe wird die Identität des eingehenden und ausgehenden Kandidaten geprüft. Werkzeugname und Argumente dürfen nicht verändert werden. Eine fachlich plausible, aber von Komponente B selbst erzeugte Reparatur wäre damit ein Vertragsverstoß und keine zulässige Intervention.

Vier Invarianten gelten über den gesamten Episodenverlauf. Erstens ist die Ereignisfolge append-only. Bestehende Nutzer-, Agenten- und Toolereignisse werden nicht nachträglich überschrieben. Zweitens referenziert jeder abgeleitete Fakt seine Quellen und die Zustandsrevision. Drittens besitzt jede Intervention eine stabile ID, die genau einer Eingriffsgelegenheit und ihrer nachfolgenden Disposition zugeordnet ist. Viertens bleiben Evaluationsdaten außerhalb des Laufzeit-Datenflusses. Diese Invarianten dienen nicht nur der Reproduzierbarkeit. Sie begrenzen zugleich die möglichen kausalen Pfade. Ein Unterschied zwischen C0 und C1 kann nicht durch Komponente B entstanden sein. Ein unmittelbares Zurückhalten eines identischen Schreibkandidaten in C2 nicht durch Komponente A. Die faktorielle Zuordnung wird somit durch Architektur und Vertrag gemeinsam gestützt.

6.3 Laufzeitverhalten

Ablauf eines Agententurns

Abbildung 5 zeigt den vollständigen Entscheidungsfluss. Eine Nutzeräußerung aktualisiert zunächst den sichtbaren Zustand. Ist Komponente A aktiv und enthält die Äußerung eine transaktionale Anfrage, wird sie höchstens viermal je Episode schema-gebunden analysiert. Der native Agent entscheidet anschließend mit oder ohne Unterstützungskarte. Natürlichsprachige Antworten und Leseaktionen passieren Komponente B unverändert. Nur ein Schreibkandidat eröffnet bei aktivierter Komponente B eine Kontrollgelegenheit.

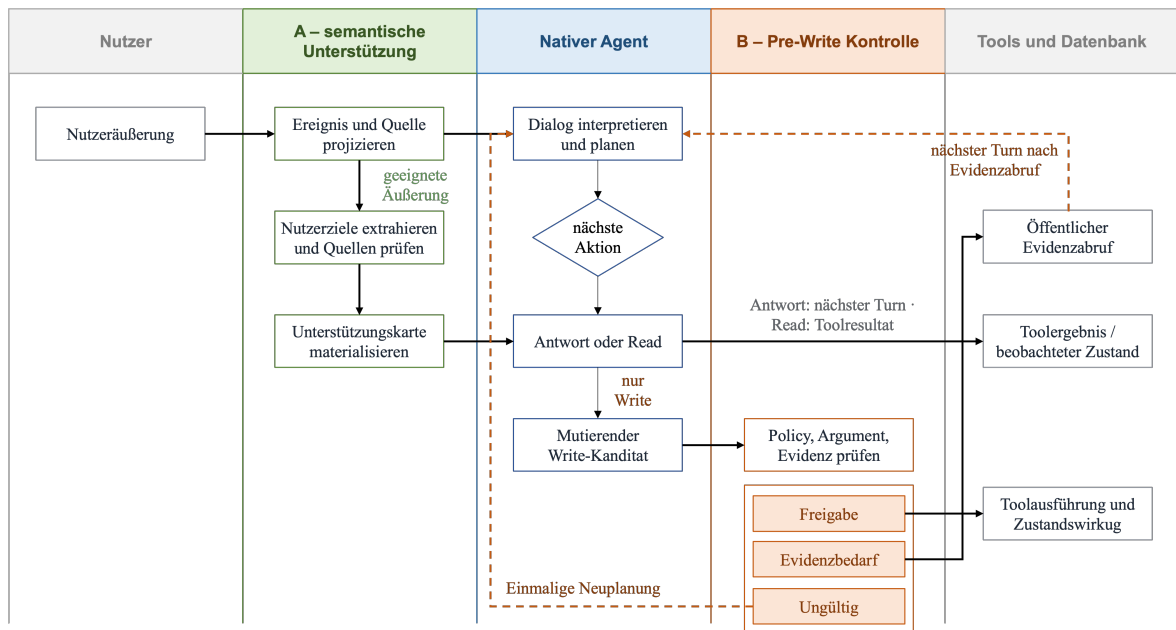


Abbildung 5. Laufzeitsequenz eines Agententurns mit semantischer Unterstützung und Pre-Write-Kontrolle

Komponente A: semantische Unterstützung

Komponente A wird nur bei einer neuen, materiellen Nutzeräußerung aktiv. Ein separater schema-gebundener Modellaufruf darf Vorschläge für sieben festgelegte Retail-Operationen erzeugen, etwa Stornierung, Adress- oder Zahlungsänderung, Rückgabe und Umtausch. Jeder vorgeschlagene Feldwert muss auf sichtbare Nutzerereignisse verweisen. Die Quellenprüfung akzeptiert einen Eintrag nur, wenn Operationsart und Handlungsanker in diesen Quellen tatsächlich belegt sind. Nutzerereignisse sind die einzige Quelle neuer Zielinhalte. Toolresultate dürfen Zustandsfakten validieren und sichtbar erfolgreiche Writes als abgeschlossen markieren, aber kein neues Nutzerziel begründen. Schema-, Transport- oder Quellenfehler enden als protokollierter Fall ohne Eingriff. Der native Pfad läuft weiter.

Die Unterstützungskarte entsteht in fünf getrennten Schritten. Erstens entscheidet ein einfacher Trigger anhand der jüngsten Nutzeräußerung, ob überhaupt eine transaktionale Anfrage vorliegt. Kurze Bestätigungen oder Dankesformeln lösen A nicht aus. Zweitens erhält der semantische Modellaufruf ausschließlich die neueste sichtbare Nutzeräußerung, das erlaubte Zielschema und die zulässigen Feldnamen. Der Modellaufruf darf damit nur strukturierte Vorschläge erzeugen. Er erstellt keine IDs, keine Statuswerte, keine Policyaussagen und keine Abschlussbehauptungen. Drittens prüft die Laufzeit jeden vorgeschlagenen Eintrag deterministisch gegen Schema, erlaubte Operation, erlaubte Felder, sichtbare Quellenverweise, Nutzerautorität und belegbare Feldwerte. Regex-Prüfungen werden dabei nur begrenzt für Trigger, Wortgrenzen und Identifier-Formate verwendet. Sie erzeugen nicht den Inhalt der Unterstützungskarte. Viertens werden akzeptierte Einträge über eine kanonische Signatur

dedupliziert und als unveränderliche Nutzerziel-Einträge gespeichert. Bereits gespeicherte Evidenz wird nicht rückwirkend überschrieben. Fünftens rendert die Laufzeit aus dem aktuellen Nutzerzielprotokoll eine Karte mit festem Präfix und kanonisch sortiertem JSON-Payload. Deterministisch ist somit nicht die semantische Extraktion selbst, sondern die Validierung, Deduplikation und Materialisierung aus akzeptierten Einträgen.

Ein einzelnes Nutzerziel bleibt bewusst unsichtbar, weil der native Agent einfache Anfragen regelmäßig selbst verarbeiten kann. Erst mindestens zwei Nutzerziele materialisieren eine Unterstützungskarte. Sie enthält Nutzerziel-ID, Operation, explizit geerdete Felder sowie bereits sichtbar abgeschlossene Schreibaktionen, aber weder eine Handlungsempfehlung noch eine Zulässigkeitsaussage. Überschreitet die Karte das Zeichenbudget oder kann sie nicht aus validierten Einträgen gebaut werden, entsteht kein Ersatztext. Der native Pfad läuft ohne Karte weiter. Dadurch ist A selektiv auf komplexere, mehrteilige Fälle gerichtet, ohne einfache Verläufe unnötig mit zusätzlichem Kontext zu belasten.

Komponente B: Pre-Write-Kontrolle

Komponente B wird ausschließlich für Tool-Aufrufe aus der eingefrorenen Registry mutierender Retail-Werkzeuge aufgerufen. Eingaben sind der unveränderte Kandidat und der zum Zeitpunkt seiner Erzeugung sichtbare Zustand. Geprüft werden unter anderem Authentifizierung, Eigentum, beobachteter Bestellstatus, Artikelzugehörigkeit, Listenlängen, Produktfamilie und Variantenverfügbarkeit sowie Zahlungs- und Refundbedingungen. Die Regeln übersetzen öffentliche Policy und Toolverträge in ausführbare Prädikate. Sie werden nicht zur Laufzeit durch ein LLM erzeugt.

Komponente B unterscheidet drei Ergebnisse. `VALID_UNCHANGED` gibt ausschließlich den identischen nativen Kandidaten frei. `EVIDENCE_REQUIRED` hält ihn zurück und führt einen eindeutig bestimmten öffentlichen Evidenzabruf aus. `INVALID` hält ihn mit einem sichtbaren Begründungscode zurück und erlaubt dem nativen Agenten genau eine Neuplanung. Bleibt auch der revidierte Kandidat invalide, endet der autonome Pfad durch den regulären Transfermechanismus. Bei mehreren gleichzeitigen Tool-Aufrufen wird kein partiell gültiges Bündel ausgeführt. Dialogische Bestätigungen bleiben Teil der nativen Policylogik und werden nicht von Komponente B übernommen.

Zustandsübergänge und Fehlersemantik

Die beiden Komponenten folgen unterschiedlichen Lebenszyklen. Ein Nutzerziel-Datensatz von Komponente A beginnt als extrahierter Vorschlag, wird entweder verworfen oder nach Quellenprüfung als geerdet gespeichert und kann später als erledigt oder überholt markiert werden. Eine Unterstützungskarte ist daher keine dauerhafte Zusatzinstruktion, sondern eine aus dem aktuellen Protokoll deterministisch erzeugte Sicht. Neue sichtbare Evidenz aktuali-

siert diese Sicht. Sie ändert jedoch nicht rückwirkend die Quellen eines zuvor akzeptierten Eintrags. Bei einem Schema-, Transport- oder Quellenfehler entsteht kein Ersatzeintrag. Stattdessen wird der Fehler mit seinem Typ protokolliert und der native Pfad ohne Karte fortgesetzt. Dieses offene Fehlerverhalten schützt die Verfügbarkeit, verhindert aber bewusst, dass technische Fehler von Komponente A als fachliche Blockierung erscheinen.

Eine Eingriffsgelegenheit von Komponente B beginnt erst mit einem vollständigen Schreibkandidaten. Dieser wird synchron gegen den unmittelbar zuvor projizierten sichtbaren Zustand geprüft. Bei `EVIDENCE_REQUIRED` wird der ursprüngliche Kandidat nicht ausgeführt. Stattdessen erfolgt genau der von Komponente B bestimmte öffentliche Leseaufruf. Dessen Ergebnis erweitert im folgenden Turn den sichtbaren Zustand, auf dessen Grundlage der native Agent erneut plant. Bei `INVALID` erhält der Agent unmittelbar genau eine Neuplanung mit den deterministischen Befunden von Komponente B. Komponente B setzt deshalb weder fehlende Argumente ein noch führt sie den ursprünglichen Call nachträglich aus. So bleibt erkennbar, ob die Folgeaktion eine Bestätigung, eine Korrektur, ein Abbruch oder eine unveränderte Wiederholung war.

Für technische Ausnahmefälle gilt eine konservative, vorab festgelegte Semantik. Die eingefrorene Registry definiert die sieben mutierenden Retail-Werkzeuge, für die Komponente B Eingriffsgelegenheiten erzeugt. Nicht verfügbare Werkzeugnamen bleiben außerhalb dieses Kontrollvertrags und werden von der Werkzeugschnittstelle des Benchmarks abgewiesen. Ein unvollständiges Bündel wird atomar zurückgehalten. Eine partielle Ausführung könnte sonst einen Zustand erzeugen, der keiner untersuchten Bedingung mehr eindeutig zugeordnet werden kann. Nach ausgeschöpftem Neuplanungsbudget erfolgt keine verdeckte weitere Intervention. Der Fall endet im vorgesehenen Transfer- oder Abbruchpfad und bleibt im Protokoll als unaufgelöste Eingriffsgelegenheit sichtbar. Damit werden technische Ausnahmen nicht als erfolgreiche Schutzwirkung fehlklassifiziert.

6.4 Technische Umsetzung und Designverifikation

Modulare Implementierung

Der Prototyp ist in Python implementiert und über einen Retail-Adapter in den nativen τ 2-bench-Ablauf eingebunden. Die sechs final ausgewerteten Seeds verwenden den in den Ausführungsverträgen hashgebundenen Pfad `retail/factorial_runtime_support`. Dieser Pfad umfasst vier fachlich getrennte Module: typisierte Datenverträge, semantische Unterstützung, deterministische Pre-Write-Kontrolle und den Adapter, der beide Faktoren in den Nachrichtenfluss einbindet. Faktorflags werden über ein gemeinsames Ausführungsmanifest gesetzt. Agenten- und Usermodell, Temperatur, Reasoning Effort, Policy, Tools, Schrittbudget und Evaluator bleiben über die Bedingungen hinweg konstant.

Die Ausgabe von Komponente A ist schemabeschränkt und wird anschließend deterministisch geordnet. Komponente B enthält keinen Modellaufruf. Beide Komponenten schreiben ereignisbasierte Einträge in dasselbe unveränderliche Audit-Protokoll. Stabile IDs für Eingriffsgelegenheiten, Karten, Bündel und Aufrufe erlauben es, eine Intervention mit ihrer folgenden nativen Entscheidung, einer tatsächlich ausgeführten Aktion und dem Episodenergebnis zu verknüpfen.

Technisch verteilen sich diese Aufgaben auf wenige klar getrennte Module: Verträge und Zustandsmodell, semantische Unterstützung, Pre-Write-Kontrolle, Adapter und Offline-Auswertung. Für die Thesis ist dabei weniger der einzelne Dateiname entscheidend als die Richtung der Abhängigkeiten. Die Laufzeit kennt keine Referenzlösung, Komponente B enthält keinen Modellaufruf, und die wirtschaftliche Entscheidung liest erst nach der Evaluation die fertigen Offline-Ergebnisse.

Konfigurations- und Ausführungsmodell

Eine Bedingung wird nicht durch unterschiedliche Runner oder bedingungsspezifische Prompts implementiert. Das gemeinsame technische Basemanifest definiert Reasoning-Einstellung, Temperatur, Toolset, Dialogschrittbudget und Laufzeitgrenzen. Der darüberliegende Studienvertrag bindet das untersuchte Nano-Modell, Aufgabenpopulation und Seeds. Die Bedingungsmanifeste referenzieren diese Basis über Hashes und setzen lediglich die beiden booleschen Faktorflags für semantische Unterstützung und Pre-Write-Kontrolle. C1, C2 und C3 werden dadurch zu Konfigurationen desselben Programms. Diese Struktur reduziert das Risiko, dass eine unbeabsichtigte Code- oder Promptdifferenz fälschlich als Komponentenwirkung erscheint.

Vor dem Start validiert der Runner Schema und Hashes der referenzierten Konfigurationen. Für jede Zelle werden Task-ID, Seed, Bedingung und Versuchskennung zu einer eindeutigen Job-ID verbunden. Ein abgeschlossener Job schreibt Ergebnis, Nutzung, Zeitangaben und Verweise auf Ausführungsspur und Protokoll. Ein Retry erzeugt keinen zweiten analytischen Datenpunkt, sondern bleibt als weiterer technischer Versuch desselben Jobs sichtbar. Dadurch können Vollständigkeit und Wiederholungen geprüft werden, ohne aus mehreren Versuchen nach Ergebnis auszuwählen.

Die Trennung setzt sich in der Auswertung fort. Der Laufzeitpfad kennt weder die erwarteten Aktionen noch den erwarteten Endzustand. Der Offline-Evaluator liest erst die abgeschlossenen, unveränderten Zellen, normalisiert die ausgeführten Mutationen und erzeugt die analytischen Einträge. Der Entscheidungsteil akzeptiert wiederum nur die gehashte Ausgabe dieses Evaluators und einen getrennten Parametervertrag. Damit existiert eine gerichtete Kette von Konfiguration über Ausführung und Evaluation bis zur betrieblichen Abwägung.

Eine spätere Bewertung von Aufwand oder Fehlerfolgen kann die Interpretation der bereits beobachteten Folgen verändern, aber weder den Agentenlauf noch den primären Score.

Beobachtbarkeit

Für jede natürliche Aktivierungsmöglichkeit wird eine konsistente Kette protokolliert:

Eingriffsgelegenheit → Entscheidung → Intervention → Folgeaktion → finale Disposition

Bei A umfasst sie unter anderem akzeptierte und verworfene Einträge, Nutzerziel- und Karten-IDs, Quellenereignisse sowie die Verknüpfung einer Karte mit einem späteren Schreibkandidaten. Bei B umfasst sie Kandidat und Argumente, Begründungscode, benötigten Evidenzabruf, Neuplanung und finale Ausführung oder Blockierung. Kosten und Laufzeit der semantischen Unterstützung werden vom nativen Agentenaufruf getrennt erfasst. Der Observer verändert keine Laufzeitobjekte und besitzt selbst keine Interventionsautorität.

Designverifikation und Freeze

Die Verifikation trennt technische Korrektheit von empirischer Wirksamkeit. Vor der ersten faktoriellen Matrix bestanden die festgelegten automatisierten Laufzeit- und Vertragsprüfungen sowie eine formative Prüfung der Ereignisspuren. Geprüft wurden insbesondere vollständige Quellenbindung, Nichtinterferenz bei ausbleibender Unterstützungskarte, persistente Compound-Nutzerziele, Kandidatenidentität, LLM-Freiheit von Komponente B, ausbleibende partielle Bündelausführung, höchstens eine Neuplanung und vollständiger Protokollabgleich. Ein zusätzlicher Generalitätscheck prüfte die technische Funktionsfähigkeit auf zuvor nicht in der Ereignisspurprüfung verwendeten Retail-Fällen.

Tabelle 36 im Anhang ordnet die neun Gestaltungsanforderungen ihren Akzeptanzkriterien, ausführbaren Prüfungen und Evidenzartefakten zu und bildet den revisionsfähigen Abnahmenachweis. Sechs Prüffragen verdichten die Einzelprüfungen auf die zentralen Freeze-Grenzen.

Akzeptiert wurde die Artefaktversion nur, wenn sechs Prüffragen bestanden waren: Ist A quellengebunden und rein beratend? Ist B eng, deterministisch und ohne Zugriff auf A? Bleiben die Faktoren über zwei Flags identifizierbar? Bleibt die Referenzlösung außerhalb der Laufzeit? Sind Eingriffe begrenzt und vollständig protokolliert? Und bleiben Evidenz, Simulator-Kosten und externe Kostenbewertung getrennt? Nach bestandener Prüfung wurden Implementierung, Konfiguration und Analysevertrag gebunden. Dies belegt technische Korrektheit, aber keinen positiven Effekt auf DB Match, Exact Mutation, Kosten oder Laufzeit. Diese Trennung führt zur Demonstration in Kapitel 7 und zur Evaluation in Kapitel 8.

7 Demonstration des Artefakts

Kapitel 6 beschreibt Komponente A als semantische Unterstützung vor der nativen Entscheidung und Komponente B als Pre-Write-Kontrolle vor einer Schreibaufführung. Dieses Kapitel demonstriert beide Eingriffspunkte innerhalb *eines* zusammenhängenden Laufzeitverlaufs. Im Sinne der Design-Science-Logik wird damit gezeigt, dass das Artefakt im vorgesehenen Anwendungskontext ausführbar ist und die entworfenen Mechanismen beobachtbar hervorbringt (vgl. Peffers et al. (2007), S. 55–56). Die Demonstration beantwortet noch nicht, wie regelmäßig die Mechanismen helfen oder ob sie dem nativen Agenten im Mittel überlegen sind. Diese Wirkung wird erst in Kapitel 8 über die vollständige Aufgaben-Seed-Matrix bewertet.

7.1 Demonstrationsrahmen und Prozessübersicht

Verwendet wird Task 35, Seed 976302 unter der kombinierten Bedingung C3. Der Fall ist für die Demonstration geeignet, weil sich die Komponenten A und B im selben Transaktionsverlauf nacheinander beobachten lassen. Der Nutzer verfolgt zwei Ziele in unterschiedlichen Bestellungen: Einen gelieferten Lautsprecher zurückgeben und einen 17-Zoll-Laptop in einer noch offenen Bestellung durch eine 13-Zoll-Variante ersetzen. Die erste Äußerung lautet auszugsweise:

„I need to return a speaker I bought because it’s overpriced and it can’t even handle a splash of water. Also, I want to change the 17-inch laptop in my other order to a 13-inch one instead.“

Der Fall verlangt mehr als das lokale Verstehen dieses Satzes. Im weiteren Dialog ändern sich die verfügbare Identitätsevidenz, das Zahlungsmittel, die betroffenen Artikel-IDs und die gewünschte Laptopvariante. Zugleich wird das erste Ziel bereits ausgeführt, während das zweite noch offen ist. Der Gestaltungsbeitrag von Komponente A liegt hier nicht in einer vermeintlich besseren Sprachinterpretation desselben Satzes, sondern in einem expliziten, quellengebundenen Zielzustand über diese Übergänge hinweg. Komponente B setzt später an einem anderen Objekt an, nämlich dem vom nativen Agenten erzeugten Schreibkandidaten.

Leseschlüssel der Ereignisspur-IDs

Die nachfolgenden Kennungen werden von der Laufzeit innerhalb einer Episode fortlaufend erzeugt. Sie verbinden Ereignisse und Zustandsstände, bewerten diese aber nicht. Eine höhere Nummer bedeutet lediglich ein späteres Auftreten und weder höhere Qualität noch Erfolg. Tabelle 25 im Anhang definiert die in der Fallrekonstruktion verwendeten Präfixe.

Die IDs sind hierarchisch zu lesen. Ein Nutzerereignis eröffnet eine AOP; akzeptierte Inhalte werden als G-Datensätze gespeichert und gegebenenfalls in einer AC materialisiert. Der native Agent erzeugt daraus oder aus seinem übrigen Dialogkontext einen Toolkandidaten. Dieser eröffnet unmittelbar vor einem Schreibauftrag eine BOP. Komponente B prüft den zugehörigen Kandidaten beziehungsweise das BB-Bundle. Erst eine freigegebene `call_...`-ID bezeichnet den tatsächlich an das Tool weitergereichten Aufruf. Damit lassen sich semantischer Zustand, nativer Kandidat, Kontrollentscheidung und Ausführung auseinanderhalten.

Chronologischer Prozessverlauf

Tabelle 13 rekonstruiert den gesamten Pfad aus den kanonischen Mechanismusereignissen. Die Sequenznummern entsprechen der Reihenfolge in der gespeicherten Ereignisspur. Ausgelassene Nummern sind wiederholte Karteninjektionen oder native Dialog- und Leseschritte.

Seq.	Stufe	Beobachteter Zustandsübergang
0–3	A: initiale Extraktion	Zwei Nutzerziele werden aus <code>message:1:user</code> extrahiert, als G-0001/G-0002 gespeichert und in AC-0001 materialisiert (AOP-0001).
4–8	A: Konkretisierung	Die korrigierte E-Mail und der zunächst gewünschte Rückzahlungsweg aus <code>message:3:user</code> aktualisieren den Zielzustand; die Karte wird als AC-0002 neu erzeugt.
15–20	A: handlungsfähiger Stand	Nach nativen Leseaktionen konkretisiert der Nutzer Lautsprecher, PayPal und Laptopziel. Komponente A akzeptiert drei Änderungen; die Karte AC-0003 enthält nun fünf quellengebundene Nutzerziele.
21–24	A → B: erste Schreibaktion	Die Karte erreicht einen nativen Rückgabekandidaten. Komponente B prüft ihn, entscheidet <code>VALID_UNCHANGED</code> und der Schreibauftrag <code>call_yzIA...</code> wird unverändert ausgeführt (BOP-0001).
25–27	A: Fortschrittsabgleich	Der sichtbare Toolerfolg wird in AC-0004 als verifiziert abgeschlossene Schreibaktion geführt; das noch offene Laptopziel bleibt erhalten.
28–31	B: Evidenzbedarf	Der native Laptopkandidat verweist auf die Variante 3778566150. Weil deren Produktzustand nicht beobachtet wurde, hält B den Kandidaten BOP-0002 zurück und fordert eine gezielte Produktabfrage an.
32–39	Leseaktion, Dialog und Aktualisierung durch Komponente A	Die Leseaktion zeigt, dass die Variante nicht verfügbar ist. Nach nativer Alternativenklärung wählt der Nutzer „the i5 in silver“; A aktualisiert G-0002 und materialisiert AC-0005.
40–43	A → B: zweite Schreibaktion	Ein neuer nativer Kandidat verwendet die verfügbare Variante 5052031638. Komponente B entscheidet <code>VALID_UNCHANGED</code> ; <code>call_eY0C...</code> wird unverändert ausgeführt (BOP-0003).
44–46	A: Endzustand	AC-0006 führt beide erfolgreichen Schreibaktionen als verifiziert abgeschlossen.

Tabelle 13. Chronologische Ereignisspur des kombinierten Demonstrationsfalls

7.2 Komponenteninteraktion im Laufzeitpfad

Komponente A: semantische Unterstützung

Beim ersten materiellen Nutzerereignis erzeugt A die Eingriffsgelegenheit AOP-0001. Die Extraktion übernimmt nur Felder, die durch die referenzierte Nutzerquelle belegt sind. Zwei Datensätze werden akzeptiert. Ein nicht hinreichend belegtes Variantenfeld wird verworfen. Der gekürzte Originalausschnitt zeigt damit zugleich Extraktion und Quellenbindung:

```
{
  "sequence": 1,
  "component": "A",
  "opportunity_id": "AOP-0001",
  "disposition": "STORED_SILENTLY",
  "reason_code": "A_LEDGER_UPDATED",
  "source_event_ids": ["message:1:user"],
  "accepted_records": 2,
  "rejected_records": 1,
  "validated_records": [
    {
      "commitment_id": "G-0001",
      "operation": "return_delivered_order_items",
      "fields": {"item_descriptions": "a speaker I bought"}},
    {
      "commitment_id": "G-0002",
      "operation": "modify_pending_order_items",
      "fields": {"item_descriptions":
        "the 17-inch laptop in my other order"}}
  ]
}
```

Listing 1. Gekürzte A-Ereignisspur: initiale Aktualisierung des Zielzustands

STORED_SILENTLY bedeutet, dass die Extraktion keine Nutzerantwort ersetzt. Weitere Äußerungen aktualisieren denselben expliziten Zielzustand. Nach den erforderlichen Leseaktionen bündelt AC-0003 fünf belegte Nutzerziele. Der folgende Ausschnitt zeigt die für den weiteren Ablauf relevanten Kartenfelder.

```

{
  "sequence": 18,
  "component": "A",
  "card_id": "AC-0003",
  "disposition": "SELECTIVE_ADVISORY",
  "trigger_reason": "A_COMPOUND_COMMITMENT_RISK",
  "accepted_records": 5,
  "customer_commitments": [
    { "commitment_id": "G-0001",
      "operation": "return_delivered_order_items",
      "item_ids": ["6704763132"] },
    { "commitment_id": "G-0002",
      "operation": "modify_pending_order_items",
      "item_descriptions": "the 17-inch laptop in my other order" },
    { "commitment_id": "G-0004",
      "desired_variant_constraints":
        "change the 17-inch to the 13-inch one you listed" }
  ],
  "verified_completed_writes": []
}

```

Listing 2. Gekürzte A-Ereignisspur: materialisierte Unterstützungskarte

Die Karte ist somit weder ein zweiter Planer noch eine Toolanweisung. Sie enthält kein Recht zur Ausführung und bestimmt keine Datenbank-ID, die der Nutzer nicht selbst genannt hat. Ihr zusätzlicher Informationswert liegt in der stabilen Trennung von Nutzerzielen, Quellen und Ausführungsstand. Dass ein leistungsfähiger nativer Agent denselben Fall auch allein aus dem Dialog bewältigen kann, bleibt ausdrücklich möglich und wird nicht durch die Demonstration entschieden.

Komponente B: Pre-Write-Kontrolle

Aus AC-0003 erzeugt der native Agent den Rückgabekandidaten BOP-0001. B belegt dessen Argumente im sichtbaren Zustand, gibt ihn unverändert frei und Komponente A übernimmt den Toolerfolg in AC-0004. Die Karte trennt damit im Mehrzielprozess offene Absichten von sichtbar abgeschlossenen Zustandsänderungen.

Beim anschließenden Laptopkandidaten fehlt der beobachtete Produktzustand. B hält die Schreibaktion zurück und fordert eine öffentliche Produktabfrage an. Die gekürzte Ereignisspur stellt Evidenzbedarf und spätere Freigabe gegenüber:

Die Gegenüberstellung ist für die Mechanismuzuordnung entscheidend: EVIDENCE_REQUIRED beendet weder Episode noch Nutzerziel, sondern fordert genau eine fehlende Beobachtung an. Die spätere Disposition VALID_UNCHANGED bezieht sich auf den neu erzeugten Kandidaten BOP-0003, nicht auf eine Reparatur von BOP-0002. Erst die emittierte Call-ID verbindet diesen zweiten Kandidaten mit der tatsächlichen Schreibausführung.

```
[
  {
    "sequence": 30,
    "opportunity_id": "BOP-0002",
    "candidate": {"item_ids": ["1684786391"], "new_item_ids": ["3778566150"]},
    "disposition": "EVIDENCE_REQUIRED",
    "reason_code": "B_PRODUCT_NOT_OBSERVED",
    "evidence_action": {"tool_name": "get_product_details", "product_id": "4760268021"}
  },
  {
    "sequence": 42,
    "opportunity_id": "BOP-0003",
    "candidate": {"item_ids": ["1684786391"], "new_item_ids": ["5052031638"]},
    "observed_product_source": "tool:factorial_b_read_22_32:get_product_details",
    "disposition": "VALID_UNCHANGED"
  },
  {"sequence": 43, "opportunity_id": "BOP-0003",
    "disposition": "executed_unchanged",
    "emitted_call_id": "call_eY0CqYfxQfcuxJNYp7cGNi21"}
]
```

Listing 3. Gekürzte B-Ereignisspur: Evidenzbedarf und unveränderte Freigabe

Zwischen beiden Entscheidungen von Komponente B liegt kein autonomer Korrekturschritt. Nach Produktabfrage und Nutzerwahl aktualisiert A das Ziel. Erst der native Agent erzeugt BOP-0003. Komponente B gibt diesen neuen, nun belegten Kandidaten unverändert frei. AC-0006 weist anschließend beide Schreibaktionen als verifiziert abgeschlossen aus.

Nachweisgrenze und Übergang zur Evaluation

Der Fall belegt in einer durchgängigen Spur die technische Funktionsfähigkeit, Informationsgrenzen und Nachvollziehbarkeit der Arbeitsteilung. Komponente A führt quellengebundene Nutzerziele fort, der native Agent interpretiert und plant, und Komponente B prüft konkrete Schreibkandidaten vor ihrer Ausführung. Eine isolierte Kausalwirkung oder Wirksamkeitsgarantie folgt daraus nicht. Komponente A kann einen nativen Pfad auch ungünstig verändern, Komponente B einen formal belegten, aber fachlich falschen Kandidaten freigeben. Kapitel 8 untersucht deshalb über alle Aufgaben und sechs Seeds Aktivierung, Ergebniswechsel und Aufwand.

8 Ergebnisse

Dieses Kapitel bewertet die finale Artefaktkonfiguration anhand der in Kapitel 4 festgelegten Retail-Evaluation mit 80 Aufgaben, sechs Seeds und vier Bedingungen, insgesamt also 1.920 vollständig auswertbaren Zellen. Die Darstellung folgt der Belegreihenfolge aus fachlichen Endpunkten, faktoriellen Kontrasten, Wiederholungsstabilität, geretteten und beschädigten Verläufen (*Saved/Broken*), Mechanismusereignissen und Ressourcenaufwand. Damit bleiben Ergebnis, Robustheit, Wirkpfad und Aufwand getrennt.

8.1 Fachliche Wirkung

Vollständigkeit und primäre Endpunkte

Für jede Bedingung liegen 480 Episoden vor. *DB Match* ist für alle 80 Aufgaben definiert und prüft den finalen Datenbankzustand. *Exact Mutation* betrachtet die vollständige Ausführung der erwarteten Schreibaktionen und ist für 72 Aufgaben beziehungsweise 432 Zellen je Bedingung auswertbar. Alle 1.920 erwarteten Zellen sind eindeutig vorhanden und offline auswertbar.

Tabelle 14 zeigt zunächst ausschließlich die fachlichen Endpunkte. C1 erreicht bei beiden Maßen die höchste Quote, gefolgt von C3, C2 und C0. Gegenüber C0 entsprechen die Abstände 29 und 27 zusätzlichen Erfolgswellen für C1, 25 und 24 für C3 sowie acht und zwölf für C2.

Bedingung	DB Match	Exact Mutation
C0 Native	342/480 = 71,3 %	293/432 = 67,8 %
C1 Semantische Unterstützung	371/480 = 77,3 %	320/432 = 74,1 %
C2 Pre-Write-Kontrolle	350/480 = 72,9 %	305/432 = 70,6 %
C3 Kombination	367/480 = 76,5 %	317/432 = 73,4 %

Tabelle 14. Fachliche Endpunkte der vier Bedingungen

Die identische Rangfolge beider Endpunkte zeigt, dass das Befundmuster nicht auf eine einzelne Operationalisierung beschränkt ist. Der niedrigere Exact-Wert identifiziert Verläufe, deren plausibler Endzustand durch eine fehlende, zusätzliche oder materiell abweichende Schreibaktion zustande kommt.

Die direkten Kontraste in Tabelle 15 sind auf Aufgabe und Seed gepaart. Als konservative Unsicherheitsangabe wird ein Zweiweg-Bootstrap über Aufgaben und Seeds berichtet. Alle Punktschätzungen sind positiv, alle Intervalle umfassen jedoch null. Die Mittelwertvorteile

sind daher als positive empirische Signale innerhalb der untersuchten Matrix zu lesen, nicht als präzise oder universell gesicherte Effekte.

Kontrast	DB Match: Differenz [95-%-KI]	Exact Mutation: Differenz [95-%-KI]
C1 minus C0	+6,0 pp [−1,9; +13,8]	+6,3 pp [−1,9; +14,1]
C2 minus C0	+1,7 pp [−5,4; +8,8]	+2,8 pp [−5,1; +10,4]
C3 minus C0	+5,2 pp [−2,1; +12,5]	+5,6 pp [−2,3; +13,4]

Tabelle 15. Gepaarte Punktschätzungen und Zweiweg-Bootstrapintervalle gegenüber C0

Die Unsicherheitsbewertung hängt sichtbar davon ab, welche Variation das Verfahren abbildet. Beim Perzentil-Bootstrap mit Clustering nach Aufgaben liegt das 95-%-Intervall für C1 bei DB Match zwischen +1,9 und +10,2 Prozentpunkten und bei Exact Mutation zwischen +1,9 und +10,9 Prozentpunkten. Für C3 liegen die entsprechenden Untergrenzen mit +0,2 Prozentpunkten ebenfalls knapp oberhalb null. Werden Aufgaben und Seeds im Zweiweg-Bootstrap gemeinsam variiert, verbreitern sich die Intervalle und schließen null ein. Die zusätzliche Seedvariation ist folglich materiell für die Präzision der Aussage. C2 umfasst bereits in der nach Aufgaben geclusterten Auswertung null und bleibt nach allen Verfahren unsicher.

Die Verfahren beantworten verschiedene Fragen. Der nach Aufgaben geclusterte Bootstrap variiert die Zusammensetzung der 80 Aufgaben bei gemeinsam erhaltenen Seeds. Der Zweiweg-Bootstrap variiert zusätzlich die realisierte Seedverteilung. Die Punktschätzungen bleiben gleich, für weiterreichende Aussagen gilt die konservativere Zweiweg-Unsicherheit.

Faktorielle Struktur

Die direkten Kontraste bewerten konkrete Konfigurationen. Die faktoriellen Kontraste isolieren dagegen den mittleren Beitrag eines Faktors über beide Stufen des jeweils anderen Faktors. Tabelle 16 berichtet deshalb sowohl ein Intervall über die sechs seedweisen Effekte als auch den konservativeren Zweiweg-Bootstrap.

Effekt	DB Match	Exact Mutation	Seed-t-KI DB / Exact	Zweiweg-KI DB / Exact
Faktor A	+4,8 pp	+4,5 pp	[+0,7; +8,9] / [+0,4; +8,6]	[-0,6; +10,4] / [-1,2; +10,3]
Faktor B	+0,4 pp	+1,0 pp	[-1,8; +2,6] / [-1,6; +3,7]	[-4,7; +5,4] / [-4,5; +6,6]
A×B	-2,5 pp	-3,5 pp	–	[-12,3; +8,1] / [-14,1; +8,1]

Tabelle 16. Faktorielle Punktschätzungen und 95-%-Unsicherheitsintervalle

Die Haupteffekte lassen sich aus den vier Zellmitteln rekonstruieren. Der Effekt von Faktor A bleibt in beiden Stufen von Faktor B positiv, fällt bei aktiviertem Faktor B jedoch kleiner aus. Faktor B zeigt dagegen bei deaktiviertem Faktor A einen leichten Anstieg und bei aktiviertem Faktor A einen leichten Rückgang. Daraus folgen die kleinen Haupteffekte von Faktor B und die negative Interaktionsschätzung in Tabelle 16. Letztere beschreibt eine beobachtete Abschwächung in C3, wegen der breiten Intervalle aber keinen gesicherten negativen Kombinationseffekt.

Faktor A zeigt einen positiven mittleren Haupteffekt. Zudem bleiben sämtliche Schätzungen beim Auslassen jeweils eines Seeds positiv: für DB Match zwischen +4,0 und +5,8 Prozentpunkten, für Exact Mutation zwischen +3,6 und +5,7 Prozentpunkten. Dass die Seed-t-Intervalle oberhalb null liegen, der Zweiweg-Bootstrap jedoch null einschließt, macht die Inferenz methodenabhängig. Der Haupteffekt von Faktor A wird deshalb als konsistenter positiver Hinweis interpretiert. Die Sensitivitätsanalyse zeigt damit, dass die positive Richtung des Kontrasts nicht ausschließlich von einem einzelnen Seed abhängt, ersetzt aber keine Replikation auf einer neuen Aufgaben- oder Modellpopulation. Für Faktor B ergibt sich nach keinem Verfahren ein belastbarer eigenständiger Effekt. Sein Haupteffekt von Faktor B für DB Match reicht in der Seed-Auslassungsanalyse von -0,3 bis +1,0 Prozentpunkten. Die negative Punktschätzung der Interaktion spricht gegen Additivität, ist wegen des breiten Intervalls aber ebenfalls nicht als gesicherter Effekt zu formulieren.

8.2 Stabilität und Ergebniswechsel

Verteilung über sechs Seeds

Die Seedanalyse folgt der in Kapitel 4 begründeten Endpunkthierarchie. DB Match dient aufgrund seiner vollständigen 80-Aufgaben-Population und seiner fachlichen Endzustandsinterpretation als Leitmetrik. Exact Mutation prüft anschließend, ob das Muster im write-positiven Teilkollektiv robust bleibt. Abbildung 6 zeigt links alle sechs beobachteten DB-Quoten je Bedingung. Rechts stehen die gepaarten Seed-Differenzen der drei Interventionen zu C0.

Die Nullachse trennt positive von negativen Richtungen. Dadurch wird keine künstliche Reihenfolge von C1, C2 und C3 unterstellt.

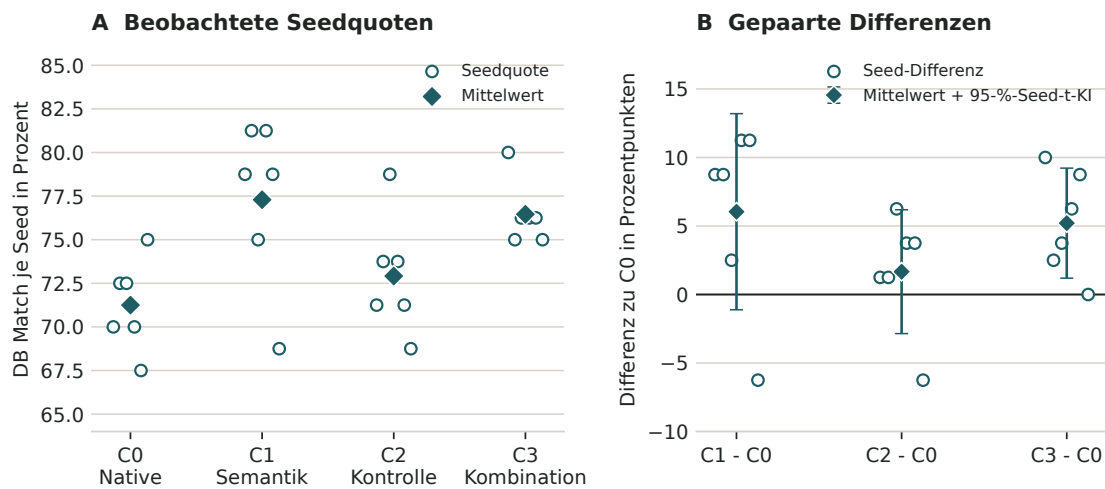


Abbildung 6. DB-Match-Verteilung und gepaarte Kontraste über sechs Seeds

Tabelle 17 quantifiziert die in Abbildung 6 sichtbaren DB-Verteilungen.

Bedingung	Mittelwert	SD	Min–Max
C0 Native	71,3	2,6	67,5–75,0
C1 Semantische Unterstützung	77,3	4,8	68,8–81,3
C2 Pre-Write-Kontrolle	72,9	3,4	68,8–78,8
C3 Kombination	76,5	1,8	75,0–80,0

Tabelle 17. DB Match: Mittelwert, Standardabweichung und Spannweite der sechs Seedquoten in Prozent

Die DB-Rohverteilungen überlappen. Besonders C0 und C2 besitzen gemeinsame Wertebereiche. C1 erreicht zugleich den höchsten Mittelwert und die größte Streuung. C3 weist die engste Verteilung auf, wobei sein Minimum dem Maximum von C0 entspricht. Ein einzelner Lauf erlaubt somit keine verlässliche Rangfolge.

Überlappung allein entscheidet jedoch nicht, ob eine beobachtete Differenz nur aus Laufvariation entsteht, weil die Bedingungen innerhalb jedes Seeds gepaart sind. Die seedweisen Differenzen zeigen, dass kein direkter Kontrast vollständig homogen ist. C1 und C2 liegen bei beiden Endpunkten in fünf Seeds über C0 und im sechsten darunter. C3 liegt bei DB Match fünfmal über und einmal gleichauf mit C0. Bei Exact Mutation liegt C3 in allen sechs Seeds darüber. Dies begrenzt eine pauschale Überlegenheit von C1 trotz des höchsten Gesamtmittels und macht Stabilität zu einer eigenständigen Ergebnisdimension. Die vollständigen seed-

und aufgabenbezogenen Detaildiagnosen beider Endpunkte sind gemeinsam in Anhang A.6 dokumentiert.

Die konsistente Richtung von C3 bei Exact Mutation spricht gegen eine nur durch einen günstigen Seed erzeugte Differenz. Wegen sechs Seeds, überlappender Rohverteilungen und null einschließender Zweiwegintervalle bleibt eine stochastische Erklärung der beobachteten Größenordnung dennoch möglich.

Seedmittelwerte können verdecken, ob stets dieselben Aufgaben erfolgreich sind. Die Verteilung über sechs Seeds unterscheidet deshalb stabile Fehl- und Erfolge von Aufgaben mit wechselndem Ausgang.

Gemischte Ergebnisse in den Klassen 1/6 bis 5/6 treten bei DB Match unter C0 für 52 von 80 Aufgaben (65,0 %), unter C1 für 50 (62,5 %), unter C2 für 49 (61,3 %) und unter C3 für 47 Aufgaben (58,8 %) auf. Die Ergebnisvariation beschränkt sich damit nicht auf einzelne Ausreißer-Seeds, sondern betrifft in jeder Bedingung die Mehrheit der Aufgabenpopulation.

Die mittlere paarweise Seedabweichung verdichtet dasselbe Muster. Bei DB Match unterscheiden sich zwei beobachtete Seeds derselben Aufgabe in C0 in 30,8 % der Paare, in C1 in 27,4 %, in C2 in 28,5 % und in C3 in 26,6 %. C3 zeigt damit die geringste beobachtete aufgabeninterne Seedabweichung. Dies ist ein deskriptives Stabilitätssignal, kein isolierter Nachweis, dass die Architektur intrinsische stochastische Modellvariation reduziert. Die Kennzahl umfasst auch User-Simulator, mehrstufige Trajektorien und mögliche Providerlaufvariation.

Die komplementären Maße pass^6 und $\text{pass}@6$ trennen wiederholbare Zuverlässigkeit von mindestens einem Erfolg. C3 erreicht bei beiden Endpunkten die höchste Stabilität, während die Aufgabenabdeckung aller Bedingungen hoch und ähnlich bleibt.

Der Verlauf über mehrere Werte von k zeigt, wann sich die Rangfolge verändert. Bei Exact Mutation entspricht pass^1 zunächst den aggregierten Anteilen. C1 erreicht 74,1 %, C3 73,4 %, C2 70,6 % und C0 67,8 %. Bei pass^3 liegt C3 mit 50,8 % bereits knapp vor C1 mit 49,7 %. C2 erreicht 46,0 % und C0 42,1 %. Bei der strengsten Forderung pass^6 vergrößert sich der Abstand von C3 zu C1 auf 6,9 Prozentpunkte. Für DB Match zeigt sich dieselbe Verschiebung. C1 führt bei pass^1 , C3 liegt bei pass^3 und pass^6 vorn. Das Muster beruht somit nicht auf einem höheren Einzelaufmittelwert von C3, sondern auf seinem höheren Anteil über mehrere Wiederholungen erfolgreicher Aufgaben. Tabelle 18 stellt dafür die strengste Wiederholungsanforderung und die mindestens einmalige Lösung gegenüber.

Bedingung	DB pass⁶	Exact pass⁶	DB pass@6	Exact pass@6
C0 Native	27/80 = 33,8 %	20/72 = 27,8 %	79/80 = 98,8 %	69/72 = 95,8 %
C1 Semantische Unterstützung	28/80 = 35,0 %	21/72 = 29,2 %	78/80 = 97,5 %	68/72 = 94,4 %
C2 Pre-Write-Kontrolle	30/80 = 37,5 %	23/72 = 31,9 %	79/80 = 98,8 %	70/72 = 97,2 %
C3 Kombination	32/80 = 40,0 %	26/72 = 36,1 %	79/80 = 98,8 %	69/72 = 95,8 %

Tabelle 18. Wiederholbare und mindestens einmalige Aufgabenerfolge

pass@6 belegt weder universelle Lösbarkeit noch zuverlässigen Einzelauf Erfolg. Der große Abstand zu pass⁶ zeigt vielmehr, dass die Interventionen vor allem Häufigkeit und Verteilung erfolgreicher Wiederholungen verändern. Eine substanzielle Erweiterung der beobachteten Aufgabenabdeckung ist nicht erkennbar.

Saved und Broken

Die gepaarten Übergänge zeigen, dass positive Nettodifferenzen aus deutlich größeren gegenläufigen Bewegungen entstehen. Keine Intervention ist damit monoton vorteilhaft. Tabelle 19 weist diese Bewegungen für beide Endpunkte aus.

Bedingung	DB Saved	DB Broken	DB Netto	Exact Saved	Exact Broken	Exact Netto
C1 Semantische Unterstützung	78	49	+29	73	46	+27
C2 Pre-Write-Kontrolle	69	61	+8	66	54	+12
C3 Kombination	83	58	+25	78	54	+24

Tabelle 19. Gepaarte Saved-/Broken-Übergänge gegenüber C0

Abbildung 7 stellt verlorene native Erfolge und gerettete native Fehlschläge divergierend gegenüber.

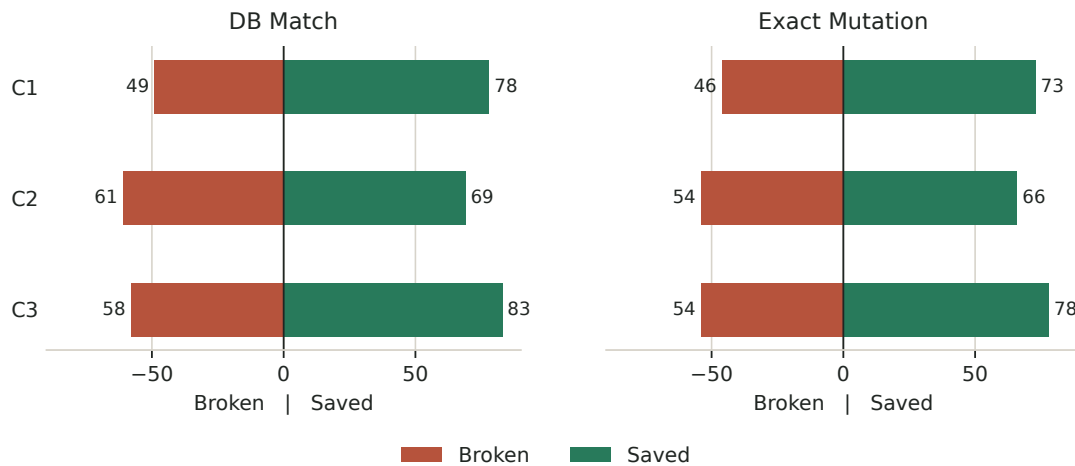


Abbildung 7. Saved- und Broken-Zellen gegenüber C0 für beide Endpunkte in der finalen Sechs-Seed-Auswertung

Auf Zellebene wechseln bei DB Match unter C1 insgesamt 127 von 480 Vergleichspaaren ihren Status, unter C2 130 und unter C3 141. Dies entspricht 26,5 %, 27,1 % und 29,4 % aller gepaarten Zellen. Für Exact Mutation beträgt die Bruttobewegung 119, 120 und 132 von 432 Zellen, also 27,5 %, 27,8 % und 30,6 %. C3 erzeugt bei beiden Endpunkten die meisten Statuswechsel. Der Abstand zum nativen Agenten besteht folglich nicht aus wenigen isolierten Korrekturen, sondern aus einer breiten Neuverteilung positiver und negativer Verläufe.

Werden die sechs Wiederholungen je Aufgabe zusammengefasst, bleibt diese Heterogenität sichtbar. C1 und C3 verbessern bei beiden Endpunkten mehr Aufgaben, als sie verschlechtern. C2 weist die schwächste Trennung auf.

Die Zellbilanz zeigt Wechsel einzelner Wiederholungen, die Aufgabenbilanz deren Verdichtung über sechs Seeds. C1 und C3 verbessern in beiden Perspektiven mehr Fälle, als sie verschlechtern. Die verbleibenden Regressionen schließen eine Nichtschadensgarantie dennoch aus.

8.3 Wirkmechanismen und Kontrollwirkung

Komponente A: semantische Unterstützung

In C1 wurden 1.190 Unterstützungskarten materialisiert. 529 davon erreichten einen späteren Schreibkandidaten. In den dadurch berührten Zellen stehen beim DB Match 62 Saved 27 Broken gegenüber. In C3 erreichten 550 von 1.200 Kartenständen einen Schreibkandidaten. Dort beträgt das Verhältnis 69 zu 32. Der positive Saldo ist mit dem beabsichtigten Wirkpfad von Komponente A vereinbar und zeigt, dass die Unterstützung im Handlungspfad tatsächlich sichtbar war. Aufgrund möglicher Unterschiede der gesamten Agententrajektorie ist dies jedoch kein isolierter Mediatoreffekt.

Die vorgelagerten Mengengerüste sind in den Bedingungen C1 und C3 nahezu gleich. In C1 entstehen aus 1.392 Extraktionsgelegenheiten 1.190 materialisierte Kartenstände, entsprechend 85,5 %. In C3 werden aus 1.400 Gelegenheiten 1.200 Kartenstände materialisiert, entsprechend 85,7 %. Von diesen Kartenständen erreichen in C1 44,5 % und in C3 45,8 % einen späteren Schreibkandidaten. Die ähnliche Exposition spricht dagegen, den Unterschied zwischen C1 und C3 durch eine grundsätzlich andere Aktivierungsmenge von Komponente A zu erklären. Sie zeigt zugleich, dass mehr als die Hälfte der materialisierten Karten nicht bis zu einer schreibnahen Entscheidung fortwirkt.

Innerhalb der schreibverknüpften C1-Zellen übersteigen die 62 Saved-Zellen die 27 Broken-Zellen um 35. In C3 beträgt der entsprechende Saldo 37. Diese Teilgruppenbilanz ist als Mechanismuskonsistenz zu lesen. Wo Komponente A nachweislich bis an einen Schreibkandidaten reicht, treten mehr positive als negative Wechsel auf. Weil die Verknüpfung selbst vom entstehenden Dialog- und Agentenpfad abhängt, ist die Gruppe jedoch nicht exogen bestimmt. Ein Vergleich exponierter und nicht exponierter Zellen würde daher keine randomisierte Wirkung von Komponente A isolieren.

Kontrastierende Ereignisspuren für Komponente A

Der ausgewählte Saved-Fall (Task 5, Seed 976302, C1) zeigt den intendierten Pfad bei einer späten Zielrevision. Der Nutzer wollte zunächst eine Wasserflasche und eine Lampe umtauschen, widerrief den Umtausch anschließend aber ausdrücklich und verlangte stattdessen die Rückgabe der Wasserflasche. Die materialisierte Karte hielt die Revisionsspur bis zum Schreibkandidaten vor. Verknüpft war schließlich `return_delivered_order_items` für genau die Wasserflasche. Der Exact-Status wechselte gegenüber C0 von Fehl- zu Erfolg. Die Ereignisspur ist damit mechanismuskonsistent. Eine späte Nutzerrevision blieb an der Schreibgrenze sichtbar. Weil Komponente A zugleich den Dialogpfad verändern kann, wird daraus kein isolierter Mediatoreffekt abgeleitet.

Der Broken-Fall (Task 3, Seed 976304, C1) markiert die Grenze derselben Logik. Der Dialog enthielt fortgeschriebene Anforderungen für zwei Bestellungen, darunter eine frühe Präferenz für lila sowie eine spätere bestellungsspezifische Festlegung auf schwarz, XXL, Polyester und V-Ausschnitt. Zwei Karten erreichten spätere Schreibkandidaten. Der verdichtete Zustand führte mehrere zeitlich verteilte Nutzerziele parallel, ohne ihren aktuellen Geltungsbereich eindeutig aufzulösen. C1 führte daraufhin für Bestellung #W6247578 eine zusätzliche Mutation aus; der Offline-Evaluator klassifizierte sie als *Extra Effect*, und ein nativer Exact-Erfolg wurde gebrochen. Quellenbindung allein verhindert somit nicht, dass ein überbreiter oder zeitlich unaufgelöster Zielzustand bis zur Schreibaktion fortwirkt. Die Ereignisspur zeigt eine plausible Revisionsgrenze, beweist aber nicht, dass die Karte allein die Regression verursachte.

Komponente B: Pre-Write-Kontrolle

Für C2 ist zwischen Kontrollgestaltung, Kontrollausführung und empirischer Kontrollwirkung zu unterscheiden. Die deterministische Prüfung ist technisch als präventiver Kontrollpunkt vor der Schreibgrenze implementiert. Über C2 und C3 wurden 1.377 Schreibkandidaten geprüft. Der nachgelagerte, nur für die Evaluation verfügbare Mutation-Oracle-Abgleich kennzeichnet 199 Kandidaten als nicht mit der erwarteten Mutation vereinbar. Davon wurden 142 unverändert freigegeben. 57 führten zu einer materiellen Entscheidung (`EVIDENCE_REQUIRED` oder `INVALID`). Zugleich löste Komponente B bei 70 oracle-kompatiblen Kandidaten eine materielle Entscheidung aus.

Die Kandidatenbilanz macht die begrenzte Trennschärfe sichtbar. Von den 199 post hoc als nicht kompatibel erkannten Kandidaten führten 57 zu einer materiellen Entscheidung von Komponente B. Dies entspricht 28,6 %. Die übrigen 142 oder 71,4 % wurden unverändert freigegeben. Unter den 1.178 kompatiblen Kandidaten lösten 70 oder 5,9 % ebenfalls eine materielle Entscheidung aus. Von insgesamt 127 materiellen Entscheidungen entfielen damit 57 auf inkompatible und 70 auf kompatible Kandidaten. Diese Anteile werden nur als nachgelagerte Diagnose berichtet. Das Mutation Oracle stand Komponente B zur Laufzeit nicht zur Verfügung und bildet nicht jede zulässige Policyentscheidung ab.

Auch auf Zellebene ist kein positiver Endpunktsaldo erkennbar. In Zellen mit materieller Entscheidung von Komponente B stehen beim DB Match in C2 sechs Saved- und zehn Broken-Fälle sowie in C3 fünf Saved- und 14 Broken-Fälle gegenüber. Der Kontrollpfad ist damit nachweislich aktiv, trennt unter der verfügbaren Informationsgrenze fachlich kompatible und inkompatible Kandidaten aber nur begrenzt und erzeugt keinen belastbaren eigenständigen Nettonutzen. Der Befund widerlegt nicht die Kontrollidee als solche. Er begrenzt die Wirksamkeit der konkret implementierten Kontrollregel.

Eine materielle Entscheidung von Komponente B ist zudem nicht mit einer erfolgreichen Korrektur gleichzusetzen. Zusammen stehen in den betroffenen Zellen elf positiven 24 negativen DB-Wechseln gegenüber. Eine Evidenzanforderung oder Neuplanung stellt zunächst nur eine Abweichung vom nativen Pfad her. Ob daraus ein korrekterer Endzustand entsteht, hängt von der Qualität der angeforderten Evidenz, der neuen Kandidatenbildung und dem anschließenden Aufgabenabschluss ab. Die Ergebnisdaten belegen damit technische Kontrollausführung, aber keinen positiven Korrekturertrag der materiellen Eingriffe von Komponente B.

Die Daten belegen damit einen protokollierten Kontrollpunkt, aber weder hinreichende fachliche Selektivität noch Schadensvermeidung.

Kontrastierende Ereignisspuren für Komponente B

Ein besonders aufschlussreicher Kontrast liegt für dieselbe Aufgabe in zwei Wiederholungen vor. Im Saved-Fall (Task 0, Seed 976301, C2) hielt Komponente B einen Umtauschkandidaten zunächst mit `B_PAYMENT_PROFILE_NOT_OBSERVED` zurück, ließ das authentifizierte Nutzerprofil lesen und gab danach denselben Kandidaten unverändert frei. Der Kandidat enthielt die korrekte Zielvariante 7706410293. Exact Mutation wechselte von nativem Fehl- zu C2-Erfolg. Komponente B stellte hier nachweisbar Evidenzvollständigkeit für die Zahlungsdurchführung her. Da der Kandidat nicht verändert wurde, wäre es jedoch zu stark, Komponente B eine semantische Korrektur der Zielvariante zuzuschreiben.

Im Broken-Fall (Task 0, Seed 976303, C2) war der Kontrollablauf strukturell gleich. Komponente B hielt den Kandidaten zurück, ließ das Zahlungsprofil lesen und gab ihn anschließend unverändert frei. Der Kandidat enthielt jedoch die Zielvariante 6342039236 statt 7706410293. Komponente B bestätigte damit die Ausführbarkeit des Zahlungsmittels, nicht die fachliche Richtigkeit des Austauschziels. Der Offline-Evaluator diagnostizierte einen *Wrong Parameter*; ein nativer Exact-Erfolg ging verloren. Das Fallpaar präzisiert die Aussage zur technischen Kontrollierbarkeit. Komponente B erzeugt einen prüfbaren Machbarkeitsnachweis an der Transaktionsgrenze, besitzt unter der implementierten Informations- und Autoritätsgrenze aber keine semantische Zielvalidierung.

Kontrastierende Ereignisspuren für C3

Auch die Kombination zeigt keine Erfolgsgarantie. Im Saved-Fall (Task 21, Seed 976304) erreichte eine Unterstützungskarte einen Schreibkandidaten für die Änderung zweier Bestellpositionen und hielt die gewünschte Geschenkkarte sichtbar. Komponente B forderte wegen nicht beobachteter Produktdaten gezielt Evidenz an und gab den unveränderten Kandidaten nach dem Read frei. Exact Mutation wechselte von Fehl- zu Erfolg. Der Pfad ist mit komplementärer Unterstützung und Machbarkeitskontrolle vereinbar, isoliert aber keinen Einzelbeitrag.

Im Broken-Fall (Task 1, Seed 976301) waren ebenfalls beide Mechanismen im Schreibpfad. Komponente A materialisierte die mehrteiligen Umtauschziele, Komponente B ergänzte das fehlende Zahlungsprofil und gab den Kandidaten anschließend unverändert frei. Die ausgeführte Mutation blieb dennoch nicht mit dem Mutation Oracle vereinbar. Der Evaluator diagnostizierte einen zusätzlichen Effekt, und ein nativer Exact-Erfolg wurde gebrochen. Die beiden C3-Vignetten erklären damit, warum Komponentenaktivität nicht mit additivem Nutzen gleichgesetzt werden darf. Sie sind mit der negativen Punktschätzung der Interaktion vereinbar, können einen Interaktionseffekt als Einzelfälle aber nicht belegen.

8.4 Ressourcenprofil

Gemessener Ressourcenaufwand

Unter C1 liegen die mittleren Agenten-API-Kosten 45,3 % und die technische Laufzeit 26,3 % über C0. Für C3 betragen die Abstände 46,1 % und 27,7 %. C2 bleibt mit 1,1 % höheren Agentenkosten und 0,04 % längerer Laufzeit praktisch auf dem Niveau von C0. Personalkosten, Kundenwartezeit, Nachbearbeitung und reale Fehlerfolgen wurden dagegen nicht gemessen.

In absoluten Mittelwerten benötigt C1 je Zelle 0,0031 US-Dollar (USD) mehr Agenten-API-Aufwand und 11,9 zusätzliche Sekunden gegenüber C0. Für C3 betragen die Differenzen 0,0032 USD und 12,5 Sekunden. Über jeweils 480 Zellen summieren sich die Agenten-API-Kosten auf 3,29 USD für C0, 4,79 USD für C1, 3,33 USD für C2 und 4,81 USD für C3. Die Kosten des simulierten Nutzers sind hiervon ausgeschlossen. Damit entspricht die Auswertung dem in Kapitel 4 festgelegten betrieblichen Agentenaufwand und nicht den gesamten Kosten der experimentellen Datenerhebung.

Die nach Aufgaben geclusterten 95-%-Intervalle der gepaarten Kostendifferenz liegen für C1 bei $[+0,0027; +0,0035]$ USD und für C3 bei $[+0,0027; +0,0036]$ USD je Zelle. Bei der Laufzeit betragen sie $[+9,6; +14,3]$ beziehungsweise $[+10,3; +14,7]$ Sekunden. Für C2 umfassen sowohl das Kostenintervall $[-0,0001; +0,0003]$ USD als auch das Laufzeitintervall $[-1,8; +1,8]$ Sekunden null. Der zusätzliche Kosten- und Zeitaufwand ist damit für C1 und C3 über die Aufgabenverteilung sichtbar, für C2 dagegen nicht. Tabelle 20 bündelt die mittleren Ressourcenprofile der vier Bedingungen.

Bedingung	Agenten-API/Zelle	Laufzeit/Zelle	Nachrichten/Zelle	Tool-Aufrufe/Zelle
C0 Native	0,0069 USD	45,2 s	27,94	7,43
C1 Semantische Unterstützung	0,0100 USD	57,1 s	27,84	7,53
C2 Pre-Write-Kontrolle	0,0069 USD	45,2 s	27,90	7,49
C3 Kombination	0,0100 USD	57,7 s	27,64	7,52

Tabelle 20. Gemessener Ressourcen- und Interaktionsaufwand der vier Konfigurationen

Die gepaarten Differenzen bei Nachrichten und Agenten-Tool-Aufrufen sind klein. Gegenüber C0 verändert sich die mittlere Nachrichtenzahl unter C1 um $-0,10$, unter C2 um $-0,04$ und unter C3 um $-0,30$. Die Tool-Aufrufzahl steigt um 0,09, 0,05 beziehungsweise 0,09 Aufrufe je Zelle. Alle zugehörigen nach Aufgaben geclusterten 95-%-Intervalle umfassen null. Der zusätzliche Aufwand der beiden Bedingungen mit aktivierter Komponente A zeigt sich damit

in den vorliegenden Daten vor allem bei Agentenkosten und Laufzeit, nicht in längeren Dialogen oder einer substanziell höheren Zahl von Tool-Aufrufen.

Die Laufzeitwerte variieren außerdem zwischen den Seeds. Bei C0 reichen die seedweisen Mittelwerte von 37,6 bis 54,6 Sekunden, bei C1 von 48,1 bis 67,3 Sekunden und bei C3 von 49,1 bis 69,5 Sekunden. Ein Teil der absoluten Laufzeitvariation lässt sich aus diesen Daten nicht eindeutig der Komponentenlogik oder der Ausführungsumgebung zuordnen. Der relative Abstand von C1 und C3 zu den jeweiligen nativen Verläufen bleibt als deskriptives Ressourcenmerkmal sichtbar. Technische Laufzeit wird deshalb nicht als direkte Kundenwartezeit oder Personalkapazität interpretiert.

8.5 Zusammenfassung der empirischen Antworten

Die Sechs-Seed-Evaluation zeigt drei unterschiedliche Leistungsprofile. C1 erreicht die höchsten mittleren Endpunkte. Alle Kontraste von Faktor A der sechs Seed-Auslassungsanalysen sind positiv, während der konservative Zweiweg-Bootstrap null einschließt. C3 besitzt die höchste strenge Stabilität über alle sechs Seeds, liefert aber keinen belastbaren additiven Kombinationsvorteil. C2 bleibt bei den Endpunkten nahe an C0 und zeigt trotz technisch aktiver Kontrolle nur geringe Selektivität. Zugleich erzeugen alle Interventionen Saved- und Broken-Fälle. C1 und C3 verbinden ihre stärkeren Ergebnisprofile mit rund 46 % höheren Agenten-API-Kosten und rund 27 % längerer technischer Laufzeit, während Nachrichten- und Tool-Aufrufzahlen nahe an C0 bleiben. Damit ist keine Konfiguration auf allen Ergebnisdimensionen überlegen. Kapitel 9 ordnet die empirischen Unterschiede, Unsicherheiten und Zielkonflikte ein.

9 Diskussion

Die Evaluation ergibt kein dominantes Profil. C1 erzielt die höchsten mittleren Endpunkte, C3 die höchste strenge Stabilität. Der überwiegend positive A-Kontrast bleibt unter konservativer Zweiweg-Unsicherheit nicht eindeutig von null getrennt; Komponente B zeigt keinen belastbaren eigenständigen Erfolgsbeitrag. Die Einordnung verbindet diese Befunde mit Forschungsfragen, Designwissen, betrieblichen Implikationen und Grenzen.

9.1 Beantwortung der Forschungsfragen

Teilforschungsfragen 1 und 2: Anforderungen und Artefaktlogik

TF1 untersucht, welche Anforderungen an semantische Unterstützung, Pre-Write-Kontrolle und deren Messung aus Literatur und transaktionalen Prozessanforderungen abzuleiten sind. Die neun Gestaltungsanforderungen aus Kapitel 5 verdichten sich zu vier Leitlinien. Laufzeitkomponenten dürfen nur zulässige Information verwenden, ihre Autorität muss eng begrenzt und ihr Einsatz getrennt schalt- und beobachtbar sein. Die Evaluation muss Erfolg, Regression, Stabilität und Ressourcen gemeinsam erfassen. A und B erhalten weder Referenzaktionen noch erwarteten Datenbankzustand oder Gesamtscore.

TF2 untersucht, wie beide Mechanismen als unabhängig schaltbare Laufzeitschicht umgesetzt werden können, ohne zusätzliche Aufgabeninformationen oder versteckte Referenzinformationen des Benchmarks einzuführen. Komponente A macht belegte Nutzerziele vor der nächsten Agentenentscheidung sichtbar, besitzt aber keine Werkzeug- oder Schreibautorität. Komponente B prüft einen bereits geplanten Schreibkandidaten deterministisch unmittelbar vor seiner Ausführung. Sie darf einen Kandidaten nur unverändert freigeben, Evidenz verlangen oder eine begrenzte Neuplanung auslösen. Tests und Laufzeit Spuren belegen die technische Realisierung dieser Autoritäts- und Informationsgrenzen. Dieser Designnachweis ist notwendig, aber ausdrücklich von empirischer Wirksamkeit zu unterscheiden.

Teilforschungsfrage 3: Ergebniswirkung und Stabilität

TF3 untersucht, welche isolierten und kombinierten Wirkungen beide Komponenten auf Aufgabenerfolg, Datenbankabgleich, exakte Mutationsausführung und Wiederholungsstabilität zeigen. Unter C1 steigt der beobachtete DB Match von $342/480 = 71,3\%$ auf $371/480 = 77,3\%$ und Exact Mutation von $293/432 = 67,8\%$ auf $320/432 = 74,1\%$. C3 erreicht mit 76,5 % beziehungsweise 73,4 % nur geringfügig niedrigere Mittelwerte; C2 bleibt näher an C0.

Die Mittelwerte allein reichen jedoch nicht aus. C1 liegt in fünf Seeds über C0, im sechsten darunter. C3 liegt bei Exact Mutation dagegen in allen sechs Seeds über C0 und erreicht

mit $26/72 = 36,1\%$ die höchste pass^6 -Stabilität. Damit entstehen zwei unterschiedliche Leistungsprofile. C1 erreicht den höchsten beobachteten Durchschnitt, C3 die höchste strenge Wiederholbarkeit. Diese Abgrenzung verhindert, C1 allein aufgrund eines Aggregats pauschal als beste Konfiguration zu bezeichnen.

Die faktorielle Auswertung stützt diese vorsichtige Lesart. Faktor A erreicht +4,8 Prozentpunkte bei DB Match und +4,5 Prozentpunkte bei Exact Mutation. Alle Schätzungen beim Auslassen jeweils eines Seeds bleiben positiv. Die Seed-t-Intervalle liegen oberhalb null, der konservativere Zweiweg-Bootstrap über Aufgaben und Seeds schließt null jedoch ein. Der Kontrast von Faktor A ist somit ein konsistenter positiver Hinweis innerhalb der untersuchten Matrix, kein universell gesicherter Effekt. Faktor B besitzt mit +0,4 beziehungsweise +1,0 Prozentpunkten keinen belastbaren Haupteffekt. Die negative Interaktionsschätzung spricht gegen Additivität, ist selbst aber ebenfalls unsicher.

Die Pass-Maße grenzen ein, worin dieser Unterschied besteht. Über die Bedingungen liegt $\text{pass}@6$ zwischen 94,4 und 98,8 %, pass^6 dagegen nur zwischen 27,8 und 40,0 %. C0 löst bereits 79 von 80 DB-Aufgaben und 69 von 72 Exact-Aufgaben mindestens einmal. Bei Exact Mutation besitzen C0 und C3 denselben $\text{pass}@6$ -Zähler, unter C3 sind jedoch 26 statt 20 Aufgaben in allen Wiederholungen erfolgreich. Gemäß der in Kapitel 2 hergeleiteten Metriklogik (vgl. Yao et al. (2025), S. 4) liegt der beobachtete Mehrwert daher kaum in neuen Aufgabenfähigkeiten, sondern in veränderten Einzelaufträgen und Wiederholungsprofilen. Eine mindestens einmal gelöste Aufgabe belegt weder zuverlässige Einzelausführung noch eine allgemeine Fähigkeit außerhalb der Domäne. Regressionen und Ressourcen bleiben deshalb mitzubewerten.

Die Variation zwischen den Seeds ist deshalb nicht nur technisches Rauschen, sondern Teil des untersuchten Zuverlässigkeitsproblems. Trotz Temperatur null, fixierter anbieterseitiger Seeds und festgelegter Modellversionen bleiben gehostete LLM-Ausführungen und die daraus entstehenden mehrstufigen Dialogtrajektorien nicht vollständig deterministisch (vgl. Atıl et al. (2025), S. 135–142). Gemischte Ergebnisse treten bei 58,8 bis 65,0 % der DB-Aufgaben und 59,7 bis 68,1 % der Exact-Aufgaben auf. Auch die vier Rohverteilungen überlappen. C1 verbindet den höchsten Mittelwert mit der größten DB-Streuung, während C3 die geringste aufgabeninterne Seedabweichung und die höchste Stabilität erreicht. Seine Differenzen zu C0 sind bei Exact Mutation in allen Seeds positiv, bei DB Match fünfmal positiv und einmal null. Überlappung belegt begrenzte Trennschärfe, aber nicht, dass jede Richtung nur zufällig entstand. Alle A-Schätzungen der sechs Seed-Auslassungen bleiben positiv; der negative C1-Kontrast in einem Seed und das null einschließende Zweiwegintervall verhindern jedoch eine von Laufvariation unabhängige Effektaussage. Die Variation lässt sich nicht in Beiträge von Agent, User-Simulator und Anbieterinfrastruktur zerlegen. Die Architektur verändert das Leistungs- und Zuverlässigkeitsprofil, eliminiert stochastische Ausführung aber nicht.

Der Befund zu Komponente A ist mit der in Kapitel 2 entwickelten Rolle expliziter Zustandsrepräsentationen vereinbar. Mehrturnige Servicefälle verlangen, dass Objekte, Teilziele und spätere Präferenzen über mehrere Dialogschritte hinweg korrekt gebunden bleiben. Komponente A externalisiert einen Teil dieser Information in einer quellengebundenen, entscheidungsnahen Darstellung. Dadurch muss der native Agent relevante Nutzerziele nicht ausschließlich aus seinem stochastischen Gesprächskontext rekonstruieren. Der positive Kontrast von Faktor A und die positive Bilanz schreibverknüpfter Karten entsprechen diesem erwarteten Wirkpfad. Gemeinsam mit den kontrastierenden Ereignisspuren plausibilisieren sie einen möglichen Nutzen bei zusammengesetzten oder über mehrere Turns verteilten Anliegen. Einen eigenständigen Vorteil für diese Aufgabengruppe weist die Studie jedoch nicht aus (vgl. Wu et al. (2024), Abschn. 3–4; vgl. Mishra et al. (2025), Abschn. 3–5).

Die Regressionen unter C1 markieren zugleich die Grenze dieses Prinzips. Quellenbindung gewährleistet, dass ein Eintrag auf ein sichtbares Nutzerereignis zurückgeht. Sie gewährleistet weder dessen fortbestehende Relevanz noch die richtige Gewichtung gegenüber späteren Informationen. Eine Unterstützungskarte verändert den Kontext, in dem das native Modell plant. Sie kann dadurch einen korrekten nativen Pfad stabilisieren, aber ebenso die Aufmerksamkeit auf einen unvollständigen Teil des Anliegens lenken oder eine andere Agententrajektorie auslösen. Dass C1 im sechsten Seed unter C0 liegt und unter C1 46 native Exact-Erfolge zu Misserfolg wechseln, ist daher keine Randnotiz, sondern begrenzt den Geltungsanspruch. Externalisierung verbessert die Verfügbarkeit von Information, nicht automatisch ihre situationsgerechte Nutzung.

Die Kombination C3 verdeutlicht eine zweite Grenze. Die Komponenten A und B sind technisch modular, wirken aber nicht auf unabhängige Prozesse. Komponente A verändert den Kontext vor der Agentenentscheidung und damit potenziell, welche Schreibkandidaten Komponente B später überhaupt sieht. Komponente B kann daraufhin Evidenz oder Neuplanung auslösen und den durch Komponente A beeinflussten Pfad erneut verändern. Die negative Interaktionsschätzung ist deshalb mit einer Trajektorieninteraktion vereinbar, beweist aber keinen bestimmten Störmechanismus. C3 erreicht zugleich die höchste Stabilität über alle sechs Seeds, ohne den höchsten Mittelwert zu erzielen. Die einfachen Effekte stützen diese vorsichtige Deutung: Der A-Kontrast sinkt bei aktiviertem B von +6,0 auf +3,5 Prozentpunkte für DB Match und von +6,3 auf +2,8 Prozentpunkte für Exact Mutation. Der B-Kontrast wechselt bei aktiviertem A das Vorzeichen. Da die vorgelagerten A-Aktivierungsmengen in C1 und C3 nahezu gleich sind, erklärt eine bloß geringere Aktivierung den Abstand nicht. Zusammen mit begrenzter B-Selektivität und zusätzlichen stochastischen Wiederherstellungsabzweigungen ergibt sich ein mit Trajektorienkopplung vereinbares Mechanismenbild, kein statistischer Nachweis eines negativen Interaktionseffekts.

Teilforschungsfrage 4: Mechanismen und Kontrollierbarkeit

TF4 untersucht, in welchen Fällen die Komponenten zu geretteten, beschädigten oder unveränderten Ergebnisverläufen führen und welche beobachtbaren Komponentenpfade und Fehlerarten mit diesen Wechseln einhergehen. Die Saved-/Broken-Analyse zeigt für alle Varianten substanzielle Bewegung in beide Richtungen. Bei Exact Mutation wechseln unter C1 73 native Fehlschläge zu Erfolg und 46 native Erfolge zu Misserfolg. C3 erreicht 78 Saved und 54 Broken. Ein positiver Mittelwert ist damit weder Nichtschadensnachweis noch Garantie für einzelne Fälle.

Für Komponente A ist die Mechanismuskette plausibel. In C1 erreichen 529 von 1.190 materialisierten Kartenständen einen späteren Schreibkandidaten. In den betroffenen Zellen ist die Saved-/Broken-Bilanz positiv. Diese Verbindung stützt den vorgesehenen Wirkpfad, isoliert aber wegen möglicher Trajektorienunterschiede keinen kausalen Mediatoreffekt.

Die kontrastierenden Ereignisspuren zu Komponente A präzisieren, was hinter dieser Bilanz liegt. Im Saved-Fall blieb ein ausdrücklicher Widerruf bis zum korrekten Rückgabekandidaten sichtbar. Im Broken-Fall erreichten dagegen mehrere zeitlich verteilte Nutzerziele parallel die Schreibgrenze, ohne dass ihr aktueller Geltungsbereich eindeutig aufgelöst war. Die Ereignisspur motiviert daher als Weiterentwicklung eine explizite Temporalitätslogik für Widerruf, Ersetzung und auftragsbezogene Priorisierung zusätzlich zur Quellenbindung. Der Mechanismusbefund lautet nicht, dass mehr gespeicherte Ziele automatisch helfen, sondern dass entscheidungsnah korrekt aufgelöste Ziele helfen können.

Bei Komponente B muss genauer getrennt werden. Erstens ist das Kontrolldesign technisch umgesetzt. Ein deterministischer Prüfpunkt liegt vor der mutierenden Systemgrenze. Zweitens wird die Kontrolle ausgeführt. Über C2 und C3 wurden 1.377 Kandidaten geprüft. Drittens ist ihre fachliche Wirkung begrenzt. Von 199 nachgelagert als oracle-inkompatibel erkannten Kandidaten wurden 142 unverändert freigegeben. Zugleich löste Komponente B bei 70 kompatiblen Kandidaten eine materielle Entscheidung aus. Auch der Saved-/Broken-Saldo materieller Entscheidungen von Komponente B ist negativ.

Die schwache Selektivität von Komponente B lässt sich durch ihre Informations- und Autoritätsgrenze plausibilisieren, aber nicht kausal auf sie zurückführen. Komponente B sieht den konkreten Schreibkandidaten, den sichtbaren Prozesszustand und deterministisch prüfbare Evidenz. Sie besitzt bewusst weder ein eigenes LLM noch Zugriff auf Goldmutation oder semantische Nutzerziel-Datensätze. Damit kann sie Schema-, Objekt-, Argument- und Evidenzbindungen prüfen, aber nicht jede fachlich falsche Zielwahl erkennen. Ein Kandidat kann formal zulässig und im aktuellen Zustand ausführbar sein, obwohl er nicht die vom Nutzer tatsächlich beabsichtigte Transaktion realisiert. Die 142 unverändert freigegebenen oracle-inkompatiblen Kandidaten sind mit genau dieser Begrenzung vereinbar.

Das kontrastierende Fallpaar zu Komponente B macht diese Grenze auf derselben Aufgabe sichtbar. In beiden Wiederholungen hielt Komponente B den Kandidaten bis zum Nachweis des Zahlungsmittels zurück und gab ihn anschließend unverändert frei. Einmal war die Zielvariante korrekt und die Zelle wurde gerettet. Einmal war sie falsch und ein nativer Erfolg ging verloren. In beiden Ereignisspuren belegt der Kontrolltyp die Ausführbarkeit mit dem angegebenen Zahlungsmittel, aber nicht die semantische Zielrichtigkeit. Eine Erweiterung um Zielprüfung wäre eine neue Kontrollfunktion mit zusätzlicher Information und Autorität, nicht lediglich eine kleine Schärfung derselben Regel.

Umgekehrt ist eine materielle Kontrollentscheidung noch keine gelungene Korrektur. Nach `EVIDENCE_REQUIRED` oder `INVALID` muss der Agent die richtige Evidenz beschaffen, einen besseren Kandidaten bilden und den Fall vollständig abschließen. Jeder zusätzliche Schritt eröffnet einen neuen stochastischen Abzweig. Der negative Saved-/Broken-Saldo in Zellen mit materiellem Eingriff von Komponente B deutet darauf hin, dass Erkennung und Wiederherstellung in der vorliegenden Ausgestaltung keine verlässlich positive Kette bilden. Für präventive Agentenkontrolle folgt daraus, dass Detektion, Korrekturpfad und sicherer Abschluss als gemeinsames Kontrollsystem gestaltet werden müssen.

Hier liegt der belastbare Beitrag von C2 zur technischen Kontrollierbarkeit. Die Komponente schafft eine definierte Kontrollstelle, begrenzte Entscheidungsrechte und protokollierbare Kontrollausführung. Das verbessert Nachvollziehbarkeit und macht Kontrollqualität prüfbar. Es wäre jedoch falsch, aus der Existenz des Kontrollpunkts bereits wirksame Risikoreduktion oder Compliance abzuleiten. Die Arbeit untersucht technische Laufzeitkontrolle, nicht ein vollständiges KI-Governance-System mit Rollen, Freigaben, Überwachung und organisatorischer Verantwortung.

Eine Governance-orientierte Bewertung muss daher mindestens fünf Fragen trennen. Welches Risiko soll kontrolliert werden? Welche Information steht der Kontrolle rechtzeitig zur Verfügung? Welche Entscheidung darf sie treffen? Welcher Korrektur- oder Eskalationspfad folgt? Und welche Wirkung zeigt sich im Endergebnis? C2 beantwortet die Fragen nach Kontrollstelle, Information und begrenztem Entscheidungsrecht technisch klar. Die Ergebnisanalyse zeigt jedoch Defizite bei Selektivität und Korrekturertrag. Gerade diese negative Evidenz ist für Governance relevant, weil sie verhindert, dass Protokollierbarkeit oder formale Kontrollpräsenz als Ersatz für Kontrollwirksamkeit behandelt werden.

Die C3-Ereignisspuren verbinden beide Argumente. Sowohl im Saved- als auch im Broken-Fall lagen Unterstützungskarte und materielle Entscheidung von Komponente B im späteren Schreibpfad. Dass nur einer der beiden Fälle korrekt endete, veranschaulicht, dass Komponentenaktivität keinen korrekten gemeinsamen Wiederherstellungspfad garantiert. Auf

Aggregatebene ergänzt die negative, aber unsichere Interaktionspunktschätzung diese Grenze, ohne Nichtadditivität zu beweisen.

Teilforschungsfrage 5: Betriebliche Bewertung

TF5 untersucht, welche zusätzlichen API-Kosten, Laufzeiten und Interaktionsaufwände entstehen und unter welchen betrieblichen Annahmen eine Aktivierung der Komponenten vertretbar erscheint. Unter C1 liegen die Agenten-API-Kosten 45,3 % und die technische Laufzeit 26,3 % über C0. C3 liegt bei 46,1 % und 27,7 %. C2 bleibt in beiden Größen nahe an C0. Nachrichtenzahl und Agenten-Tool-Aufrufe unterscheiden sich nur geringfügig. Ihre nach Aufgaben geclusterten Intervalle umfassen null. Diese Größen bilden einen gemessenen Konfigurationstrade-off, jedoch keine Unternehmensrendite.

Das betriebswirtschaftliche Entscheidungsproblem besteht damit in der Konfiguration eines Serviceprozesses unter konkurrierenden Zielgrößen. Fachliche Bearbeitungsqualität, Prozessstabilität, Kontrollqualität und Ressourceneinsatz sind getrennt zu gewichten. Ein kontextfreier Gesamtscore wäre willkürlich (vgl. Van Looy; Shafagatova (2016), Abschn. 3–4; vgl. Grönroos; Ojasalo (2004), S. 414–416). Der Benchmark liefert dafür gemessene technische und fachliche Proxys, aber keine monetäre Bewertung unternehmerischer Folgen.

C0 bildet die ressourcenärmste Referenz, C1 den höchsten mittleren fachlichen Erfolg und C3 die höchste strenge Stabilität. Beide erhöhen API-Kosten und Laufzeit; C2 stellt eine protokollierbare Kontrollgrenze bereit, ist durch Leistung allein aber nicht zu rechtfertigen. Welche Konfiguration vorzuziehen ist, hängt von Fallvolumen, Service-Level, Fehlerfolgen und der Kritikalität von Schreibaktionen ab. Dafür sind im konkreten Serviceprozess zusätzliche Prozessdaten zu erheben.

Die zentrale Forschungsfrage ist damit konditional zu beantworten: Quellengebundene semantische Unterstützung weist unter gleicher Informations- und Bewertungsgrundlage einen positiven beobachteten Kontrast beim mittleren fachlichen Erfolg auf. Die Pre-Write-Kontrolle schafft einen überprüfbaren Kontrollpunkt, erzeugt aber in der implementierten Informationsgrenze keinen belastbaren eigenständigen Leistungsbeitrag. Die kombinierte Bedingung erreicht die höchste strenge Wiederholungsstabilität, liefert aber keinen belastbaren Nachweis eines additiven Kombinationsvorteils. Welche Konfiguration vorzuziehen ist, hängt vom betrieblichen Zielprofil ab.

Die Prüfannahmen werden entsprechend geschlossen. H1 wird wegen des positiven C1-Mittelwerts, aber konservativer Unsicherheit nur teilweise gestützt. H2 wird mangels Erfolgsbeitrags und positiven Saldos materieller Kontrollentscheidungen nicht gestützt. H3 wird durch das eigenständige C3-Profil gestützt, nicht jedoch als gesicherter subadditiver Effekt. H4 bewährt sich, weil Nettodifferenzen größere Saved-/Broken-Bewegungen verdecken.

H5 relativiert den in allen Seed-Auslassungsanalysen positiven Kontrast von Faktor A durch das null einschließende Zweiwegintervall. Keine Annahme begründet eine allgemeine Überlegenheit zusätzlicher Interventionen.

9.2 Einordnung in den Forschungsstand

Laufzeitkontrolle und Agentenevaluation

Die Ergebnisse passen zu Arbeiten, die externes Prozesswissen, semantische Unterstützung und kontrollierte Toolnutzung als Mittel gegen Agentenfehler untersuchen (vgl. Wu et al. (2024), Abschn. 3–4; vgl. Mishra et al. (2025), Abschn. 3–5; vgl. Wang; Poskitt; Sun (2026), Abschn. 4–7). Sie zeigen zugleich eine wichtige Grenze. Ein lokal plausibler Mechanismus muss im gesamten Kundenfall wirksam werden. Eine technische Prüfung vor einem Tool-Aufruf ist nur dann nützlich, wenn sie problematische Kandidaten selektiv erkennt und der Agent anschließend einen zulässigen Abschluss findet. Aktivierungs- oder Blockraten allein belegen daher keinen Nutzen.

Damit bestätigt die Arbeit Forderungen nach breiteren Evaluationsmaßen für agentische Systeme. Erfolg, Fehler, Stabilität, Kosten und Sicherheit sollten gemeinsam betrachtet werden (vgl. Yehudai et al. (2025), S. 1). Der spezifische Beitrag liegt darin, diese Forderung für transaktionale Serviceagenten mit ausführbarer Referenzgrundlage, persistentem Datenbankzustand und gepaarten Saved-/Broken-Übergängen umzusetzen.

Abgrenzung zu fastWorkflow und SABER

fastWorkflow ist die engste Gegenarbeit zur semantischen Unterstützung. Die Architektur verbindet eine kaskadierte Intent- und Parameterextraktion mit Validierung, hierarchischer Kontextrepräsentation und expliziter Planung und prüft dieses Komponentenbündel in Ablationen auf den Retail- und Airline-Aufgaben von τ -bench (vgl. Satija et al. (2026), Abschn. 3–5 und Anhang B). Damit widerlegt die Arbeit eine Neuheit strukturierter semantischer Unterstützung und deterministischer Validierung vor Toolaktionen. Ihr Erkenntnisziel ist jedoch die Leistungs- und Kostenoptimierung einer integrierten Architektur. Eine policy- und evidenzgebundene Kontrolle mutierender Schreibkandidaten wird jedoch weder als eigenständiger Faktor operationalisiert noch von der semantischen Unterstützung getrennt.

SABER ist die engste Gegenarbeit, weil auch dort Reflection und Verification im τ -Bench-Umfeld untersucht werden (vgl. Cuadron et al. (2025), Abschn. 5–8 sowie Anhang). Ein direkter Ranglistenvergleich wäre dennoch nicht sauber. Artefaktlogik, Modelle, Verifikationspfade und Benchmarkvarianten unterscheiden sich. Die vorliegende Arbeit rekonstruiert SABER nicht, sondern untersucht eine beratende, quellengebundene Komponente A und eine

LLM-freie Komponente B unter gemeinsamer Informations- und Bewertungsgrundlage in einem faktoriellen Design.

fastWorkflow und SABER begrenzen die Neuheit integrierter semantischer Unterstützung sowie verbundener Reflexions- und Verifikationsmechanismen. Die vorliegende Arbeit ergänzt ihre unabhängige Faktortrennung und bewertet Mittelwert, Wiederholungsstabilität, Saved/Broken-, Mechanismus- und Ressourcenprofile gemeinsam. Die adressierte Design- und Entscheidungslücke betrifft damit die zurechenbare Beurteilung von Einzelwirkung, Kombination, Regression, Stabilität und Aufwand (vgl. Sah et al. (2026), Abschn. 3–5; vgl. Cuadron et al. (2025), Abschn. 5–8 sowie Anhang; vgl. Chen et al. (2026), Abschn. 3–6). Zwei autoritätsbegrenzte Faktoren, sechs Wiederholungen und konditionales Designwissen adressieren sie.

9.3 Designwissen und betriebswirtschaftliche Implikationen

Über die Beantwortung der Forschungsfragen hinaus ergeben sich aus Gestaltung, Demonstration und Evaluation fünf Gestaltungsregeln für transaktionale LLM-Agenten. Ihr Geltungsbereich umfasst Systeme, in denen typisierte Tools auf einen sichtbaren Prozesszustand und persistente Geschäftsdaten wirken. Die Regeln beschreiben daher keine allgemein optimale Agentenarchitektur, sondern übertragbare Zusammenhänge zwischen Eingriffszeitpunkt, Autorität, Wirkungsnachweis und betrieblicher Aktivierung (vgl. Hevner et al. (2004), S. 82–87; vgl. Gregor; Hevner (2013), S. 341–344 und 351–352; vgl. Gregor; Jones (2007), S. 312–316).

(1) Erstens benötigen Laufzeitinterventionen einen *Nettonachweis* auf Ebene des vollständigen Falls. C1 erreicht zwar die höchsten mittleren Endpunkte, erzeugt gegenüber C0 aber zugleich 73 gerettete und 46 beschädigte Exact-Verläufe. C3 verbessert die strenge Wiederholungsstabilität, bleibt im Mittel leicht hinter C1 und verursacht ähnlich hohen Zusatzaufwand. Eine technisch plausible Komponente oder ein positiver Mittelwert reichen daher nicht als Aktivierungsbegründung. Erforderlich ist ein gemeinsames Profil aus fachlichem Erfolg, Regressionen, Stabilität und Ressourcen. Die Regel konkretisiert für transaktionale Agenten die Forderung, Leistungs- und Risikomaße nicht getrennt zu optimieren (vgl. Yehudai et al. (2025), S. 1).

(2) Zweitens bilden *Prävention und Korrekturpfad eine Designeinheit*. Komponente B schafft einen deterministischen Prüfpunkt vor der Zustandsänderung und wurde bei 1.377 Schreibkandidaten ausgeführt. Dennoch wurden 142 nachgelagert als oracle-inkompatibel erkannte Kandidaten unverändert freigegeben, während bei 70 kompatiblen Kandidaten eine materielle Entscheidung erfolgte. Selbst eine zutreffende Zurückhaltung löst den Fall erst, wenn die erforderliche Evidenz beschafft, ein geeigneter Kandidat neu geplant und die Transaktion sicher abgeschlossen wird. Pre-Write-Kontrolle ist deshalb als Kette aus

Erkennung, Evidenzgewinn, begrenzter Neuplanung und Abschlusskriterium zu gestalten. Diese Folgerung stimmt mit Befunden überein, nach denen Verifikation und Blockierung keinen erfolgreichen Wiederherstellungspfad garantieren (vgl. Sah et al. (2026), Abschn. 3–5; vgl. Cuadron et al. (2025), Abschn. 5–8 sowie Anhang).

(3) Drittens muss *semantische Unterstützung selektiv, quellengebunden und zeitlich aufgelöst* sein. Komponente A entlastet den nativen Agenten, indem belegte Nutzerziele entscheidungsnah als kontrollierter Anwendungszustand bereitgestellt werden. Der positive Kontrast von Faktor A und die Saved-Fälle stützen diesen Wirkpfad. Die gleichzeitig beobachteten Broken-Fälle zeigen jedoch, dass Quellenbindung nur die Herkunft eines Ziels absichert. Sie entscheidet nicht, ob das Ziel nach Widerruf oder Ersetzung noch gilt, welchem Auftrag es zuzuordnen ist oder welche von mehreren Äußerungen Vorrang besitzt. Eine weiterentwickelte Unterstützungskarte benötigt daher Regeln für Gültigkeit, Ersetzung und Priorisierung. Bloß mehr gespeicherter Kontext wäre keine hinreichende Verbesserung (vgl. Wu et al. (2024), Abschn. 3–4; vgl. Mishra et al. (2025), Abschn. 3–5).

(4) Viertens erfordert technische Modularität eine *gemeinsame faktorielle Evaluation*. Die Komponenten A und B besitzen getrennte Eingriffsrechte, wirken aber nacheinander auf dieselbe stochastische Trajektorie. Komponente A verändert den Entscheidungskontext und damit die Kandidaten, die Komponente B später prüft. Komponente B kann durch Evidenzanforderung oder Neuplanung den weiteren Verlauf erneut verändern. In den Daten sinkt der einfache Kontrast von Faktor A bei aktiviertem Faktor B, während der Kontrast von Faktor B bei aktiviertem Faktor A das Vorzeichen wechselt. Die negative Interaktionspunktschätzung bleibt statistisch unsicher und rechtfertigt daher keine additive Nutzenannahme. Komponenten sind deshalb einzeln und kombiniert unter identischen Eingriffsgelegenheiten zu messen. Erst dieses Design trennt isolierte Wirkung, Kombinationsprofil und Trajektorienkopplung.

(5) Fünftens ist die *Governance einer Kontrollstelle wirkungsbezogen* zu gestalten. Komponente B macht Kontrollentscheidungen und deren Folgewirkung protokollierbar. Daraus folgt jedoch noch keine wirksame Risikoreduktion. Für eine belastbare Bewertung sind vier Ebenen getrennt festzulegen: das zu begrenzende Risiko, die dafür verfügbare Information, das zulässige Entscheidungsrecht und die Wirkung auf den finalen Geschäftszustand. Die vorliegende Komponente beantwortet Kontrollort und Autoritätsgrenze klar, zeigt aber Defizite bei Selektivität und Korrekturertrag. Organisatorisch folgt daraus, dass Freigabe-, Abbruch-, Übergabe- und Wiederherstellungsregeln nicht aus der technischen Existenz des Prüfpunktes abgeleitet, sondern anhand beobachteter Kontrollqualität festgelegt werden müssen.

Überführung in betriebliche Aktivierungsfragen

Die Gestaltungsregeln beantworten, wie Laufzeitinterventionen zu entwerfen und zu evaluieren sind. Eine betriebliche Aktivierungsentscheidung benötigt zusätzlich die Ziele und

Risikogrenzen des konkreten Serviceprozesses. Die Überführung folgt deshalb einer dreistufigen Ableitungslogik. Zunächst wird festgehalten, welchen empirischen Befund die Evaluation unmittelbar trägt. Danach wird bestimmt, welche betriebliche Ziel- oder Risikofrage sich daraus für den Serviceprozess ergibt. Abschließend werden die Prozessdaten benannt, die für eine kontextbezogene Entscheidung zusätzlich erforderlich sind. Diese Trennung folgt der Prozessperspektive des IT-Geschäftswerts, nach der technische Ressourcen erst im Zusammenwirken mit Prozess und Organisation bewertet werden können (vgl. Ray; Muhanna; Barney (2005), S. 625–629). Sie berücksichtigt zugleich, dass Prozessleistung anhand kontextabhängiger Zeit-, Kosten- und Qualitätsziele zu beurteilen ist und dass Effizienz und wahrgenommene Servicequalität miteinander verbunden sein können (vgl. Dumas et al. (2018), Kap. 1; vgl. Van Looy; Shafagatova (2016), Abschn. 3–4; vgl. Grönroos; Ojasalo (2004), S. 414–416).

Tabelle 21 wendet diese Logik auf die zentralen Ergebnisse an. Die Verweise in der ersten Spalte machen die empirische Grundlage jedes Befunds unmittelbar rückverfolgbar.

Empirischer Befund	Betriebliche Prüffrage	Zusätzlich benötigte Prozessdaten
Hohe Lösungsabdeckung, aber geringe Wiederholungsstabilität über sechs Seeds (Tabellen 14 und 18)	Soll die Konfiguration primär Lösungsabdeckung oder wiederholbare Fallbearbeitung optimieren, und welches Mindestniveau an Zuverlässigkeit gilt?	Service-Level, Mindestzuverlässigkeit, Wiederholungs- und Transfertoleranz sowie Folgen inkonsistenter Ausführungen
C1 mit höchstem Mittelwert, positivem, konservativ unsicherem Kontrast von Faktor A sowie Saved- und Broken-Fällen (Tabellen 16 und 19)	Überwiegt der erwartete Qualitätsgewinn die Regressionen und Effektunsicherheit?	Reale Fallmischung, Fehlerfolgen, Nachbearbeitungs- und Eskalationsaufwand
C3 mit höchster strenger Stabilität, aber ohne belastbaren additiven Kombinationsvorteil (Tabellen 18 und 16)	Ist Stabilität wichtiger als der höchste Mittelwert, und rechtfertigt sie die zusätzliche Architektur?	Prozesskritikalität, tolerierte Leistungsschwankung und betrieblicher Wert stabiler Ausführung
C2 mit protokollierbarer Kontrollstelle, kleinem Faktor-B-Effekt und gegenläufigen Saved- und Broken-Fällen (Tabellen 16 und 19)	Welches Risiko soll Komponente B kontrollieren, und welcher Abbruch-, Übergabe- oder Wiederherstellungspfad folgt auf einen Eingriff?	Risikoklassen, Toleranz unnötiger Eingriffe sowie Eskalations- und Abschlussregeln
C1 und C3 mit rund 46 % höheren Agenten-API-Kosten und rund 27 % längerer technischer Laufzeit (Tabelle 20)	Ist der Zusatzaufwand für das angestrebte Qualitäts- oder Stabilitätsprofil tragfähig?	Fallvolumen, Kapazitäts- und Service-Level-Grenzen sowie API-, Betriebs- und Wartezeitfolgen

Tabelle 21. Überführung der empirischen Befunde in betriebliche Aktivierungsfragen

Die Tabelle empfiehlt keine der vier Konfigurationen. Sie zeigt vielmehr, welche zusätzlichen Unternehmensdaten benötigt werden, um die beobachteten Qualitäts-, Stabilitäts-, Kontroll- und Aufwandsprofile im jeweiligen Serviceprozess begründet zu gewichten. Damit bleibt die unmittelbar beobachtete Benchmarkevidenz von der kontextabhängigen betrieblichen Aktivierungsentscheidung getrennt.

9.4 Güte, Geltungsgrenzen und weiterer Forschungsbedarf

Validität der Befunde

Konstruktvalidität. Getrennte Maße stützen die Konstruktvalidität: DB Match erfasst den Geschäftsendzustand, Exact Mutation die Schreibaktionen, Saved/Broken Ergebniswechsel, Mechanismusergebnisse die Kontrollausführung und Ressourcenmetriken den Aufwand. Die ausführbare Referenzgrundlage misst jedoch Korrektheit nur relativ zu den im Benchmark hinterlegten Policies und Zielzuständen. Sie bildet weder alternative, im Unternehmenskontext ebenfalls vertretbare Lösungen noch vollständige Kommunikations- oder Beziehungsqualität ab.

Interne und Schlussfolgerungvalidität. Vollständige Task-Seed-Blöcke, identische Laufzeitparameter und getrennte Referenzbewertung stützen die interne Validität. Für die Schlussfolgerungvalidität gilt, dass ein positiver Punktwert für sich allein keinen verallgemeinerbaren Effekt belegt und ein null einschließendes Intervall keine Evidenz für Gleichheit liefert (vgl. Dror et al. (2018), S. 1383–1387). Die Arbeit berichtet deshalb Richtung, Größenordnung, Seed-Verteilung und konservative Unsicherheit gemeinsam.

Die sechs Seeds erhöhen die Beobachtungsdichte, nicht die Zahl unabhängiger Aufgaben von 80 auf 480. Cluster- und Zweiwegintervalle berücksichtigen die geteilten Aufgabenstrukturen, beseitigen aber nicht den begrenzten Umfang (vgl. Owen (2007), S. 386–392; vgl. Agarwal et al. (2021), Abschn. 1–4). Sechs bezeichnet den einheitlichen Wiederholungsumfang, keine allgemeine Powergrenze. Er macht Seedverteilung, Pass-Maße und Einzelaufabhängigkeit sichtbar, schätzt kleine oder variable Unterschiede aber unpräzise. Deshalb wird kein günstiges Intervallverfahren herausgegriffen.

Mechanismenvalidität. Kartenexposition und Entscheidungen von Komponente B entstehen endogen aus dem Agentenpfad. Der nachgelagerte Referenzabgleich diagnostiziert Selektivität, stand Komponente B aber zur Laufzeit nicht zur Verfügung. Die Mechanismusanalyse plausibilisiert daher Wirkpfade und Designgrenzen, ist jedoch keine randomisierte Mediationsanalyse.

Externe Validität. Die externe Validität bleibt auf Retail-Aufgaben, simulierte Nutzer, eine Modellkonfiguration und eine Artefaktinstanziierung begrenzt. Agent und Komponente A verwenden dasselbe Modell. Der User-Simulator nutzt ein separates Modell desselben externen Dienstes. Die zustandslosen, voneinander getrennten API-Aufrufe verhindern ein anwendungsseitig geteiltes Gedächtnis, belegen aber keine Unabhängigkeit von gemeinsamen Modell- oder Providercharakteristika. Übertragbar ist daher vor allem das Evaluationsmuster, nicht die konkrete Effektgröße.

Weitere Forschung

Erstens ist der Befund zu Komponente A auf einer neuen Aufgabenpopulation, weiteren Modellen und realitätsnäheren Serviceprozessen zu replizieren. Zweitens sollte Komponente A nur bei zusätzlichem Entscheidungswert materialisiert werden und zeitliche Konflikte zwischen Widerruf, Ersetzung und auftragsbezogener Präferenz explizit auflösen. Drittens sind für Komponente B Evidenzgewinn, Neuplanung, Abbruch und sicherer Abschluss als gemeinsamer Kontrollvertrag zu entwickeln. Ob Komponente B neben Machbarkeit auch semantische Zieltreue prüfen soll, ist dabei als neuer Informations- und Autoritätsumfang zu behandeln und erneut faktoriell zu evaluieren.

Der nächste betriebliche Schritt ist ein Pilot im Beobachtungsmodus mit realen Prozessdaten. Dort werden Interventionen zunächst protokolliert, ohne produktive Schreibentscheidungen zu verändern. So lassen sich Kontrollausführung, Fehlerprofile, Prozessfolgen und Ressourcen unter realen organisatorischen Bedingungen beobachten, bevor anhand vorab definierter Grenzwerte über eine begrenzte Aktivierung entschieden wird.

Insgesamt lokalisiert die Arbeit einen positiven, aber konservativ unsicheren Befund zu Faktor A, die Wirkgrenze der implementierten Kontrolle und die Evidenzbasis einer verantwortbaren Aktivierungsentscheidung.

10 Fazit und Ausblick

Kern der Arbeit

Transaktionale LLM-Agenten verändern Geschäftsdaten und verbinden dadurch Nutzen mit Fehlerrisiken. Die Arbeit entwickelte eine modular schaltbare Laufzeitinterventionsschicht. Komponente A stellt belegte Nutzerziele beratend vor der Agentenentscheidung bereit, Komponente B prüft mutierende Toolkandidaten deterministisch vor ihrer Ausführung. Beide autoritätsgetrennten Komponenten wurden in einem vollständigen 2×2 -Design anhand von Erfolg, Mutationsgenauigkeit, Stabilität, Saved-/Broken-Wechseln, Mechanismusspuren und Aufwand evaluiert.

Beantwortung der zentralen Forschungsfrage

Die Retail-Evaluation umfasst 80 Aufgaben, sechs Seeds und vier Bedingungen, also 1.920 Zellen. C1 erreicht die höchsten mittleren Endpunkte. Gegenüber C0 steigt DB Match von 71,3 % auf 77,3 % und Exact Mutation von 67,8 % auf 74,1 %. Der Haupteffekt von Faktor A bleibt in allen Auswertungen positiv, in denen jeweils ein Seed ausgelassen wird. Der konservative Zweiweg-Bootstrap umfasst jedoch null. Der Befund ist daher ein konsistenter positiver Hinweis und kein universell gesicherter Effekt.

Unter C1 wechseln 73 native Exact-Fehlschläge zu Erfolg, aber auch 46 Erfolge zu Misserfolg. C3 erreicht nicht den höchsten Mittelwert, besitzt mit $26/72 = 36,1 \%$ jedoch die höchste strenge Sechs-Seed-Stabilität. C2 zeigt, dass eine technisch ausgeführte Pre-Write-Kontrolle nicht automatisch fachlich selektiv ist oder Gesamtnutzen erzeugt. Ihr eigenständiger Haupteffekt ist nicht belastbar. Auch für die Kombination ist kein additiver Vorteil belastbar nachgewiesen.

Bereits C0 löst innerhalb der sechs Wiederholungen 79 von 80 DB-Aufgaben und 69 von 72 schreibpositiven Aufgaben mindestens einmal. Während $\text{pass}@6$ über alle Bedingungen 94,4 bis 98,8 % erreicht, liegt pass^6 nur bei 27,8 bis 40,0 %. Die zentrale Lücke ist damit die unzuverlässige Realisierung grundsätzlich beobachteter Fähigkeiten. Die Interventionen verändern vor allem Mittelwert, Stabilität und Fehlerverläufe, nicht die Aufgabenabdeckung.

C1 und C3 erhöhen Agenten-API-Kosten um rund 46 % und technische Laufzeit um rund 27 % gegenüber C0. Eine Aktivierung hängt daher von der betrieblichen Gewichtung mittleren Erfolgs, wiederholbarer Zuverlässigkeit, Regressionen und Aufwand ab.

Die zentrale Forschungsfrage ist konditional zu beantworten. Quellengebundene semantische Unterstützung weist in der untersuchten Konfiguration einen positiven beobachteten Kontrast beim mittleren fachlichen Erfolg auf, verändert Agentenpfade aber in beide Ergebnisrichtungen. Die Pre-Write-Kontrolle schafft eine nachvollziehbare und begrenzte Kontrollstelle, erzielt unter ihrer aktuellen Informationsgrenze aber keinen belastbaren eigenständigen

Leistungsbeitrag. Die Kombination erreicht die höchste strenge Wiederholungsstabilität, ohne belastbaren additiven Vorteil. Nutzen folgt damit nicht aus technischer Plausibilität, sondern aus dem beobachteten Profil aus Erfolg, Stabilität, Regressionen und Ressourcen.

Beitrag

Der Artefaktbeitrag besteht in einer modularen Laufzeitinterventionsschicht, die semantische Unterstützung und Pre-Write-Kontrolle unter gleicher Informations- und Bewertungsgrundlage unabhängig schaltbar macht. Dadurch lassen sich beide Eingriffsprinzipien getrennt gestalten und dennoch innerhalb desselben Agentenpfads untersuchen.

Der empirische Beitrag verbindet ein vollständiges faktorielles Design mit sechs Wiederholungen sowie Erfolgs-, Fehler-, Stabilitäts-, Mechanismus- und Aufwandsmaßen. Damit wird nicht nur sichtbar, ob eine Intervention den mittleren Erfolg verändert, sondern auch, welche zuvor korrekten Verläufe sie beschädigt, wie stabil ihre Wirkung ausfällt und welchen zusätzlichen Aufwand sie verursacht.

Der betriebliche Erkenntnisbeitrag liegt in einer evidenzgebundenen Entscheidungslogik für die Aktivierung solcher Interventionen. Die Ergebnisse werden nicht unmittelbar in eine allgemeine Einsatzempfehlung übersetzt, sondern mit den zusätzlich benötigten Prozessdaten zu Fehlerfolgen, Zuverlässigkeitsanforderungen und Ressourcen verknüpft. Das daraus abgeleitete Designwissen verdichtet die Befunde in fünf Regeln zu Nettonachweis, Wiederherstellung, selektiver Unterstützung, faktorieller Evaluation und wirkungsbezogener Kontrolle.

Grenzen und Ausblick

Die Ergebnisse gelten für die untersuchte Retail-Domäne, die verwendete Modellkonfiguration, simulierte Nutzer und die eingefrorene Artefaktversion. Die Benchmarkendpunkte messen Korrektheit relativ zu den hinterlegten Policies und Zielzuständen. Agent und Komponente A verwenden dasselbe Modell, während der User-Simulator ein separates Modell desselben externen Dienstes nutzt. Getrennte zustandslose Aufrufe verhindern ein anwendungsseitig geteiltes Gedächtnis, schließen gemeinsame Modell- oder Providercharakteristika als Einfluss jedoch nicht aus. Da zudem keine realen Kunden-, Prozess- oder Organisationsdaten erhoben wurden, lassen sich die beobachteten Wirkungswerte nicht unmittelbar auf reale Unternehmen oder andere Domänen übertragen. Übertragbar ist vor allem das Evaluations- und Entscheidungsmuster. Es umfasst die native Referenz, getrennt geschaltete Interventionen, die gepaarte Messung von Ergebnis und Fehlerfolgen, die Stabilitätsprüfung sowie die Trennung gemessener Ressourcen von offenen betrieblichen Bewertungsgrößen.

Anschlussforschung sollte den Befund zu Komponente A auf neuen Aufgaben und Modellen replizieren, den Trigger von Komponente A selektiver gestalten und für Komponente B Evidenzgewinn, Neuplanung und sicheren Abschluss gemeinsam untersuchen. Ein Pilot im Beobachtungsmodus mit realen Prozessdaten kann Interventionen zunächst nur protokollieren und erst nach stabil positiver Nettonutzenbilanz aktivieren.

Die Arbeit zeigt keine pauschale Überlegenheit zusätzlicher Architektur, sondern verschiebt die Entscheidungsgrundlage vom technischen Kontrollumfang zum belegten Nettonutzen einer Intervention.

Literaturverzeichnis

- Adams, R. J., Smart, P. & Huff, A. S. (2017). Shades of Grey: Guidelines for Working with the Grey Literature in Systematic Reviews for Management and Organizational Studies. *International Journal of Management Reviews*, 19(4), 432–454. <https://doi.org/10.1111/ijmr.12102>
- Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A. C. & Bellemare, M. G. (2021). Deep Reinforcement Learning at the Edge of the Statistical Precipice, In *Advances in Neural Information Processing Systems*. Verfügbar 2026-06-13 unter <https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html>
- Arcadinho, S., Aparicio, D. O. & Almeida, M. S. C. (2024). Automated test generation to evaluate tool-augmented LLMs as conversational AI agents, In *Proceedings of the 2nd GenBench Workshop on Generalisation (Benchmarking) in NLP*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.genbench-1.4>
- Atil, B., Aykent, S., Chittams, A., Fu, L., Passonneau, R. J., Radcliffe, E., Rajagopal, G. R., Sloan, A., Tudrej, T., Ture, F., Wu, Z., Xu, L. & Baldwin, B. (2025). Non-Determinism of “Deterministic” LLM System Settings in Hosted Environments, In *Proceedings of the 5th Workshop on Evaluation and Comparison of NLP Systems*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.eval4nlp-1.12>
- Balaji, S., Mishra, P., Sachdeva, A. & Agrawal, S. (2026). Beyond IVR: Benchmarking Customer Support LLM Agents for Business-Adherence (Y. Matuskevych, G. Eryiğit & N. Aletras, Hrsg.). In Y. Matuskevych, G. Eryiğit & N. Aletras (Hrsg.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 5: Industry Track)*, Rabat, Morocco, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2026.eacl-industry.15>
- Barres, V., Dong, H., Ray, S., Si, X. & Narasimhan, K. (2026). τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment, In *Proceedings of the 43rd International Conference on Machine Learning*. Verfügbar 2026-05-20 unter <https://openreview.net/forum?id=OC2z7iSQKa>
- Braggaar, A., Liebrecht, C., Miltenburg, E. v. & Krahmer, E. (2026). Evaluating Task-oriented Dialogue Systems: A Systematic Review of Measures, Constructs and their Operationalisations. *Northern European Journal of Language Technology*, 12(1), 1–38. <https://doi.org/10.3384/nejlt.2000-1533.2026.5940>
- Brynjolfsson, E., Li, D. & Raymond, L. R. (2025). Generative AI at Work. *The Quarterly Journal of Economics*, 140(2), 889–942. <https://doi.org/10.1093/qje/qjae044>
- Cai, Y., Zhu, Y., Gao, L., Tang, W. & Qin, S. (2026). Push Your Agent: Measuring and Enforcing Quantitative Goal Persistence in Long-Horizon LLM Agents. Verfügbar 2026-06-29 unter <https://arxiv.org/abs/2605.23574>
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021). Evaluating Large Language Models Trained on Code. <https://doi.org/10.48550/arXiv.2107.03374>
- Chen, Z., Liu, H., Xu, D., Dong, D., Li, J., Pu, B. & Zhai, J. (2026). Cordon: Semantic Transactions for Tool-Using LLM Agents. <https://doi.org/10.48550/arXiv.2606.17573>
- Choi, J., Yoon, J., Le, L. T., Jha, S. & Pfister, T. (2026). PolicyBank: Evolving Policy Understanding for LLM Agents, In *ICLR 2026 Workshop on Lifelong Learning of Agents*. Verfügbar 2026-06-29 unter <https://openreview.net/group?id=ICLR.cc/2026/Workshop/LLA>

- Colas, C., Sigaud, O. & Oudeyer, P.-Y. (2018). How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments. <https://doi.org/10.48550/arXiv.1806.08295>
- Collier, D. (2011). Understanding Process Tracing. *PS: Political Science & Politics*, 44(4), 823–830. <https://doi.org/10.1017/S1049096511001429>
- Cuadron, A., Yu, P., Liu, Y. & Gupta, A. (2025). SABER: Small Actions, Big Errors – Safeguarding Mutating Steps in LLM Agents, In *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems*. <https://doi.org/10.48550/arXiv.2512.07850>
- Ding, T. & De la Cruz Weinstein, J. P. (2026). When Retrieval Metrics Mislead: Measuring Policy Signal in Long-Horizon Tool-Use Agents. Verfügbar 2026-06-15 unter <https://arxiv.org/abs/2606.23937>
- Dong, X., Zhang, X., Bu, W., Zhang, D. & Cao, F. (2024). A Survey of LLM-based Agents: Theories, Technologies, Applications and Suggestions, In *2024 3rd International Conference on Artificial Intelligence, Internet of Things and Cloud Computing Technology (AIOTC)*. <https://doi.org/10.1109/AIOTC63215.2024.10748304>
- Dror, R., Baumer, G., Shlomov, S. & Reichart, R. (2018). The Hitchhiker’s Guide to Testing Statistical Significance in Natural Language Processing, In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/P18-1128>
- Dumas, M., La Rosa, M., Mendling, J. & Reijers, H. A. (2018). *Fundamentals of Business Process Management* (2. Aufl.). Berlin, Heidelberg, Springer. <https://doi.org/10.1007/978-3-662-56509-4>
- Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1–26. <https://doi.org/10.1214/aos/1176344552>
- El-Yaniv, R. & Wiener, Y. (2010). On the Foundations of Noise-free Selective Classification. *Journal of Machine Learning Research*, 11, 1605–1641. Verfügbar 2026-06-14 unter <https://jmlr.org/papers/v11/el-yaniv10a.html>
- Eurostat. (2026). *The Use of Artificial Intelligence Technologies in the European Union: Key Results – 2026 Edition* (Statistical report KS-01-26-009). Publications Office of the European Union. <https://doi.org/10.2785/9221093>
- Gnewuch, U., Morana, S., Adam, M. T. P. & Maedche, A. (2022). Opposing Effects of Response Time in Human–Chatbot Interaction: The Moderating Role of Prior Experience. *Business & Information Systems Engineering*, 64(6), 773–791. <https://doi.org/10.1007/s12599-022-00755-x>
- Gregor, S. & Hevner, A. R. (2013). Positioning and Presenting Design Science Research for Maximum Impact. *MIS Quarterly*, 37(2), 337–355. <https://doi.org/10.25300/MISQ/2013/37.2.01>
- Gregor, S. & Jones, D. (2007). The Anatomy of a Design Theory. *Journal of the Association for Information Systems*, 8(5), 312–335. <https://doi.org/10.17705/1jais.00129>
- Grönroos, C. & Ojasalo, K. (2004). Service Productivity: Towards a Conceptualization of the Transformation of Inputs into Economic Results in Services. *Journal of Business Research*, 57(4), 414–423. [https://doi.org/10.1016/S0148-2963\(02\)00275-8](https://doi.org/10.1016/S0148-2963(02)00275-8)
- Gupta, A. (2026). ReliabilityBench: Evaluating LLM Agent Reliability Under Production-Like Stress Conditions. <https://doi.org/10.48550/arXiv.2601.06112>
- Härder, T. & Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. *ACM Computing Surveys*, 15(4), 287–317. <https://doi.org/10.1145/289.291>
- Hevner, A. R., March, S. T., Park, J. & Ram, S. (2004). Design Science in Information Systems Research. *MIS Quarterly*, 28(1), 75–106. <https://doi.org/10.2307/25148625>

- Huang, K.-H., Prabhakar, A., Dhawan, S., Mao, Y., Wang, H., Savarese, S., Xiong, C., Laban, P. & Wu, C.-S. (2025). CRMArena: Understanding the Capacity of LLM Agents to Perform Professional CRM Tasks in Realistic Environments, In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.naacl-long.194>
- Kamath, A., Zhang, S., Xu, C., Ugare, S., Singh, G. & Misailovic, S. (2026). Enforcing Temporal Constraints for LLM Agents, In *ICLR 2026 Workshop on VerifAI: AI Verification in the Wild*. Verfügbar 2026-06-15 unter <https://openreview.net/group?id=ICLR.cc/2026/Workshop/VerifAI-2>
- Kampik, T., Warmuth, C., Rebmann, A., Agam, R., Egger, L. N. P., Gerber, A., Hoffart, J., Kolk, J., Herzig, P., Decker, G., van der Aa, H., Polyvyanyy, A., Rinderle-Ma, S., Weber, I. & Weidlich, M. (2025). Large Process Models: A Vision for Business Process Management in the Age of Generative AI. *KI – Künstliche Intelligenz*, 39, 81–95. <https://doi.org/10.1007/s13218-024-00863-8>
- Kitchenham, B. & Charters, S. (2007). *Guidelines for Performing Systematic Literature Reviews in Software Engineering* (Technical Report EBSE-2007-01). Keele University und Durham University. Verfügbar 2026-05-27 unter <https://www.cin.ufpe.br/~in1037/leitura/kitchenham2007guidelines.pdf>
- Li, Z., Huang, S., Wang, J. L., Zhang, N., Antoniadou, A., Hua, W., Zhu, K., Zeng, S., Wang, C. C., Wang, W. Y. & Yan, X. (2025). SOPBench: Evaluating Language Agents at Following Standard Operating Procedures and Constraints. <https://doi.org/10.48550/arXiv.2503.08669>
- Lim, M. D., Bin Sharudin, I. D., Chen, W. H., Lim, C. & Wynter, L. (2026). Declarative Skills for AI Agents in Knowledge-Grounded Tool-Use Workflows. <https://doi.org/10.48550/arXiv.2606.06923>
- Lu, J., Holleis, T., Zhang, Y., Aumayer, B., Nan, F., Bai, H., Ma, S., Ma, S., Li, M., Yin, G., Wang, Z. & Pang, R. (2025). ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities, In *Findings of the Association for Computational Linguistics: NAACL 2025*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.findings-naacl.65>
- Lu, R., Li, Y. & Huo, Y. (2025). Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks. arXiv. <https://doi.org/10.48550/ARXIV.2508.13143>
- Ma, C., Zhang, J., Zhu, Z., Yang, C., Yang, Y., Jin, Y., Lan, Z., Kong, L. & He, J. (2024). AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents, In *Advances in Neural Information Processing Systems* 37. <https://doi.org/10.52202/079017-2365>
- Ma, R., Qu, J., Bobu, A. & Hadfield-Menell, D. (2025). Flexible Agent Alignment with Goal Inference from Open-Ended Dialog. Verfügbar 2026-06-12 unter <https://arxiv.org/abs/2508.15119>
- March, S. T. & Smith, G. F. (1995). Design and natural science research on information technology. *Decision Support Systems*, 15(4), 251–266. [https://doi.org/10.1016/0167-9236\(94\)00041-2](https://doi.org/10.1016/0167-9236(94)00041-2)
- Mishra, V., Saeidi, A., Raj, S., Nakamura, M., Liu, G., Payani, A., Srinivasa, J. & Baral, C. (2025). How Can Input Reformulation Improve Tool Usage Accuracy in a Complex Dynamic Environment? A Study on τ -bench, In *Findings of the Association for Computational Linguistics: EMNLP 2025*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.findings-emnlp.1250>

- Mohammadi, B., Potamitis, N., Klein, L., Arora, A. & Bindschaedler, L. (2026). Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. <https://doi.org/10.48550/arXiv.2602.14849>
- Moreau, L. & Missier, P. (2013). *PROV-DM: The PROV Data Model* (W3C Recommendation). World Wide Web Consortium. Verfügbar 2026-06-22 unter <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>
- Nakash, I., Kour, G., Lazar, K., Vetzler, M., Uziel, G. & Anaby Tavor, A. (2025). Effective Red-Teaming of Policy-Adherent Agents, In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.emnlp-main.114>
- National Institute of Standards and Technology. (n. d.). *Fail Safe*. Computer Security Resource Center Glossary. Verfügbar 2026-06-22 unter https://csrc.nist.gov/glossary/term/fail_safe
- Okoli, C. (2015). A Guide to Conducting a Standalone Systematic Literature Review. *Communications of the Association for Information Systems*, 37(43), 879–910. <https://doi.org/10.17705/1CAIS.03743>
- OpenAI. (2026). *Chat Completions API Reference*. Verfügbar 2026-06-10 unter <https://developers.openai.com/api/reference/resources/chat>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J. & Lowe, R. (2022). Training Language Models to Follow Instructions with Human Feedback, In *Advances in Neural Information Processing Systems* 35. <https://doi.org/10.48550/arXiv.2203.02155>
- Owen, A. B. (2007). The Pigeonhole Bootstrap. *The Annals of Applied Statistics*, 1(2), 386–411. <https://doi.org/10.1214/07-AOAS122>
- Paez, A. (2017). Gray Literature: An Important Resource in Systematic Reviews. *Journal of Evidence-Based Medicine*, 10(3), 233–240. <https://doi.org/10.1111/jebm.12266>
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., ... Moher, D. (2021). The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews. *BMJ*, 372, n71. <https://doi.org/10.1136/bmj.n71>
- Palinkas, L. A., Horwitz, S. M., Green, C. A., Wisdom, J. P., Duan, N. & Hoagwood, K. (2015). Purposeful Sampling for Qualitative Data Collection and Analysis in Mixed Method Implementation Research. *Administration and Policy in Mental Health and Mental Health Services Research*, 42(5), 533–544. <https://doi.org/10.1007/s10488-013-0528-y>
- Peffer, K., Tuunanen, T., Rothenberger, M. A. & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Peyrard, M., Zhao, W., Eger, S. & West, R. (2021). Better than Average: Paired Evaluation of NLP Systems, In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.acl-long.179>

- Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J. & Wen, J.-r. (2025). Tool learning with large language models: a survey. *Frontiers of Computer Science*, 19(8), 198343. <https://doi.org/10.1007/s11704-024-40678-2>
- Rana, M., Man, C., Msiwa, A. E., Paine, J., Zhu, K., Dev, S., Sharma, V. & R, A. M. (2025). AgentChangeBench: A Multi-Dimensional Evaluation Framework for Goal-Shift Robustness in Conversational AI, In *NeurIPS 2025 Workshop on Multi-Turn Interactions in Large Language Models*. Verfügbar 2026-06-12 unter <https://openreview.net/forum?id=ZCi58UP9uR>
- Ray, G., Muhanna, W. A. & Barney, J. B. (2005). Information Technology and the Performance of the Customer Service Process: A Resource-Based Analysis. *MIS Quarterly*, 29(4), 625–652. <https://doi.org/10.2307/25148703>
- Reimers, N. & Gurevych, I. (2017). Reporting Score Distributions Makes a Difference: Performance Study of LSTM-networks for Sequence Tagging, In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/D17-1035>
- Renze, M. (2024). The Effect of Sampling Temperature on Problem Solving in Large Language Models, In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.findings-emnlp.432>
- Rethlefsen, M. L., Kirtley, S., Waffenschmidt, S., Ayala, A. P., Moher, D., Page, M. J. & Koffel, J. B. (2021). PRISMA-S: An Extension to the PRISMA Statement for Reporting Literature Searches in Systematic Reviews. *Systematic Reviews*, 10(1), 39. <https://doi.org/10.1186/s13643-020-01542-z>
- Rinderle-Ma, S., Winter, K. & Benzin, J. V. (2023). Predictive Compliance Monitoring in Process-Aware Information Systems: State of the Art, Functionalities, Research Directions. *Information Systems*, 115, 102210. <https://doi.org/10.1016/j.is.2023.102210>
- Runeson, P. & Höst, M. (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Saeidi, A., Mishra, V., Mukhopadhyay, S., Liu, G., Payani, A., Srinivasa, J. & Baral, C. (2026). FAMA: Failure-Aware Meta-Agentic Framework for Open-Source LLMs in Interactive Tool Use Environments, In *Findings of the Association for Computational Linguistics: ACL 2026*, San Diego, California, United States, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2026.findings-acl.1716>
- Sah, T., Srivastava, V., Sah, D. & Jordan, K. (2026). The Verifier Tax: Horizon Dependent Safety Success Tradeoffs in Tool Using LLM Agents, In *Proceedings of the ACM Conference on AI and Agentic Systems*. <https://doi.org/10.1145/3786335.3813160>
- Saltzer, J. H. & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- Satija, S., Bhatt, A., Jani, P. & Rawal, D. (2026). fastWorkflow: Closing the Performance Gap Between Small and Frontier Language Models for Conversational Agents, In *Proceedings of the ACM Conference on AI and Agentic Systems*, Association for Computing Machinery. <https://doi.org/10.1145/3786335.3813158>
- Sheng, S., Wang, Z. & Zhou, L. (2026). When Agents Overtrust Environmental Evidence: An Extensible Agentic Framework for Benchmarking Evidence-Grounding Defects in LLM Agents. Verfügbar 2026-06-12 unter <https://arxiv.org/abs/2605.08828>

- Shi, Q., Zytek, A., Razavi, P., Narasimhan, K. & Barres, V. (2026). τ -Knowledge: Evaluating Conversational Agents over Unstructured Knowledge, In *Proceedings of the 43rd International Conference on Machine Learning*. Verfügbar 2026-05-20 unter <https://openreview.net/forum?id=XHZK5abtw2>
- Shi, Y., Cai, S., Xu, Z., Qin, Y., Li, G., Shao, H., Chen, J., Yang, D., Li, K. & Sun, X. (2025). FlowAgent: Achieving Compliance and Flexibility for Workflow Agents. Verfügbar 2026-06-17 unter <https://arxiv.org/abs/2502.14345>
- Tranfield, D., Denyer, D. & Smart, P. (2003). Towards a Methodology for Developing Evidence-Informed Management Knowledge by Means of Systematic Review. *British Journal of Management*, 14(3), 207–222. <https://doi.org/10.1111/1467-8551.00375>
- van der Aalst, W. M. P. (2022). Six Levels of Autonomous Process Execution Management. <https://doi.org/10.48550/arXiv.2204.11328>
- van der Aalst, W. M. P. & van Hee, K. M. (2002). *Workflow Management: Models, Methods, and Systems*. Cambridge, MA, MIT Press. <https://doi.org/10.7551/mitpress/7301.001.0001>
- Van Looy, A. & Shafagatova, A. (2016). Business Process Performance Measurement: A Structured Literature Review of Indicators, Measures and Metrics. *SpringerPlus*, 5(1), 1797. <https://doi.org/10.1186/s40064-016-3498-1>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017). Attention Is All You Need, In *Advances in Neural Information Processing Systems 30*. <https://doi.org/10.48550/arXiv.1706.03762>
- Venable, J., Pries-Heje, J. & Baskerville, R. (2016). FEDS: a Framework for Evaluation in Design Science Research. *European Journal of Information Systems*, 25(1), 77–89. <https://doi.org/10.1057/ejis.2014.36>
- Wang, A., Hager, S., Asija, A., Khashabi, D. & Andrews, N. (2025). Hell or High Water: Evaluating Agentic Recovery from External Failures, In *Conference on Language Modeling*. Verfügbar 2026-06-12 unter <https://openreview.net/forum?id=Zk224WPT42>
- Wang, H., Poskitt, C. M. & Sun, J. (2026). AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents, In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*. Verfügbar 2026-06-15 unter <https://arxiv.org/abs/2503.18666>
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z. & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6). <https://doi.org/10.1007/s11704-024-40231-1>
- Webster, J. & Watson, R. T. (2002). Analyzing the Past to Prepare for the Future: Writing a Literature Review. *MIS Quarterly*, 26(2), xiii–xxiii. Verfügbar 2026-05-21 unter <https://www.jstor.org/stable/4132319>
- Winston, C., Winston, C. & Just, R. (2026). Solver-Aided Verification of Policy Compliance in Tool-Augmented LLM Agents. Verfügbar 2026-06-14 unter <https://arxiv.org/abs/2603.20449>
- Wohlin, C. (2014). Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering, In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, ACM. <https://doi.org/10.1145/2601248.2601268>
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B. & Wesslén, A. (2024). *Experimentation in Software Engineering* (2. Aufl.). Berlin, Heidelberg, Springer. <https://doi.org/10.1007/978-3-662-69306-3>

- Wooldridge, M. & Jennings, N. R. (1995). Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*, 10(2), 115–152. <https://doi.org/10.1017/S0269888900008122>
- Wu, Y., Yue, T., Zhang, S., Wang, C. & Wu, Q. (2024). StateFlow: Enhancing LLM Task-Solving through State-Driven Workflows, In *Proceedings of the First Conference on Language Modeling*. Verfügbar 2026-06-17 unter <https://openreview.net/forum?id=3nTbuygoop>
- Yao, S., Shinn, N., Razavi, P. & Narasimhan, K. (2025). τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, In *International Conference on Learning Representations*. Verfügbar 2026-05-27 unter <https://openreview.net/forum?id=roNSXZpUDN>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models, In *International Conference on Learning Representations*. Verfügbar 2026-06-13 unter https://openreview.net/forum?id=WE_vluYUL-X
- Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A. & Shmueli-Scheuer, M. (2025). Survey on Evaluation of LLM-based Agents. arXiv. <https://doi.org/10.48550/ARXIV.2503.16416>
- Zhang, S., Wang, A. & Lei, S. (2026). Catching One in Five: LLM-as-Judge Blind Spots in Production Multi-Turn Transaction Agents. Verfügbar 2026-06-12 unter <https://arxiv.org/abs/2606.10315>
- Zhang, X., Gao, M., Zhao, Y., Tan, X., Yao, Y., Wang, F., Wang, Y., Dingsiyi & Yang, T. (2026). Formal Skill: Programmable Runtime Skills for Efficient and Accurate LLM Agents. Verfügbar 2026-06-29 unter <https://arxiv.org/abs/2605.19604>
- Zwerdling, N., Boaz, D., Rabinovich, E., Uziel, G., Amid, D. & Anaby Tavor, A. (2025). Towards Enforcing Company Policy Adherence in Agentic Workflows, In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.emnlp-industry.41>

Anhang A Ergänzende Forschungsartefakte

Der Anhang ergänzt die für das Verständnis erforderliche Darstellung im Haupttext um prüfbare Detailnachweise. Aufgenommen werden nur Artefakte, die mindestens eine von vier Funktionen erfüllen: Reproduzierbarkeit des Versuchsaufbaus, Nachvollziehbarkeit der Auswertung, Einsicht in die Wirkpfade oder Abgrenzung der zulässigen Aussagen. Der digitale Forschungsanhang ergänzt diese Nachweise um den unmittelbar relevanten Codekern, finale Evidenztabelle, aggregierte maschinenlesbare Auswertungen und die sechs ausgewählten Ereignisspurvignetten. Dadurch bleibt der gedruckte Anhang lesbar, ohne mehr Material als für die Nachvollziehbarkeit erforderlich offenzulegen.

Alle nachfolgend mit `code_core/` beginnenden Pfade sind relativ zur Wurzel des abgegebenen Forschungsrepositorys zu lesen. Die hier ausgewiesenen Ergebnisse beziehen sich ausschließlich auf die finale Matrix aus 80 Retail-Aufgaben, sechs gemeinsamen Seeds und vier Bedingungen.

A.1 Ergänzende Tabellen des Argumentations- und Nachweispfads

Die folgenden Detailtabellen ergänzen die im Haupttext verdichtete Argumentation. Ihre Reihenfolge folgt dem Kapitelverlauf von Forschungsfrage und Grundlagen über Literaturstudie, Methode, Anforderungen und Gestaltung bis zu Demonstration und ergänzenden Seedanalysen. Die Tabellen werden im Haupttext an ihrer fachlich einschlägigen Stelle eingeordnet.

Dimension	Theoretischer Ursprung	Arbeitsdefinition im transaktionalen Support
Nutzerziel	Nutzerziel und Task Success in task-orientierten Dialogen	Noch offenes, vom Kunden geäußertes Geschäftsanliegen, etwa die Rückgabe eines bestimmten Artikels oder der Umtausch in eine konkrete Variante.
Evidenz	Toolbeobachtung und Informationsprovenienz	Beobachtbare, auf Nutzeräußerung oder Toolresultat rückführbare Information, die Kunden-, Bestell-, Artikel- oder Variantenbeziehungen stützt.
Prozesszu- stand	Fall- und Ausführungszustand in Workflow- und Prozesssystemen	Explizite Laufzeitrepräsentation belegter, offener oder widersprüchlicher Prozessinformationen und bereits ausgeführter Schritte.
Zustandsän- derung	Logisch zusammengehörige Änderung in Prozess- und Transaktionssystemen	Operative Änderung, deren Zielobjekt, Voraussetzungen, Argumente und gegebenenfalls Kundenbestätigung zusammen geprüft werden.
Quelle: vgl. Braggaar et al. (2026), S. 2–3, Qu et al. (2025), S. 3 und 5–11, Moreau; Missier (2013), Abschn. 2–5, van der Aalst; van Hee (2002), Kap. 2–3, Dumas et al. (2018), Kap. 1 und Härder; Reuter (1983), S. 289–290.		

Tabelle 22. Herleitung des analytischen Bezugsrahmens für transaktionale Agentenfehler

Rolle	Erkenntnisfunktion	Studien	Beitrag zur Arbeit
R1	Problem- und Fehlerbelege	16	Ergebnis-, Prozess- und Zuverlässig- keitsprobleme
R2	Semantische Unterstützung	7	Ziel-, Policy- und Evidenzrepräsen- tationen
R3	Deterministische Laufzeit- kontrolle	9	Prüfung, Autorisierung, Staging und Begrenzung
R4	Benchmark und Evaluation	16	Referenzgrundlage, Endpunkte und Wiederholungslogik
R5	Korrektur und Wiederherstel- lung	6	Neuplanung, Reparatur und Schaden- begrenzung
Anmerkung: Mehrfachzuordnungen sind möglich; die Spaltensumme übersteigt deshalb den Synthesekorpus.			

Tabelle 23. Analytische Rollen der 25 eingeschlossenen Studien

Formative Beobachtung	Designentscheidung	Finale Grenze und Nachweis
Breit gekoppelte Eingriffe erschwerten die Zuordnung von Unterstützung und Kontrolle.	Trennung in zwei zeitlich und technisch unabhängige Interventionspunkte mit separaten Faktorflags.	Komponente A wirkt vor der nativen Entscheidung, Komponente B vor der Schreibausführung; Faktorbelegung und Ports werden vertraglich geprüft.
Semantische Zustände waren entweder zu nah an harten Aktionsgrenzen oder für den nativen Pfad zu wenig sichtbar.	Reduktion auf quellengebundene Nutzerziel-Datensätze und eine selektiv materialisierte Unterstützungskarte.	Komponente A bleibt beratend; Quellenbindung, Schema und fehlende Schreibautorität werden automatisiert geprüft.
Eine kontrollseitige Reparatur hätte Kandidatenprüfung und Neuplanung vermischt.	Bindung von Komponente B an den unveränderten nativen Kandidaten mit drei Dispositionen und höchstens einer nativen Neuplanung.	Komponente B erzeugt und verändert keine Zielaktion; Kandidatenidentität, Evidenzpfad und Neuplanungsbudget sind Invarianten.
Lokale Komponentenentscheidungen ließen sich ohne gemeinsame Spur nicht belastbar mit Folgeaktionen und Episodenergebnis verbinden.	Einführung stabiler Gelegenheits-, Entscheidungs-, Kandidaten- und Aufruf-IDs in einem gemeinsamen Ereignisprotokoll.	Mechanismustelemetrie und Offline-Evaluator trennen lokale Funktion, Pfadwirkung und Gesamtergebnis.

Tabelle 24. Formative DSR-Iterationskette zur finalen Artefaktversion

Präfix	Objekt der Ereignisspur	Bedeutung im Lauf
message:n:user	Nutzerereignis	Die <i>n</i> -te protokollierte Nutzernachricht; sie dient Komponente A als zitierbare Quelle für übernommene Feldwerte.
AOP-nnnn	Aktivierungsgelegenheit von Komponente A	Eine konkrete Gelegenheit zur Aktivierung von Komponente A nach einer materiellen Nutzeräußerung. Sie verknüpft Trigger, Extraktionsentscheidung und deren Ergebnis.
G-nnnn	Nutzerziel-Datensatz	Stabile ID eines von Komponente A validierten, quellengebundenen Nutzerziels. Derselbe Datensatz kann durch spätere Nutzeraussagen um weitere belegte Felder ergänzt werden.
AC-nnnn	Unterstützungskarte	Materialisierter Kartenstand aus den aktuell gültigen G-Datensätzen und den sichtbar abgeschlossenen Schreibaktionen; ein neuer Stand erhält eine neue ID.
BOP-nnnn	Prüfgelegenheit von Komponente B	Eine konkrete Prüfgelegenheit der Pre-Write-Kontrolle für genau einen nativen Schreibkandidaten. An ihr hängen die Entscheidung von Komponente B und finale Disposition.
BB-nnnn-O/R	B-Kandidatenbundle	Deterministisch geprüfte Gruppe zusammengehöriger Tool-Aufrufe; O bezeichnet den ursprünglichen Kandidaten, R gegebenenfalls einen nach Neuplanung erzeugten Kandidaten.
call_...	Tool-Aufruf-ID	Eindeutige ID des nativen Tool-Aufrufs. Sie ermöglicht den Abgleich zwischen vorgeschlagenem, freigegebenem und tatsächlich ausgeführtem Schreibaufruf.
sequence	Ereignisposition	Fortlaufende Position eines Mechanismuserignisses in der gemeinsamen A-/B-Ereignisspur.

Tabelle 25. Leseschlüssel der episodischen Ereignisspur-IDs

A.2 Auditspur der Literaturstudie

Die kompakte Auditspur zu Kapitel 3 liegt unter `digital_research_appendix/literature_review_final/`. Tabelle 26 nennt den Nachweiskern. Die Dateien dokumentieren Such- und Auswahlrahmen, finale Auswahlzahlen, Evidenzgewichtung und konzeptzentrierte Synthese.

Nachweis	Funktion
00_START_HERE_FINAL. md, 00_PROTOCOL_ FREEZE.md, 02_EVIDENCE_ HIERARCHY.md und 03_ SEARCH_STRATEGIES.md	Kanonischer Einstieg, Suchrahmen, quellenspezifische Suchstrategien, Evidenzhierarchie und finale Auswahlregeln.
formal/evidence_ tables/final_study_ matrix.csv	Finaler Korpus von 25 Studien mit Identifikationsweg, Publikationsstatus, analytischen Rollen und Einschlussbegründung.
formal/evidence_ tables/quality_ appraisal.csv	Kriterienbezogene Qualitätsprüfung aller eingeschlossenen Studien ohne scheinpräzisen Gesamtrang.
formal/evidence_ tables/claim_evidence_ matrix.csv	Zuordnung zentraler Syntheseaussagen zu Belegen, Evidenzstufen und Aussagegrenzen.
formal/evidence_ tables/concept_matrix_ coding.csv	Zellenweise Begründung und Volltextfundstelle der Konzeptmatrix aus Kapitel 3.
formal/evidence_ tables/prisma_counts. json	Maschinenlesbare PRISMA-Arithmetik einschließlich aggregierter Ausschlusszahlen.

Tabelle 26. Zentrale Nachweise der Literaturstudie

A.3 Finaler Synthesekorpus der Literaturstudie

Tabelle 27 macht die 25 in Kapitel 3 tatsächlich synthetisierten Studien einzeln sichtbar. Die Tabelle ersetzt nicht das Literaturverzeichnis, sondern ergänzt dessen bibliographische Vollangaben um die im Review relevante Herkunft, Evidenzstufe und Erkenntnisfunktion. Dabei bezeichnet E1 regulär peer-reviewte Beiträge, E2 begutachtete Workshopbeiträge und E3 aktuelle Preprints. E1 darf zentrale Aussagen tragen; E2 wird triangulierend und E3 ausschließlich ergänzend verwendet. E3-V kennzeichnet zwei Preprints, deren Aussagen aufgrund zusätzlicher Qualitätsgrenzen mit erhöhter Vorsicht behandelt werden. Die analytischen Rollen lauten: R1 Problem- und Fehlerbeleg, R2 semantische Laufzeitunterstützung, R3 deterministische Laufzeitkontrolle, R4 Benchmark und Evaluation sowie R5 Korrektur und Wiederherstellung. Mehrfachrollen sind möglich.

Tabelle 27. Finaler Synthesekorpus mit Identifikationsweg, Evidenzstufe und Erkenntnisfunktion

ID und Quelle	Weg und Evidenz	Rolle	Funktion in der Synthese
2406.12045 Yao et al. (2025), Abschn. 3–5 „ τ -bench“	Snowballing E1, zentral	R1, R4	Policygebundene Airline- und Retail-dialoge, Datenbankendzustände und Wiederholungszuverlässigkeit.
2408.04682 Lu et al. (2025), Abschn. 3–5 „ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities“	Snowballing E1, zentral	R1, R4	Zustandsbehaftete Tools, implizite Abhängigkeiten, Nutzerinteraktion und Trajektorienmeilensteine.
2503.08669 Li et al. (2025), Abschn. 3–6 „SOPBench: Evaluating Language Agents at Following Standard Operating Procedures and Constraints“	Snowballing E3, ergänzend	R3, R4	Ausführbare Policies und regelbasierte Verifikatoren für sieben Kundenservice-domänen.
2506.07982 Barres et al. (2026), Abschn. 3–5 „ τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment“	Snowballing E1, zentral	R1, R4	Koordinierte Zustandsänderungen durch Nutzer und Agent in dynamischen Supportumgebungen.
2506.09600 Nakash et al. (2025), Abschn. 3–5 „Effective Red-Teaming of Policy-Adherent Agents“	Snowballing E1, zentral	R1, R4	Adversariale Nutzervarianten zur gezielten Prüfung der Policy-Adhärenz.
RS-0019 Rana et al. (2025), Abschn. 3–5 „AgentChangeBench: A Multi-Dimensional Evaluation Framework for Goal-Shift Robustness in Conversational AI“	Datenbank E2, trianguliert	R1, R4	Zielwechselrobustheit, Wiederherstellung, Redundanz, Effizienz und Aufgabenerfolg in mehrstufigen Dialogen.
RS-0060 Arcadinho; Aparicio; Almeida (2024), Abschn. 3–5 „Automated test generation to evaluate tool-augmented LLMs as conversational AI agents“	Datenbank E2, trianguliert	R1, R4	Prozedurgebundene Testgenerierung für vollständige toolgestützte Support-dialoge.

ID und Quelle	Weg und Evidenz	Rolle	Funktion in der Synthese
RS-0071 Balaji et al. (2026), S. 193–208 „Beyond IVR: Benchmarking Customer Support LLM Agents for Business-Adherence“	Datenbank E1, zentral	R2, R3, R4	Kundenservice-Workflows, Business-Adherence und strukturierte dynamische Orchestrierung.
RS-0084 Zhang; Wang; Lei (2026), Abschn. 4–7 „Catching One in Five: LLM-as-Judge Blind Spots in Production Multi-Turn Transaction Agents“	Datenbank E3, ergänzend	R1, R4	Blindstellen automatisierter LLM-Bewertungen bei turnübergreifendem Zustand, Schutzregeln und Wiederherstellung.
RS-0103 Chen et al. (2026), Abschn. 3–6 „Cordon: Semantic Transactions for Tool-Using LLM Agents“	Datenbank E3, ergänzend	R3	Transaktionales Staging, Validierung, Commit, Rollback und Audit zusammengesetzter Toolwirkungen.
RS-0107 Huang et al. (2025), Abschn. 3–5 „CRMArena: Understanding the Capacity of LLM Agents to Perform Professional CRM Tasks in Realistic Environments“	Datenbank E1, zentral	R1, R4	Realistische CRM-Aufgaben mit ausführbaren Tools und Endzustandsbewertung.
RS-0121 Kamath et al. (2026), Abschn. 3–6 „Enforcing Temporal Constraints for LLM Agents“	Datenbank E2, trianguliert	R3	Formale Spezifikation und Laufzeitdurchsetzung temporaler Regeln über Aktionsfolgen.
RS-0132 Ma et al. (2025), Abschn. 4–6 „Flexible Agent Alignment with Goal Inference from Open-Ended Dialog“	Datenbank E3, ergänzend	R2	Dynamische Extraktion und Aktualisierung natürlichsprachlicher Ziele aus offenen Dialogen.
RS-0153 Wang et al. (2025), Abschn. 3–5 „Hell or High Water: Evaluating Agentic Recovery from External Failures“	Datenbank E1, zentral	R1, R4	Kontrolliert lösbare externe Toolausfälle und Wiederherstellung über alternative Aktionspläne.
RS-0217 Choi et al. (2026), Abschn. 3 und 6 „PolicyBank: Evolving Policy Understanding for LLM Agents“	Datenbank E2, trianguliert	R2, R5	Turnweise Aktualisierung und Nutzung strukturierter Policy-Erkenntnisse in langen Transaktionsdialogen.

ID und Quelle	Weg und Evidenz	Rolle	Funktion in der Synthese
RS-0225 Cai et al. (2026), Abschn. 3–7 „Push Your Agent: Measuring and Enforcing Quantitative Goal Persistence in Long-Horizon LLM Agents“	Datenbank E3, ergänzend	R1, R2, R4	Verifizierter quantitativer Zielfortschritt sowie Messung von Zielpersistenz und Drift.
RS-0231 Gupta (2026), Abschn. 3–5 „ReliabilityBench: Evaluating LLM Agent Reliability Under Production-Like Stress Conditions“	Datenbank E3-V, ergänzend	R1, R4	Wiederholte Ausführung, Perturbation und Fehlerinjektion zur Messung von Stabilität und Wiederherstellung.
RS-0243 Cuadron et al. (2025), Abschn. 3–6 „SABER: Small Actions, Big Errors – Safeguarding Mutating Steps in LLM Agents“	Datenbank E2, trianguliert	R1, R3, R5	Mutierende Aktionen als Fehlerpunkt und Verifikation unmittelbar vor ihrer Ausführung.
RS-0263 Winston; Winston; Just (2026), Abschn. 3–5 „Solver-Aided Verification of Policy Compliance in Tool-Augmented LLM Agents“	Datenbank E3, ergänzend	R3, R5	Übersetzung natürlichsprachlicher Policies in Solver-Constraints vor Tool-Aufrufen.
RS-0280 Shi et al. (2026), Abschn. 4–6 „ τ -Knowledge: Evaluating Conversational Agents over Unstructured Knowledge“	Datenbank E1, zentral	R1, R4	Evidenzabruf, Policy-Reasoning, Tools und verifizierbare Zustandsänderungen im Banking.
RS-0292 Sah et al. (2026), Abschn. 3–5 „The Verifier Tax: Horizon Dependent Safety Success Tradeoffs in Tool Using LLM Agents“	Datenbank E1, zentral	R1, R3, R5	Sicherheits-, Erfolgs-, Wiederherstellungs- und Aufwandskonflikte durch Laufzeitmediation in Transaktionsaufgaben.
RS-0301 Zwerdling et al. (2025), Abschn. 3–6 „Towards Enforcing Company Policy Adherence in Agentic Workflows“	Datenbank E1, zentral	R3, R5	Kompilierung von Unternehmensrichtlinien in transparente Guards vor jeder Toolaktion.

ID und Quelle	Weg und Evidenz	Rolle	Funktion in der Synthese
RS-0323 Sheng; Wang; Zhou (2026), Abschn. 3–5 <i>„When Agents Overtrust Environmental Evidence: An Extensible Agentic Framework for Benchmarking Evidence-Grounding Defects in LLM Agents“</i>	Datenbank E3, ergänzend	R1, R2, R4	Provenienz-, Aktualitäts-, Autoritäts- und Handlungsbindungsfehler externer Evidenz.
RS-0327 Ding; De la Cruz Weinstein (2026), Abschn. 2–4 <i>„When Retrieval Metrics Mislead: Measuring Policy Signal in Long-Horizon Tool-Use Agents“</i>	Datenbank E3-V, ergänzend	R2, R3, R5	Policy-Signal im Retrieval und strukturierter Entscheidungszustand für Pre-Action-Compliance.
ACM-0001 Satija et al. (2026), Abschn. 3–5 und Anhang B <i>„fastWorkflow: Closing the Performance Gap Between Small and Frontier Language Models for Conversational Agents“</i>	Datenbank E1, zentral	R1, R2, R4	Modulare semantische Laufzeitarchitektur mit Komponentenablationen, Mehrfachläufen, Stabilitäts- und Kostenanalyse auf τ -bench Retail und Airline.

A.4 Eingefrorener Versuchs- und Laufzeitvertrag

Tabelle 28 weist die über alle Bedingungen und Seeds konstant gehaltenen Laufzeitparameter aus. Interventionsbedingte Zusatzaufrufe von Komponente A sind demgegenüber Teil der Intervention und werden nicht aus der Aufwandsmessung entfernt.

Parameter	Festgelegter Wert
Benchmark und Domäne	τ -bench, Tag v1.0.0, Retail
Modell für Agent und Komponente A	openai/gpt-5.4-nano-2026-03-17
User-Simulator	openai/gpt-4.1-2025-04-14
Sampling und Reasoning	Temperatur 0,0; Reasoning Effort <i>medium</i>
Episodengrenzen	maximal 120 Schritte und maximal fünf Laufzeitfehler
Zeitgrenzen	120 Sekunden je API-Aufruf; 900 Sekunden je Episode
Wiederholungsregel	maximal zwei technische Versuche je Zelle; keine verhaltensbedingte Wiederholung
Fachliche Laufzeitinformation	Kein Zugriff auf Task-ID, Goldaktionen, Reward oder erwarteten Zielzustand

Tabelle 28. Konstante Modelle und Laufzeitparameter der finalen Evaluation

A.5 Fairness- und Informationsparitätsprüfung

Tabelle 29 fasst die Prüfung der internen Vergleichbarkeit zusammen. Gleiche Ausgangsbedingungen bedeuten nicht gleichen Rechenaufwand: Zusätzliche Aufrufe und Prüfschritte sind Bestandteil der jeweiligen Intervention und gehen in die Aufwandsmessung ein.

Prüfgegenstand	Umsetzung	Ergebnis
Aufgaben und Seeds	Identische Task-Seed-Blöcke für C0 bis C3.	erfüllt
Agent und Umgebung	Gleiches Agentenmodell, gleicher User-Simulator, gleiche Policy, Werkzeuge und Laufzeitgrenzen.	erfüllt
Referenzzugriff	Kein Laufzeitzugriff auf Task-ID, Referenzaktionen, Reward oder erwarteten Zielzustand.	erfüllt
Nativer Referenzpfad	Originaler Systemprompt; die Komponenten A und B sind in C0 deaktiviert.	erfüllt
Faktortrennung	Die Bedingungsdateien variieren ausschließlich die Faktoren A und B.	erfüllt
Ausführungsfolge	Randomisierte, vollständige Viererblöcke je Aufgabe und Seed.	erfüllt
Zusatzaufwand	Interventionsbedingte Modellaufrufe, Informationsabrufe und Neuplanungen werden über Kosten und Laufzeit erfasst.	erfasst

Tabelle 29. Fairness- und Informationsparitätsprüfung der vier Bedingungen

Die Prüfung belegt interne Vergleichbarkeit und Informationsparität, nicht die Gleichheit des Rechenaufwands oder externe Validität.

A.6 Ergänzende Stabilitätswerte

Die ersten drei Tabellen dokumentieren seedweise Kontraste, die Verteilung der Aufgabenerfolge und die aufgabenbezogene Ergebnisrichtung. Das Ergebniskapitel berichtet daraus nur die für die Argumentation zentralen Muster.

Kontrast	976301	976302	976303	976304	976305	976306
C1–C0 DB	+8,8	+8,8	+2,5	+11,3	+11,3	–6,3
C2–C0 DB	+1,3	+1,3	+6,3	+3,8	+3,8	–6,3
C3–C0 DB	+10,0	+2,5	+3,8	+6,3	+8,8	0,0

Tabelle 30. Seedweise DB-Match-Differenzen gegenüber C0 in Prozentpunkten

Bedingung	0/6	1/6	2/6	3/6	4/6	5/6	6/6
C0 Native	1	5	8	11	14	14	27
C1 Semantische Unterstützung	2	2	3	10	10	25	28
C2 Pre-Write-Kontrolle	1	6	5	13	10	15	30
C3 Kombination	1	4	6	6	14	17	32

Tabelle 31. DB Match: Anzahl der Aufgaben nach erfolgreichen Wiederholungen innerhalb der sechs Seeds

Bedingung	DB besser	DB gleich	DB schlechter	Exact besser	Exact gleich	Exact schlechter
C1 Semantische Unterstützung	29	36	15	27	31	14
C2 Pre-Write-Kontrolle	24	39	17	24	33	15
C3 Kombination	30	32	18	29	27	16

Tabelle 32. Aufgaben mit mehr, gleich vielen oder weniger erfolgreichen Seeds als unter C0

Im Haupttext werden für die Stabilitätskurven die Punkte $k = 1$, $k = 3$ und $k = 6$ berichtet. Die Tabellen 33 und 34 ergänzen die vollständigen Verläufe für alle k von eins bis sechs. pass^k verlangt Erfolg in allen k gezogenen Wiederholungen; $\text{pass}@k$ mindestens einen Erfolg. Die Werte sind deskriptive Stabilitätsmaße und keine zusätzlichen unabhängigen Hypothesentests.

Bedingung	pass ^k						pass@k					
	1	2	3	4	5	6	1	2	3	4	5	6
C0	71,3	55,8	46,7	40,8	36,7	33,8	71,3	86,7	92,9	96,0	97,7	98,8
C1	77,3	63,6	53,8	46,3	40,2	35,0	77,3	91,0	94,9	96,4	97,1	97,5
C2	72,9	58,7	50,2	44,6	40,6	37,5	72,9	87,2	92,9	95,8	97,5	98,8
C3	76,5	63,2	54,5	48,3	43,5	40,0	76,5	89,8	94,4	96,6	97,9	98,8

Tabelle 33. DB Match: vollständige pass^k- und pass@k-Kurven in Prozent

Bedingung	pass ^k						pass@k					
	1	2	3	4	5	6	1	2	3	4	5	6
C0	67,8	51,9	42,1	35,6	31,0	27,8	67,8	83,8	90,0	93,0	94,7	95,8
C1	74,1	60,0	49,7	41,6	35,0	29,2	74,1	88,1	91,9	93,3	94,0	94,4
C2	70,6	55,4	46,0	39,8	35,4	31,9	70,6	85,8	91,7	94,5	96,1	97,2
C3	73,4	59,7	50,8	44,4	39,6	36,1	73,4	87,0	91,7	93,9	95,1	95,8

Tabelle 34. Exact Mutation: vollständige pass^k- und pass@k-Kurven in Prozent

Exact Mutation als Robustheitsanalyse der Seedverteilung

Die Hauptdarstellung der Seedvariation verwendet DB Match als Leitendpunkt. Die folgenden Werte dokumentieren ergänzend die vollständige Verteilungsdiagnostik für Exact Mutation auf den 72 write-positiven Aufgaben. Die qualitative Ordnung der Mittelwerte bleibt erhalten; zugleich überlappen auch hier die beobachteten Wertebereiche.

Panel A: beobachtete Seedverteilung			
Bedingung	Mittelwert	SD	Min–Max
C0 Native	67,8	3,1	63,9–72,2
C1 Semantische Unterstützung	74,1	4,2	66,7–77,8
C2 Pre-Write-Kontrolle	70,6	4,1	65,3–77,8
C3 Kombination	73,4	2,4	70,8–77,8
Panel B: gepaarte Differenzen gegenüber C0			
Kontrast	Mittelwert	Positive Seeds	Min–Max
C1–C0	+6,3	5 von 6	–5,6 bis +11,1
C2–C0	+2,8	5 von 6	–6,9 bis +8,3
C3–C0	+5,6	6 von 6	+1,4 bis +12,5

Tabelle 35. Exact Mutation: Seedverteilung und gepaarte Differenzen gegenüber C0 in Prozentpunkten

A.7 Implementations- und Prüfausschnitte

Die folgenden sechs gekürzten Originalausschnitte decken Konfiguration, Komponentenverträge, Laufzeitentscheidung und Beobachtbarkeit ab. Sie ersetzen keine Quellcodedokumentation, machen aber prüfbar, dass die im Text beschriebenen Vergleichbarkeits- und Kontrollgrenzen ausführbar implementiert sind.

Der Konfigurationsloader bindet jede Bedingung an dieselbe gehashte Basiskonfiguration. Bedingungsmanifeste dürfen nur die festgelegte Faktorbelegung enthalten. Darüber hinaus erzwingt der Loader, dass B ohne LLM arbeitet und weder semantische Zieldatensätze noch Unterstützungskarten verarbeitet; Task-ID und Referenzlösung bleiben zur Laufzeit gesperrt.

```

111 if manifest.get("base_config") != BASE_CONFIG_REL:
112     raise ValueError("condition manifest references an unexpected base config")
113 if manifest.get("base_config_sha256") != base_hash:
114     raise ValueError("condition manifest base hash is stale")
115 if manifest.get("condition") != condition:
116     raise ValueError("condition manifest ID mismatch")
117 expected = base["factor_contract"]["conditions"][condition]
118 if manifest.get("factors") != expected:
119     raise ValueError("condition factor assignment differs from the frozen base")
120 if set(manifest) != {
121     "schema_version",
122     "base_config",
123     "base_config_sha256",
124     "condition",
125     "factors",
126 }:
127     raise ValueError("condition manifests may contain no runtime overrides")
128
129 module_a = base["module_a"]
130 module_b = base["module_b"]
131 shared = base["shared_runtime"]
132 if module_b.get("implementation") != "DETERMINISTIC_NO_LLM":
133     raise ValueError("module B must remain deterministic and LLM-free")
134 if module_b.get("goal_record_input_permitted") is not False:
135     raise ValueError("module B cannot consume semantic goal records")
136 if module_b.get("support_card_input_permitted") is not False:
137     raise ValueError("module B cannot consume semantic support cards")
138 if shared.get("runtime_ground_truth_access") is not False:
139     raise ValueError("runtime ground-truth access must remain disabled")
140 if shared.get("runtime_task_id_access") is not False:
141     raise ValueError("runtime task-ID access must remain disabled")

```

Listing 4. Konfigurations-Freeze und Informationsgrenzen

Komponente A akzeptiert Felder nur, wenn deren Quellen sichtbar sind, aus Nutzerereignissen stammen und den übernommenen Wert tatsächlich enthalten. Nicht belegte Felder werden verworfen, statt als Nutzerziel materialisiert zu werden.

```

282 if not source_ids or any(
283     item not in catalog for item in source_ids
284 ):
285     raise ValueError(
286         f"field {key!r} cites non-visible evidence"
287     )
288 if any(item not in user_source_ids for item in source_ids):
289     raise ValueError(
290         f"field {key!r} is not grounded in user authority"
291     )
292 evidence = [
293     value
294     for source_id in source_ids
295     for value in catalog[source_id]
296 ]
297 value = raw.get("value")
298 if not _entailed(value, evidence):
299     raise ValueError(
300         f"field {key!r} value is not entailed by cited evidence"
301     )

```

Listing 5. Gekürzte Quellen- und Autoritätsprüfung von Komponente A

Komponente B besitzt genau drei Dispositionen. Der Datenvertrag erzwingt, dass nur EVIDENCE_REQUIRED genau eine lesende Evidenzaktion tragen darf. Die Kontrolle kann daher einen Kandidaten unverändert freigeben, Evidenz anfordern oder ihn als ungültig markieren, aber keine fachliche Zielaktion selbst erzeugen.

```

class FeasibilityDisposition(StrEnum):
    VALID_UNCHANGED = "VALID_UNCHANGED"
    EVIDENCE_REQUIRED = "EVIDENCE_REQUIRED"
    INVALID = "INVALID"

@dataclass(frozen=True, slots=True)
class FeasibilityEvaluation:
    call_id: str
    action: ToolAction
    disposition: FeasibilityDisposition
    reason_code: str
    evidence_action: ToolAction | None = None

    def __post_init__(self) -> None:
        needs_evidence = (
            self.disposition == FeasibilityDisposition.EVIDENCE_REQUIRED
        )
        if needs_evidence != (self.evidence_action is not None):
            raise ValueError(
                "only EVIDENCE_REQUIRED has exactly one evidence action"
            )

```

Listing 6. Dispositionen und Evidenzinvariante von Komponente B

Die konkrete Auswertungsfunktion übersetzt die deterministische B-Prüfung in genau diese drei Dispositionen. Auch eine native Bestätigungsanforderung verbleibt im nativen Dialogpfad; nur fehlende sichtbare Evidenz erzeugt eine konkrete Leseaktion.

```

81 decision = DeterministicPreWriteControl.evaluate(state, action)
82 if decision.disposition in {
83     PreWriteDisposition.ALLOW_UNCHANGED,
84     PreWriteDisposition.REQUEST_CONFIRMATION,
85 }:
86     return FeasibilityEvaluation(
87         identifier,
88         action,
89         FeasibilityDisposition.VALID_UNCHANGED,
90         "B_FEASIBILITY_VALID_CONFIRMATION_NATIVE_SCOPE",
91     )
92 if decision.disposition == PreWriteDisposition.REQUEST_EVIDENCE:
93     return FeasibilityEvaluation(
94         identifier,
95         action,
96         FeasibilityDisposition.EVIDENCE_REQUIRED,
97         decision.reason_code,
98         evidence_action=decision.evidence_action,
99     )
100 return FeasibilityEvaluation(
101     identifier,
102     action,
103     FeasibilityDisposition.INVALID,
104     decision.reason_code,
105 )

```

Listing 7. Deterministische Laufzeitentscheidung von Komponente B

Schließlich serialisiert der Adapter Faktorbelegung, Ereigniszähler, Komponentenkosten, Laufzeiten, das vollständige Ereignisledger und beide Integritätsaudits gemeinsam. Damit stammen Ergebnis-, Mechanismus- und Aufwandsanalyse aus derselben zellbezogenen Laufzeitspur.

```

260 def factorial_runtime_record(state: FactorialRuntimeState) -> dict[str, Any]:
261     config = state._runtime_config
262     events = state.intervention.ledger.serialise()
263     return {
264         "schema_version": state.artifact_schema_version,
265         "domain": "retail",
266         "condition": state.condition,
267         "factors": {
268             "semantic_support": config.factors.semantic_support,
269             "prewrite_control": config.factors.prewrite_control,
270         },
271         "counts": {
272             "a_opportunities": state.intervention.a_opportunities,
273             "a_extractions": state.intervention.a_extractions,
274             "a_support_cards": state.intervention.a_support_cards,
275             "a_commitments": len(state.commitments),
276             "b_opportunities": state.intervention.b_opportunities,
277             "b_evidence_reads": state.intervention.b_evidence_reads,
278             "b_replans": sum(
279                 row["component"] == "B"
280                 and row["reason_code"] == "B_SINGLE_REPLAN"
281                 for row in events
282             ),
283         },
284         "cost_usd": {
285             "native_agent": state.native_cost_usd,
286             "semantic_support": state.semantic_cost_usd,
287             "prewrite_validation": 0.0,
288         },
289         "seconds": {
290             "native_agent": state.native_seconds,
291             "semantic_support": state.semantic_seconds,
292             "prewrite_validation": state.prewrite_seconds,
293         },
294         "events": events,
295         "observer_audit": state.intervention.ledger.audit(),
296         "semantic_linkage_audit": _card_trace_audit(events),
297     }

```

Listing 8. Serialisierung des faktoriellen Laufzeitdatensatzes

Der gepinnte Ausführungspfad konstruiert für jede Zelle neue Instanzen von Umgebung, Agent, User-Simulator und Orchestrator. Der Wiederholungs-Seed wird anschließend an beide Dialogseiten weitergegeben. Modellaufrufe übertragen die aktuelle Nachrichtenhistorie direkt an die Completion-Schnittstelle; eine fallübergreifende Conversation-, Thread- oder Previous-Response-Referenz ist nicht Bestandteil des Request-Pfads. Der Ausschnitt dokumentiert damit Zustandsisolation und Seedweitergabe, nicht die Abwesenheit providerseitiger technischer Logs.

```

environment = build_environment(
    domain, solo_mode=solo_mode, env_kwargs=env_kwargs
)
agent = build_agent(
    config.effective_agent, environment,
    llm=config.llm_agent, llm_args=config.llm_args_agent,
    task=task, solo_mode=solo_mode,
)
user = build_user(
    config.effective_user, environment, task,
    llm=config.llm_user, llm_args=config.llm_args_user,
    solo_mode=solo_mode,
)
orchestrator = Orchestrator(
    domain=domain, agent=agent, user=user,
    environment=environment, task=task, seed=seed,
)

if self.seed is not None:
    self.agent.set_seed(self.seed)
    self.user.set_seed(self.seed)

def set_seed(self, seed: int) -> None:
    self.llm_args["seed"] = seed

litellm_messages = to_litellm_messages(messages)
response = completion(
    model=model, messages=litellm_messages,
    tools=tools_schema, tool_choice=tool_choice, **kwargs,
)

```

Listing 9. Gekürzter Zell-, Seed- und Request-Isolationpfad

Die thesis-relevanten Implementationen der faktoriellen Verträge und der Retail-Anbindung sind im Codekern dokumentiert. Das digitale Materialverzeichnis in Tabelle 39 weist die zugehörigen Repository-Pfade aus. Die Ausschnitte dienen der Architekturprüfung; die Wirksamkeit wird ausschließlich durch die in Kapitel 7 berichteten Endpunkte beurteilt.

Verifikationsmatrix. Tabelle 36 schließt die Nachweiskette zwischen den neun Gestaltungsanforderungen, ihren technischen Akzeptanzkriterien und den zugehörigen Evidenzartefakten. Ein bestandener technischer Nachweis bestätigt die spezifikationsgemäße Implementierung, nicht die empirische Vorteilhaftigkeit der betreffenden Komponente.

DR	Akzeptanzkriterium	Ausführbare Prüfung	Status und Evidenz
DR1	Ergebnis und Prozess bleiben zellbezogen verknüpfbar.	Serialisierung von Faktorbelegung, Eingriffsgelegenheiten, Entscheidungen, Folgeaktionen, Kosten und finaler Disposition.	bestanden; Listing 8, Tabelle 37
DR2	Komponente A verwendet ausschließlich sichtbare, nutzerautorisierte Quellen.	Quellen-, Wert- und Informationsparitätsprüfung gegen den sichtbaren Ereigniskatalog.	bestanden; Listing 5, Tabelle 29
DR3	Komponente B bleibt kandidatengebunden, deterministisch und LLM-frei.	Vertragsprüfung der drei Dispositionen, fehlender Eingaben von Komponente A und unveränderter Freigabe.	bestanden; Listings 4, 6 und 7
DR4	Beitrag und Aussage bleiben auf die untersuchten Laufzeitinterventionen begrenzt.	Dokumentprüfung von Artefaktdefinition, Interventionsgrenze und Trennung der nachgelagerten Entscheidungsauswertung.	dokumentiert; Kapitel 6 und Analysevertrag
DR5	Die Bedingungen unterscheiden sich nur in zwei Faktorflags.	Hash-, Schema- und Manifestprüfung gegen eine gemeinsame Basiskonfiguration.	bestanden; Listing 4, Tabelle 7
DR6	Zurückhaltung führt höchstens zu einem begrenzten nativen Folgepfad.	Prüfung von Evidenzabruf, Neuplanungsbudget, Kandidatenidentität und atomarer Bündelbehandlung.	bestanden; Listing 7, Tabelle 13
DR7	Ergebnis, Regression und Aufwand stammen aus derselben Zellspur.	Gemeinsame Serialisierung und nachgelagerte Aggregation von Endpunkt-, Kosten-, Zeit- und Mechanismusdaten.	bestanden; Listing 8, digitales Analysematerial
DR8	Referenzlösung und primäre Bewertung bleiben außerhalb der Laufzeit.	Import-, Manifest- und Laufzeitprüfung gesperrter Aufgaben-, Referenz- und Bewertungsobjekte.	bestanden; Listing 4, Tabelle 29
DR9	Beobachtung und natürliche Eingriffsgelegenheiten werden bedingungsübergreifend gleich erfasst.	Prüfung des passiven Beobachters, gemeinsamer Schemata für Eingriffsgelegenheiten und vollständiger Aktivierungsketten.	bestanden; Tabellen 29 und 37

Tabelle 36. Verifikationsmatrix der Gestaltungsanforderungen und ihrer Akzeptanznachweise

A.8 Mechanismusprüfung und qualitative Ereignisspurvignetten

Die automatische Auditspur umfasst sämtliche 1.920 Zellen. Sie prüft strukturelle Integrität und verbindet Komponentenereignisse mit den Offline-Endpunkten. Tabelle 37 zeigt den Zensus der zentralen Ereignisse. Automatische Prüfwerte belegen dabei weder semantische Richtigkeit noch individuelle Kausalität.

Prüfgröße	Wert	Bedeutung
Task-Seed-Bedingungszellen	1.920	vollständige Ergebnismatrix
Vollständige Task-Seed-Blöcke	480	je vier Bedingungen
Materialisierte Unterstützungskarten	2.390	C1 und C3 gemeinsam
Unterstützungskarten mit spätem Schreibkandidaten	1.079	sichtbare Nähe zum Handlungspfad
Von B geprüfte Schreibkandidaten	1.377	C2 und C3 gemeinsam
Oracle-kompatible Kandidaten	1.178	nachgelagerter diagnostischer Abgleich
Oracle-inkompatible Kandidaten	199	nachgelagerter diagnostischer Abgleich
Automatische A-Quellenreferenzfehler	0	strukturelle Prüfung der Ereignisspur
Automatische B-Kandidatenbindungs- und Konsistenzfehler	0	strukturelle Prüfung der Ereignisspur

Tabelle 37. Zentrale Mengen- und Integritätswerte des Mechanismusaudits

Für die qualitative Vertiefung wurden sechs kontrastierende, informationsreiche Fälle zweckgerichtet ausgewählt: je ein geretteter und beschädigter Übergang (*Saved/Broken*) für Komponente A, Komponente B und C3. Jeder Fall wurde entlang derselben Kette gelesen: sichtbares Nutzerziel, Komponentendatensatz, verknüpfter Schreibkandidat, gegebenenfalls Disposition von Komponente B, ausgeführte Mutation, Offline-Übergang und alternative Erklärung. Die Auswahl erklärt Mechanismen und Grenzen; sie schätzt keine Häufigkeiten oder Mediationseffekte.

Tabelle 38. Einheitlich codierte Übersicht der sechs qualitativen Ereignisspurvignetten

Komponente und Fall	Zelle	Beobachtete Evidenzkette	Interpretationsgrenze
Komponente A: Saved	Task 5, Seed 976302, C1	Späte Revision von Austausch zu Rückgabe; Karte AC-0001 hält die Revision fest; verknüpfter Rückgabekandidat; C0 Exact Fail zu C1 Exact Pass.	Konsistent mit dem Bewahren einer späten Revision, aber kein isolierter Mediationseffekt.
Komponente A: Broken	Task 3, Seed 976304, C1	Mehrere zeitlich verteilte Bestellziele; Karten AC-0002/AC-0003; zusätzliche Mutation; C0 Exact Pass zu C1 Exact Fail.	Quellenbindung löst zeitliche Konflikte nicht automatisch; Komponente A ist nicht als alleinige Ursache identifiziert.
Komponente B: Saved	Task 0, Seed 976301, C2	Fehlendes Zahlungsprofil; EVIDENCE_REQUIRED; Profilabruf; unveränderte Freigabe; C0 Exact Fail zu C2 Exact Pass.	Belegt Evidenzvollständigkeit, nicht eine semantische Korrektur durch Komponente B.
Komponente B: Broken	Task 0, Seed 976303, C2	Gleicher Kontrollpfad; unveränderte Freigabe eines Kandidaten mit falschem Zielparameter; C0 Exact Pass zu C2 Exact Fail.	Zeigt die Informations- und Autoritätsgrenze der implementierten Kontrolle.
C3: Saved	Task 21, Seed 976304, C3	Unterstützungskarte für zwei Artikel und Zahlungsmittel; Komponente B fordert eine Produktabfrage; unveränderte Freigabe; C0 Exact Fail zu C3 Exact Pass.	Mit komplementärem Pfad vereinbar, aber Komponentenbeiträge bleiben ungetrennt.

Komponente und Fall	Zelle	Beobachtete Evidenzkette	Interpretationsgrenze
C3: Broken	Task 1, Seed 976301, C3	Komponente A verknüpft Austauschziele; Komponente B prüft nur das Zahlungsprofil; falscher Zielartikel und Extra Effect; C0 Exact Pass zu C3 Exact Fail.	Gemeinsame Aktivierung garantiert keine Zielkorrektheit und belegt für sich keine Interaktion.

A.9 Verzeichnis des ergänzenden digitalen Materials

Das ergänzende digitale Material hält die umfangreichen, für den Fließtext ungeeigneten Nachweise in ihren nativen Formaten vor. Die Formatkürzel Markdown (MD), Comma-Separated Values (CSV) und JavaScript Object Notation (JSON) bezeichnen die im Materialverzeichnis verwendeten Text- und Datenaustauschformate. Tabelle 39 ist zugleich der Wegweiser von den Claims der Arbeit zu den zugrunde liegenden Daten und Programmen. Die Pfade sind relativ zur Wurzel des abgegebenen Forschungsrepositorys angegeben.

Tabelle 39. Verzeichnis des ergänzenden digitalen Materials

Pfad	Format	Funktion
digital_research_appendix/literature_review_final/	MD, CSV, JSON	Suchrahmen, finale Auswahlzahlen, Studienmatrix, Qualitätsbewertung und Synthesenachweise der Literaturstudie.
code_core/artifact/factorial/ und code_core/artifact/retail/	Python	Thesis-relevante Implementationen, Verträge, Orchestrierung und Telemetrie der Komponenten A und B.
code_core/tests/	Python	Ausgewählte Unit-, Vertrags-, Faktor-Isolations- und Entscheidungstests.
code_core/config/final_design.json	JSON	Faktorbelegung und zentrale Verträge der vier Evaluationsbedingungen.

Pfad	Format	Funktion
digital_research_ appendix/results_ aggregated/six_seed_ distribution/six_seed_ distribution_analysis. json	JSON	Aggregierte Sechs-Seed-Auswertung einschließlich Verteilungsdiagnostik.
digital_research_ appendix/results_ aggregated/six_seed_ mechanisms/00_AUDIT_ SUMMARY.json	JSON	Aggregierter Zensus und auto- matische Integritätsprüfungen des Mechanismusaudits.
digital_research_ appendix/results_ aggregated/six_ seed_mechanisms/05_ QUALITATIVE_TRACE_ VIGNETTES.md	MD	Strukturierte Beschreibung der sechs ausgewählten Ereignisspur- vignetten.

A.10 Endpunkt-, Analyse- und Anspruchsgrenzen

DB Match wird für alle 80 Evaluationsaufgaben ausgewiesen. Exact Mutation wird für die 72 Aufgaben mit eindeutig definiertem Mutationsoracle analysiert. Berichtet werden direkte Bedingungskontraste, die beiden faktoriellen Haupteffekte und die $A \times B$ -Interaktion. Seeds werden nicht als unabhängige Aufgaben behandelt. Bootstrapziehungen mit Clustering nach Aufgaben und über beide Dimensionen quantifizieren die Unsicherheit; absolute Zähler, Nenner und Prozentpunktdifferenzen bleiben die zentralen Berichtseinheiten.

Die Mechanismusgruppen trennen tatsächliche A-Card-Exposure und B-Dispositionen und verbinden diese mit Saved, Broken, Stable Pass und Stable Fail. Weil Exposure vom Dialogpfad abhängt, dienen diese Strata der Prüfung von Wirkpfadkonsistenz und nicht der Schätzung eines kausalen Mediationeffekts.

1. DB Match misst den kodierten Geschäftsendzustand; Exact Mutation die benchmarkdefinierte Mutationsausführung. Beide ersetzen keine reale Servicequalitätsmessung.
2. Eine Komponentenaktivierung belegt allein keinen Nutzen. Erst ihre Verknüpfung mit Endergebnis und Korrekturpfad erlaubt eine begrenzte Wirkpfadanalyse.

3. Saved und Broken sind gepaarte Ergebniswechsel, keine direkt beobachteten individuellen Kausalfakten.
4. Mechanismusgruppen entstehen nach Aktivierung der Bedingungen; sie identifizieren keine kausale Mediation.
5. Kosten, Laufzeit, Nachrichten und Tool-Aufrufe sind technische Betriebsproxies und kein Nachweis produktiver Wirtschaftlichkeit.
6. Die 80 Retail-Aufgaben bestimmen die Schätzpräzision. Ein den Nullwert umfassendes Intervall ist kein Äquivalenznachweis.
7. Aussagen gelten für den untersuchten Artefaktstand, das gepinnte Nano-Agentenmodell und die Retail-Domäne. Weitere Modelle, Domänen und reale Organisationen erfordern Replikationen.
8. Entwicklungs- und Bewertungsdaten werden nicht zu einer größeren Wirksamkeitsstichprobe zusammengeführt.

Dokumentation KI-gestützter Hilfsmittel

Im Entstehungsprozess dieser Arbeit wurden die nachfolgend aufgeführten KI-gestützten Hilfsmittel verwendet. Die Unterstützung durch OpenAI Codex war auf Code-Unterstützung sowie sprachliches und strukturelles Sparring begrenzt. Die wissenschaftliche Konzeption, die Auswahl und Auswertung der Quellen, die methodischen Entscheidungen, die Interpretation der Ergebnisse sowie die finale inhaltliche Verantwortung verblieben beim Verfasser. Konzepte wurden nicht ungeprüft aus KI-Ausgaben übernommen. KI-Ausgaben wurden nicht als wissenschaftliche Quellen verwendet.

Hilfsmittel	Art der Nutzung
OpenAI Codex	Code-Unterstützung sowie sprachliche und redaktionelle Überarbeitung, Formulierungsvorschläge, $\text{\LaTeX}$ -Formatierung sowie technische Konsistenz- und Darstellungsprüfungen. Vorschläge wurden geprüft und bei Bedarf verworfen.
OpenAI-API	Ausführung der in Kapitel 4 beschriebenen experimentellen Agenten- und Nutzersimulationsläufe. Modelle, Parameter und Kontrollbedingungen sind dort sowie im Anhang dokumentiert. Die Modellaufrufe bilden einen Untersuchungsbestandteil und wurden nicht als Literaturquelle verwendet.

Die inhaltliche Herleitung, die Prüfung der Originalquellen und sämtliche in der Arbeit vertretenen Aussagen wurden vom Verfasser verantwortet. Gleiches gilt für Forschungsdesign, Artefaktgestaltung, statistische Auswertung und Schlussfolgerungen.



Eigenständigkeitserklärung

Hiermit versichere ich,

von Angern, Moritz
Name, Vorname: _____

dass ich die vorliegende Arbeit

Gestaltung und faktorielle Evaluation modularer Laufzeitinterventionen für transaktionale LLM-Agenten im Kundenservice
Titel der Arbeit: _____

Consulting & Business Analytics
Studienfach: _____

Prof. Dr. Tim Straub
Prüfer*in: _____

selbst verfasst, alle wörtlichen oder sinngemäß übernommenen Stellen in der Arbeit gekennzeichnet und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

In Bezug auf KI-Tools (bitte ankreuzen):

☐ Option 1: Keine Nutzung von KI-Tools

Ich versichere außerdem, keine KI-basierten, inhaltsgenerierenden Hilfsmittel verwendet zu haben.

☒ Option 2: Kennzeichnungspflicht von KI-Tools bei erlaubter Nutzung

Ich versichere außerdem, dass die Nutzung von KI-Tools von der/m Prüfenden explizit erlaubt wurde und ich in meiner Arbeit davon Gebrauch gemacht habe. Die verwendeten KI-Tools sind der Arbeit nach fakultätsinternen Zitier- und Dokumentationsvorgaben angefügt. Ich habe inhaltsgenerierende KI-Tools lediglich unterstützend eingesetzt und die Kernthesen meiner Arbeit nicht von der KI übernehmen lassen. Ich bin mir bewusst, dass die Nutzung von KI-Tools keine Garantie für Richtigkeit bietet und ich die Verantwortung für alle KI-generierten Inhalte in dieser Arbeit trage.

☐ Option 3: Erlaubnis inhaltsgenerierender KI-Tools ohne Kennzeichnungspflicht

Ich versichere außerdem, dass ich inhaltsgenerierende KI-Tools lediglich unterstützend eingesetzt und die Kernthesen meiner Arbeit nicht von der KI habe übernehmen lassen. Ich bin mir bewusst, dass die Nutzung von KI-Tools keine Garantie für Richtigkeit bietet und ich die Verantwortung für alle KI-generierten Inhalte in dieser Arbeit trage.

Reutlingen, 27.08.2026, Moritz von Angern

Ort, Datum, Name Verfasser*in